Royaume du Maroc
Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique
et d'Analyse des Systèmes

Support de cours

Ingénierie Dirigée par les Modèles

IDM · Méta-modélisation (EMF/Ecore) · OCL · MDA
Génération de code (Acceleo, Java/EMF) · Transformation de modèles (ATL)

Pr. Aissam BERRAHOU
Département Ingénierie des Systèmes Embarqués
aissam.berrahou@ensias.um5.ac.ma

Année universitaire 2026–2027

Avant-propos

Ce support accompagne le module *Ingénierie Dirigée par les Modèles* (IDM, en anglais *Model-Driven Engineering*, MDE). Il rassemble, sous forme rédigée, le contenu des cinq séances de cours, complété par deux chapitres dédiés à la méta-modélisation avec EMF/Ecore et au langage de contraintes OCL, travaillés en TD et en TP.

Organisation. Chaque chapitre suit la même structure :

- un **résumé**, des **objectifs** d'apprentissage et la **table des matières** du chapitre ;
- le cours, où chaque notion est illustrée par **plusieurs exemples** numérotés (trois à quatre lorsque c'est possible) et par des **figures** ;
- des remarques *À retenir* et *Attention* pour les points clés et les pièges fréquents ;
- une série d'**exercices**, une **synthèse** et un **quiz**.

Les corrigés commentés de tous les exercices et de tous les quiz sont regroupés en annexe A. L'annexe B propose un aide-mémoire des syntaxes (OCL, Acceleo, API EMF, ATL) et un glossaire.

Domaines d'illustration. Les exemples sont tirés de quelques domaines familiers, que l'on retrouve d'un chapitre à l'autre :

- le **feu de signalisation** (structure et machine à états) ;
- la **bibliothèque** (`Library` : `Book`, `Member`, `Loan`), suivie du besoin (CIM) jusqu'au code Java, Python et SQL ;
- la **boutique en ligne** (`Client`, `Commande`, `Produit`) ;
- l'**entreprise** (départements et employés) pour OCL, et les **familles** (`Families2Persons`) pour ATL.

Prérequis. Conception orientée objet et UML 2 (diagrammes de classes, d'objets, de cas d'utilisation, d'états-transitions) ; programmation orientée objet en Java ; notions de bases de données relationnelles et de XML.

Outils. JDK 17 ou plus, *Eclipse Modeling Tools* (EMF, Ecore Tools), Obeo Designer / Sirius, Eclipse OCL (OCLinEcore), Acceleo 3.7, ATL.

Conventions. Le code source est présenté dans des zones grises ; les termes techniques anglais sont en *italique* ; les noms d'éléments de modèles et de code sont en `chasse fixe`.

Table des matières

Avant-propos **2**

Table des figures **6**

Liste des tableaux **9**

1 Introduction à l'Ingénierie Dirigée par les Modèles **1**

1.1 Pourquoi modéliser ? . . . 2
1.2 Modèle et IDM . . . 4
1.3 L'architecture à quatre niveaux de l'OMG . . . 6
1.4 Études de cas corrigées . . . 12
1.5 Exemples complémentaires . . . 13
1.6 Exercices . . . 14
1.7 Synthèse . . . 15
1.8 Quiz . . . 15

2 Méta-modélisation avec EMF et Ecore **18**

2.1 EMF et Ecore . . . 19
2.2 Les choix de modélisation en détail . . . 24
2.3 Exemples complets . . . 26
2.4 Exercices . . . 28
2.5 Synthèse . . . 28
2.6 Quiz . . . 29

3 Le langage de contraintes OCL **31**

3.1 Pourquoi OCL ? . . . 32
3.2 Le modèle d'étude . . . 33
3.3 Contexte et `self` . . . 35
3.4 Accéder aux éléments du modèle . . . 36
3.5 Les types de base et les expressions . . . 42
3.6 Les collections . . . 42
3.7 Spécifier des opérations : pré- et post-conditions . . . 45
3.8 Définitions : `def`, `body`, `derive`, `init` . . . 46
3.9 OCL au niveau méta et dans les outils . . . 46

3.10 Méthode et pièges fréquents . . . 47
3.11 Exercices . . . 48
3.12 Synthèse . . . 49
3.13 Quiz . . . 49

4 L'approche MDA (Model Driven Architecture) 51
4.1 Pourquoi MDA ? . . . 52
4.2 Les modèles de MDA . . . 54
4.3 Les transformations de modèles . . . 57
4.4 Les approches de transformation . . . 60
4.5 Études de cas corrigées . . . 65
4.6 Exemples complémentaires . . . 66
4.7 Exercices . . . 67
4.8 Quiz . . . 68

5 Transformation M2T par template avec Acceleo 70
5.1 Pourquoi générer du code ? . . . 71
5.2 Premier projet Acceleo . . . 72
5.3 Le langage pas à pas . . . 74
5.4 Cas complet : `library.ecore` vers Python, SQL et Java . . . 79
5.5 Bonnes pratiques et erreurs fréquentes . . . 83
5.6 Exemples complémentaires . . . 83
5.7 Exercices . . . 85
5.8 Quiz . . . 86

6 Transformation par programmation (Java et API EMF) 88
6.1 Pourquoi programmer une transformation ? . . . 89
6.2 Lire un modèle avec EMF . . . 91
6.3 Le générateur pas à pas . . . 93
6.4 Cas complet : Python, SQL et Java . . . 95
6.5 Bonnes pratiques et synthèse . . . 100
6.6 Exemples complémentaires . . . 100
6.7 Exercices . . . 102
6.8 Quiz . . . 103

7 Transformation M2M avec ATL 105
7.1 Pourquoi une transformation M2M ? . . . 106
7.2 Le langage ATL . . . 107
7.3 Règles et helpers . . . 108
7.4 Cas complet : Families2Persons dans Eclipse . . . 112

7.5 Aller plus loin . . . 113
7.6 Exemples complémentaires . . . 118
7.7 Exercices . . . 119
7.8 Quiz . . . 119

A Corrigés des exercices et des quiz **122**
A.1 Chapitre 1 – Introduction à l'IDM . . . 122
A.2 Chapitre 2 – Méta-modélisation avec EMF et Ecore . . . 123
A.3 Chapitre 3 – Le langage OCL . . . 124
A.4 Chapitre 4 – MDA . . . 126
A.5 Chapitre 5 – Acceleo . . . 127
A.6 Chapitre 6 – Transformation par programmation . . . 129
A.7 Chapitre 7 – ATL . . . 131

B Aide-mémoire et glossaire **133**
B.1 OCL . . . 133
B.2 Acceleo 3 . . . 133
B.3 API EMF (Java) . . . 133
B.4 ATL . . . 134
B.5 Glossaire . . . 134

Références bibliographiques **136**

Table des figures

1.1 Coût relatif de correction d'une erreur selon la phase où elle est découverte (ordres de grandeur) . . . 3
1.2 Code d'abord ou modèle d'abord : où se trouve la source de vérité ? . . . 5
1.3 La pyramide de l'OMG : les quatre niveaux de modélisation . . . 6
1.4 Vue structurelle (M1) : diagramme de classes du feu de signalisation . . . 7
1.5 Vue comportementale (M1) : machine à états du feu de signalisation . . . 7
1.6 Méta-modèle simplifié du diagramme de classes (M2) . . . 8
1.7 Conformité : exemples et contre-exemples . . . 9
1.8 La chaîne complète « instance de » du feu de signalisation . . . 10
1.9 Méta-modèle (M2) simplifié d'une base de données relationnelle . . . 12

2.1 Méta-modèle Ecore simplifié : les concepts utiles dans ce cours . . . 19
2.2 Le méta-modèle `library.ecore` . . . 21
2.3 `library.ecore` : méta-modèle pour EMF, modèle pour un générateur . . . 21
2.4 `library.ecore` vu comme un graphe d'objets Ecore (`EPackage`, `EClass`, `EAttribute`, `EReference`) . . . 22

3.1 Modèle d'étude OCL : entreprise, départements et employés . . . 34

4.1 Le métier est stable, la plateforme change : sans MDA, on réécrit le métier à chaque technologie . . . 52
4.2 Le principe de MDA (cycle « en Y ») : le PIM et le PDM se rejoignent dans le PSM 53
4.3 MDA et les niveaux M0 à M3 : CIM, PIM et PSM sont tous des modèles M1 . . 53
4.4 Les modèles de MDA, du plus abstrait au plus concret . . . 54
4.5 CIM de la bibliothèque : cas d'utilisation et exigences SysML numérotées . . . 54
4.6 PIM de la bibliothèque : diagramme de classes et contrainte OCL . . . 55
4.7 PDM : correspondances entre concepts du PIM et des plateformes Java/JPA et SQL 55
4.8 PSM Java/JPA de la bibliothèque . . . 56
4.9 PSM relationnel (SQL) de la bibliothèque . . . 56
4.10 Un seul PIM, plusieurs PSM : le métier n'est pas réécrit . . . 57
4.11 Schéma général d'une transformation : la transformation est elle-même un modèle 58
4.12 Les six transformations de MDA . . . 58
4.13 Raffinement PIM → PIM : réification de l'association en classe Emprunt . . . 59
4.14 Taxonomie : endogène/exogène, horizontale/verticale . . . 60

4.15 Selon la nature de la cible : M2M, M2T, T2M 60
4.16 Approche par template : du texte fixe et des trous remplis par le modèle 61
4.17 Un document XML, deux feuilles XSLT, deux résultats 62
4.18 Approche par programmation : un programme lit et écrit les modèles via des API 62
4.19 Règles R1 à R4 de la transformation PIM UML → relationnel 63
4.20 Approche par modélisation : la transformation est un modèle conforme au méta-modèle d'un langage de transformation 64
4.21 Récapitulatif de la chaîne MDA 66

5.1 Place d'Acceleo dans MDA : la dernière marche, du modèle au texte 72
5.2 Principe de la génération par template : modèle + template + moteur 72
5.3 Structure d'un projet Acceleo 73
5.4 Anatomie d'un module Acceleo (`.mtl`) 73
5.5 Le point suit une flèche du méta-modèle Ecore 74
5.6 Fonctionnement de la boucle `[for]` 75
5.7 La conditionnelle `[if]` appliquée aux attributs 76
5.8 Templates et queries : découper un générateur en morceaux 78
5.9 Un `[file]` dans une boucle : un fichier Java par EClass 78
5.10 Méthode « l'exemple d'abord » pour écrire un template 79
5.11 Traçabilité modèle → code Python : d'où vient chaque ligne ? 80
5.12 Un modèle, plusieurs cibles… et plusieurs modèles (`school.ecore`) 83
5.13 Les erreurs les plus fréquentes en Acceleo 83

6.1 La programmation : une approche générale, utilisable à chaque marche de MDA 89
6.2 Le principe : charger, parcourir, écrire 89
6.3 Programmation ou template : deux écritures du même morceau de générateur . . 90
6.4 Des concepts Ecore aux interfaces et méthodes Java de l'API EMF 91
6.5 Charger le modèle en 4 étapes : du fichier `.ecore` à l'objet `EPackage` 92
6.6 `library.ecore` chargé : un graphe d'objets Java 92
6.7 Méthode « l'exemple d'abord » : écrire le fichier visé, repérer le fixe et le variable 93
6.8 Deux boucles imbriquées : ce qui se passe à l'exécution 94
6.9 Règles de traduction d'une EClass en table SQL 96
6.10 Organisation d'un générateur : une méthode par concept + des méthodes utilitaires 97
6.11 Zones protégées : relire l'ancien fichier pour conserver le code écrit à la main . . 98
6.12 Un modèle, trois cibles… et plusieurs modèles 99
6.13 Synthèse : la transformation par programmation 101

7.1 Trois façons de transformer un modèle : ATL produit un modèle, Acceleo et Java surtout du texte 106
7.2 Place de la transformation M2M dans MDA : du PIM au PSM 106

7.3 Schéma général d'une transformation ATL : six fichiers, trois niveaux 107
7.4 Les méta-modèles Families (source) et Persons (cible) 108
7.5 Structure d'un module ATL : quatre parties, dans cet ordre 108
7.6 Anatomie d'une règle ATL : motif source, filtre, motif cible, liaisons 109
7.7 Un helper : une fonction OCL attachée à un type du méta-modèle 110
7.8 Les trois phases d'exécution du moteur ATL 111
7.9 Exécution pas à pas sur la famille March 112
7.10 Configuration de l'exécution : modèles IN et OUT et leurs méta-modèles 113
7.11 Méta-modèles SimpleClass et Relational 113
7.12 Class2Relational : Package → Schema, Class → Table, Attribute → Column . . 114
7.13 La résolution automatique par la trace 115
7.14 Extension : une règle Family2Household qui utilise la trace 115
7.15 Synthèse : la transformation M2M avec ATL 117

Liste des tableaux

1.1 Classification des modèles 4
1.2 Correspondance MOF – Ecore et exemples 10
1.3 Les quatre niveaux dans différents espaces technologiques 11
1.4 Pièges fréquents sur les niveaux de modélisation 11
1.5 Synthèse du chapitre 1 15

2.1 Principaux concepts d'Ecore 20
2.2 Code généré par EMF pour le méta-modèle `library` 22
2.3 Multiplicités Ecore et leur signification 24
2.4 Synthèse du chapitre 2 29

3.1 Diagramme d'objets d'étude (niveau M0) 34
3.2 Les formes de contexte OCL 35
3.3 Opérations sur les types 41
3.4 Types de base d'OCL et exemples évalués 42
3.5 Types de collections OCL 43
3.6 Itérateurs OCL évalués sur le diagramme d'étude 43
3.7 Pièges fréquents en OCL 48
3.8 Synthèse du chapitre OCL 49

4.1 Ce que le PIM contient et ne contient pas 55
4.2 Synthèse des modèles de MDA 57
4.3 Les trois grandes approches de transformation 61
4.4 Lire un template XSLT 62
4.5 Règles de la transformation PIM → relationnel 63

5.1 Expressions de navigation dans un modèle Ecore 75
5.2 Opérations OCL utiles dans les templates 75
5.3 Correspondance des types Ecore, Java et SQL 83
5.4 Diagnostic des erreurs Acceleo 84

6.1 Template (Acceleo) et programmation (Java + EMF) 90
6.2 Méthodes de l'API EMF et leur valeur sur `library.ecore` 91
6.3 Trois façons d'écrire le texte en Java 95

6.4 Pièges de l'écriture de texte 95
6.5 Erreurs fréquentes avec l'API EMF 100

7.1 Expressions ATL / OCL courantes 109
7.2 Navigation sur la famille March (m1 = Jim, père ; m4 = Brenda, fille) 109
7.3 Helper opération et helper attribut 111
7.4 Quelle règle pour quel membre ? 112
7.5 Étapes du tutoriel Families2Persons 112
7.6 Les sortes de règles ATL 116
7.7 Erreurs fréquentes en ATL 117
7.8 ATL, Acceleo ou programmation ? 117

Introduction à l'Ingénierie Dirigée par les Modèles

Résumé

*Les logiciels actuels sont complexes, doivent évoluer en permanence et tourner sur des plateformes qui changent tous les quelques ans. « Coder directement » ne permet plus de maîtriser cette complexité : la connaissance métier se retrouve dispersée dans le code, les erreurs sont découvertes tard et coûtent cher. L'**Ingénierie Dirigée par les Modèles** (IDM) place le **modèle** au centre du développement : on le construit, on le vérifie, on le **transforme** et on en **génère** tout ou partie du code. Ce chapitre définit les notions de modèle et de méta-modèle, présente l'architecture à **quatre niveaux** de l'OMG (M0 réel, M1 modèle, M2 méta-modèle, M3 méta-méta-modèle MOF/Ecore) et la relation de **conformité** qui relie un modèle à son méta-modèle. Chaque notion est illustrée sur plusieurs domaines : le feu de signalisation, la bibliothèque, la boutique en ligne et le distributeur de billets.*

Objectifs d'apprentissage

À l'issue de ce chapitre, vous serez capable de :

- expliquer pourquoi « coder directement » ne suffit plus pour les systèmes complexes ;
- définir un **modèle**, distinguer modèle descriptif/prescriptif et contemplatif/productif ;
- définir l'**IDM** et en citer les apports et les limites ;
- distinguer les **4 niveaux** de l'OMG : M0, M1, M2, M3 ;
- vérifier qu'un modèle est **conforme** à son méta-modèle ;
- situer **Ecore** par rapport au MOF et lire un fichier **XMI**.

Contenu du chapitre

1.1 Pourquoi modéliser ? . . . 2
1.1.1 Les autres ingénieries modélisent déjà . . . 2
1.1.2 Le « code d'abord » et ses limites . . . 2
1.1.3 Chacun comprend le besoin à sa façon . . . 3
1.2 Modèle et IDM . . . 4
1.2.1 Qu'est-ce qu'un modèle ? . . . 4
1.2.2 L'IDM : définition, apports et limites . . . 5

1.3 L'architecture à quatre niveaux de l'OMG 6
1.3.1 Niveau M0 : le système réel 7
1.3.2 Niveau M1 : le modèle 7
1.3.3 Niveau M2 : le méta-modèle 8
1.3.4 La relation de conformité 8
1.3.5 Niveau M3 : le méta-méta-modèle (MOF, Ecore) 9
1.3.6 Le même principe dans plusieurs espaces technologiques 10
1.3.7 Méthode : reconnaître le niveau d'un élément 11
1.4 Études de cas corrigées 12
1.5 Exemples complémentaires 13
1.6 Exercices 14
1.7 Synthèse 15
1.8 Quiz 15

1.1 Pourquoi modéliser ?

1.1.1 Les autres ingénieries modélisent déjà

Un architecte ne pose pas directement des briques. Il dessine d'abord **plusieurs plans** : plan de masse, coupe, façade, plan électrique… Chaque plan est une **vue** destinée à un intervenant (le client, le maçon, l'électricien). Les erreurs sont détectées **sur le plan**, à un moment où elles coûtent encore peu. Il en va de même en génie civil, en mécanique ou en électronique : on simule un pont ou un circuit *avant* de le construire.

Une maquette de garage posée sur une table *ressemble* à un vrai garage, mais on ne peut pas y garer une voiture : c'est un **modèle**. Elle permet pourtant de répondre à des questions (l'emplacement des portes, la circulation…) sans construire le bâtiment.

1.1.2 Le « code d'abord » et ses limites

En logiciel, on commence encore trop souvent par écrire le code. Les conséquences sont connues :

- pas de **vision globale** : la préoccupation technique passe avant le métier ;
- des classes **fortement couplées**, aux responsabilités mélangées (« code spaghetti ») ;
- un code difficile à **comprendre**, à **maintenir** et à **réutiliser** ;
- une connaissance qui reste dans la tête du développeur.

Le coût de correction d'une erreur augmente fortement avec la phase où elle est découverte (figure 1.1). Les valeurs ci-dessous sont des **ordres de grandeur** couramment cités (Boehm) : elles varient selon les projets, mais la tendance est constante.

Exemple 1.1 – Une règle métier dispersée dans le code (feu). On programme le cycle d'un feu de signalisation :

```
class Controleur {
  void cycle() {
```

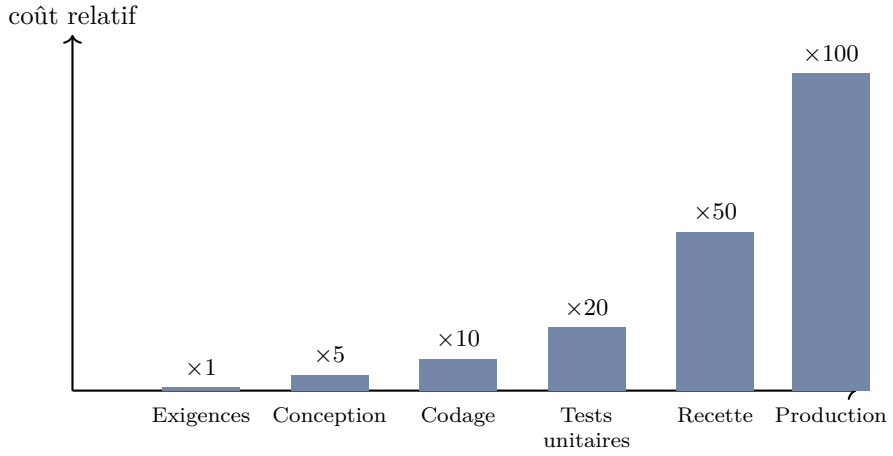


Figure 1.1 – Coût relatif de correction d'une erreur selon la phase où elle est découverte (ordres de grandeur)

```
    allumer(ROUGE);  attendre(25);
    allumer(VERT);   attendre(20);
    allumer(ORANGE); attendre(5);
  }
}
class Simulateur { int dureeVert = 20; /* ... */ }
class Rapport    { String txt = "vert : 20 s"; }
```

Le client demande « vert = 30 s ». Il faut **retrouver** les trois endroits où la valeur 20 est écrite… sans en oublier un. Dans une démarche « modèle d'abord », la durée est écrite **une seule fois**, dans le modèle (transition `after(20 s)`) ; le contrôleur, le simulateur et le rapport sont **générés** à partir de ce modèle et restent cohérents. □

Exemple 1.2 – La même règle dans trois couches (bibliothèque). La règle « un adhérent emprunte au plus 3 livres » est écrite dans le formulaire web (`if (nb >= 3) desactiverBouton()`), dans le service Java (`if (emprunts.size() >= 3) throw...`) et dans une procédure SQL. La bibliothèque passe à 5 livres : trois équipes, trois langages, trois risques d'oubli. Dans un modèle, la règle est **un invariant OCL** unique (`self.emprunts->size() <= 3`) d'où l'on dérive les trois vérifications. □

Exemple 1.3 – Changer de plateforme (boutique en ligne). Une boutique écrite en PHP doit passer à Java/Spring puis proposer une application mobile. Le métier (`Client`, `Commande`, `Produit`, « une commande contient au moins une ligne ») n'a pas changé ; pourtant, sans modèle, il est **réécrit** à chaque technologie. Avec un modèle indépendant de la plateforme, seules les **règles de génération** changent (chapitre 4). □

1.1.3 Chacun comprend le besoin à sa façon

Client, chef de projet, analyste, programmeur : chacun a *sa* vision du besoin (la célèbre caricature de la balançoire). Sans **modèle partagé**, les écarts n'apparaissent qu'à la livraison. Le modèle sert donc aussi de **langage commun**.

À retenir. Le code seul ne suffit plus pour gérer la complexité ni garantir la **pérennité** (maintenance, réutilisation, évolution). Il faut un niveau d'abstraction supérieur : le **modèle**.

1.2 Modèle et IDM

1.2.1 Qu'est-ce qu'un modèle ?

Définition 1.1 (Modèle). *Un **modèle** est une **simplification** (une abstraction) d'un système, construite dans un **objectif** précis : mieux comprendre, communiquer, analyser, simuler ou produire ce système. Un modèle doit pouvoir **se substituer** au système pour répondre aux questions qu'on se pose sur lui.*

Un bon modèle possède trois propriétés : il **représente** un système (relation de représentation), il **simplifie** (il omet ce qui n'est pas utile à l'objectif) et il est **utile** pour cet objectif. Un modèle n'est donc pas jugé « vrai » ou « faux » dans l'absolu : il est *pertinent* ou non pour une question donnée, et il doit rester *fidèle* au système sur les points qu'il retient. Test pratique : peut-on répondre à une question sur le système en regardant *seulement* le modèle ?

Exemple 1.4 – Quatre modèles, quatre objectifs.

- **Carte routière** d'une région : elle omet les bâtiments et la végétation ; elle répond à « quelle route mène de Fès à Meknès ? ».
- **Diagramme de classes** du feu de signalisation : il omet la couleur du boîtier et le câblage ; il répond à « quelles informations le logiciel mémorise-t-il pour chaque feu ? ».
- **Machine à états** du feu : elle omet la structure des données ; elle répond à « après le vert, quelle couleur s'allume, et au bout de combien de temps ? ».
- **Diagramme de cas d'utilisation** d'une boutique en ligne : il omet toute la technique ; il répond à « qui fait quoi avec le système ? ».

□

Un même système admet donc **plusieurs modèles**, chacun étant une **vue** : pour un feu de signalisation, un diagramme de classes (structure), une machine à états (comportement), un schéma électrique, un plan d'implantation…

Tableau 1.1 – Classification des modèles

Critère	Catégorie	Exemple
Rôle	**Descriptif** : décrit un système *existant* **Prescriptif** : décrit un système *à construire*	rétro-ingénierie d'une base de données diagramme de classes de conception
Usage	**Contemplatif** : un dessin pour comprendre ou communiquer **Productif** : un outil l'exploite (vérification, génération, simulation)	schéma au tableau, effacé après la réunion fichier `.ecore` à partir duquel on génère du code

Exemple 1.5 – Classer quatre modèles selon les deux critères.

Modèle	**Rôle**	**Usage**
schéma de la base d'une boutique existante, obtenu par rétro-ingénierie et chargé dans un outil	descriptif	productif
machine à états d'un nouveau feu, simulée puis traduite en code C	prescriptif	productif
croquis du futur écran de prêt, dessiné en réunion	prescriptif	contemplatif
photo annotée du tableau électrique d'un carrefour existant	descriptif	contemplatif

□

À retenir. L'idée centrale de l'IDM : faire passer les modèles du statut **contemplatif** au statut **productif**. Pour cela, un modèle doit être **lisible par une machine**, donc écrit dans un langage précisément défini : son **méta-modèle**.

1.2.2 L'IDM : définition, apports et limites

Définition 1.2 (Ingénierie Dirigée par les Modèles (IDM / MDE)). *Approche du génie logiciel dans laquelle les **modèles sont au centre** du développement. C'est une ingénierie **générative** : tout ou partie de l'application est **produite automatiquement** à partir des modèles. L'IDM fournit des **langages** (de modélisation, de contraintes, de transformation), des **concepts** (modèle, méta-modèle, conformité, transformation) et des **outils** pour créer, vérifier et transformer les modèles.*

Apports.

- erreurs détectées **tôt**, sur les modèles, quand elles coûtent peu ;
- **langage commun** entre experts métier et développeurs ;
- **productivité** par la génération et la réutilisation ; qualité homogène du code généré ;
- **traçabilité** des exigences jusqu'au code ;
- **pérennité** : le savoir-faire métier est capitalisé dans des modèles indépendants des technologies.

Limites. Coût d'apprentissage des outils et des langages ; outillage parfois lourd ; un mauvais modèle génère un mauvais code ; nécessité de synchroniser modèle et code (d'où les *zones protégées*, chapitre 5).

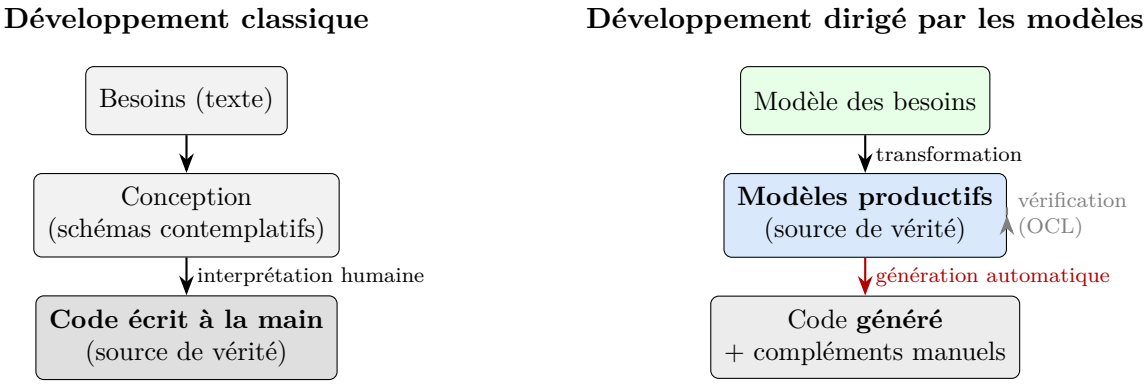


Figure 1.2 – Code d'abord ou modèle d'abord : où se trouve la source de vérité ?

Exemple 1.6 – Du modèle au code : le feu. À partir de la classe `FeuDeSignalisation` (attributs `id : String`, `etat : Couleur`, `dureeVert : Integer`), un générateur produit :

```
public class FeuDeSignalisation {
    private String id;
    private Couleur etat;
    private int dureeVert;
```

```
    public String getId() { return id; }
    public void setId(String id) { this.id = id; }
    // ... autres accesseurs générés ...
    public void changerEtat() {
        // corps écrit par le développeur (comportement)
    }
}
```

Le générateur écrit le code répétitif (attributs, accesseurs) **sans faute de frappe** ; le développeur se concentre sur le **comportement**. Si le modèle change, on **régénère** : modèle et code restent synchronisés. □

Exemple 1.7 – Du modèle à trois artefacts : la bibliothèque. De la seule classe `Book(isbn, title, available)` on génère : la classe Java `Book.java`, la table SQL `CREATE TABLE Book (isbn VARCHAR(100) NOT NULL, ...)` et un formulaire HTML de saisie à trois champs. Ajouter l'attribut `year` dans le modèle met à jour les **trois** artefacts à la génération suivante (chapitres 5 et 6). □

Exemple 1.8 – Du modèle à la vérification : la boutique. L'invariant « le total d'une commande est la somme de ses lignes » est attaché au modèle. L'outil signale, **avant toute écriture de code**, qu'un jeu de données de test contient une commande de total 120 DH dont les lignes totalisent 100 DH. Le modèle est ici *productif* sans générer une seule ligne de code : il sert à **vérifier**. □

1.3 L'architecture à quatre niveaux de l'OMG

L'**OMG** (*Object Management Group*), consortium à l'origine d'UML, organise langages et modèles selon une pile de **quatre niveaux** (figure 1.3). Chaque niveau est décrit par le niveau **au-dessus** ; le niveau M3 se décrit lui-même, ce qui arrête la pile.

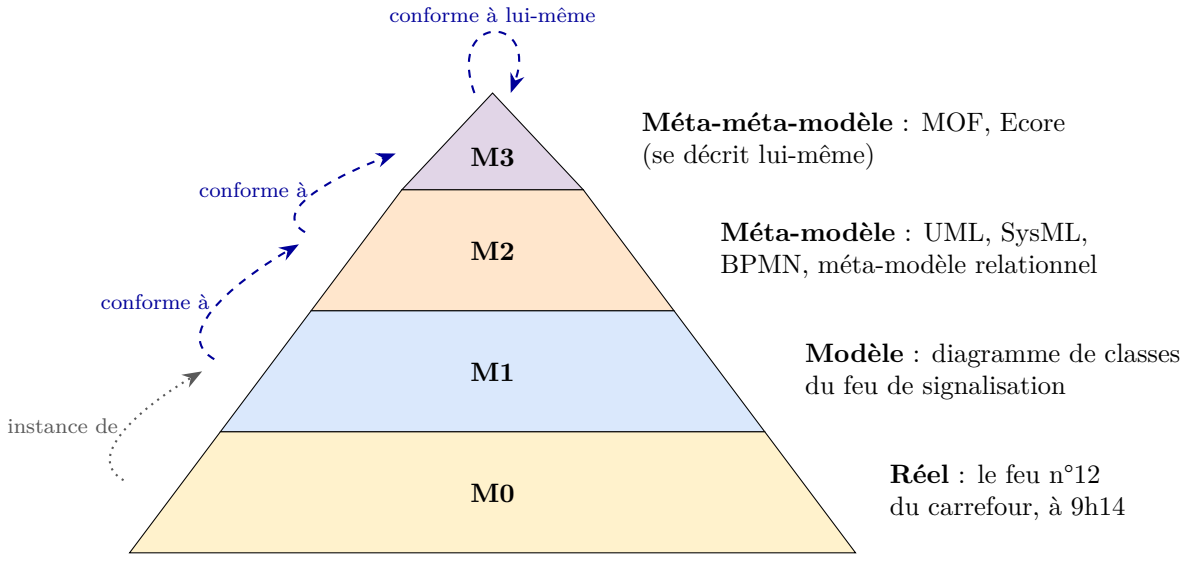


Figure 1.3 – La pyramide de l'OMG : les quatre niveaux de modélisation

Attention. Deux relations différentes relient les niveaux. Un **élément** est *instance de* un élément du niveau supérieur (l'objet `f12` est instance de la classe `FeuDeSignalisation` ; la classe

`FeuDeSignalisation` est instance de la méta-classe `Class`). Un **modèle** tout entier est *conforme à* son méta-modèle. Pour M0, on parle d'*instances* (les objets du système en fonctionnement) plutôt que de conformité.

1.3.1 Niveau M0 : le système réel

Le niveau M0 contient les **objets réels** (les instances) qui existent pendant le fonctionnement du système : le feu **n°12** du carrefour Hassan II, actuellement **vert** ; Mme Amrani qui retire 500 DH au GAB de l'Agdal le 03/10 à 9h14 ; la ligne (111, « Le Petit Prince ») de la table `LIVRE` ; la commande n°4521 passée par M. Idrissi le 12/09.

À retenir. Règle pratique : un élément **daté**, avec des **valeurs réelles** et une **identité** appartient au niveau M0.

1.3.2 Niveau M1 : le modèle

Un modèle décrit **un système particulier**. C'est une abstraction du réel : la classe `FeuDeSignalisation` décrit *tous* les feux possibles, pas un seul. Chaque élément M0 est une **instance** d'un élément M1 (la classe est un *moule*, les objets sont les *gâteaux* : même forme, valeurs différentes). Le modèle doit être **conforme** à un méta-modèle (M2).

Le feu se modélise selon deux vues complémentaires :

- la **vue structurelle** (figure 1.4) : classe `FeuDeSignalisation` (attributs `id`, `etat`, `dureeVert` ; opérations `changerEtat()`, `eteindre()`), énumération `Couleur` = {ROUGE, ORANGE, VERT}, composition : un `Carrefour` possède 1..* feux ;
- la **vue comportementale** (figure 1.5) : une machine à états avec un état initial (le feu démarre en `Rouge`) et des transitions temporelles `after(25 s)`, `after(20 s)`, `after(5 s)`.

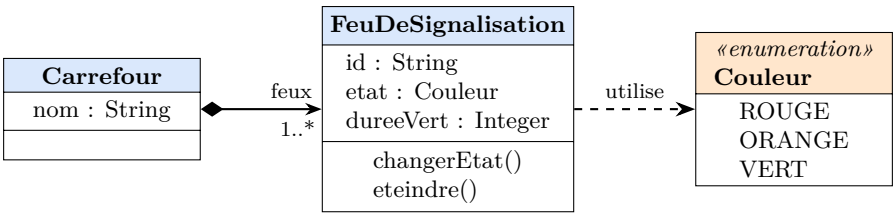


Figure 1.4 – Vue structurelle (M1) : diagramme de classes du feu de signalisation

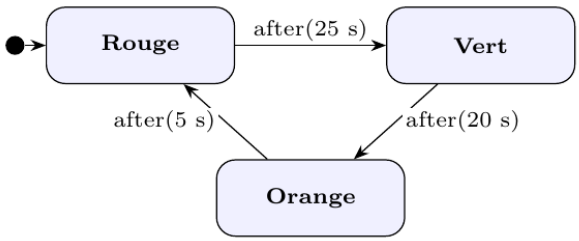


Figure 1.5 – Vue comportementale (M1) : machine à états du feu de signalisation

L'exécution de ce modèle par le feu f12 produit une **trace** : `Rouge` (9h00:00) → `Vert` (9h00:25) → `Orange` (9h00:45) → `Rouge` (9h00:50)... Cette trace datée appartient au niveau **M0**.

1.3.3 Niveau M2 : le méta-modèle

Définition 1.3 (Méta-modèle). *Un **méta-modèle** est un modèle qui définit un **langage de modélisation** : les **concepts** (méta-classes), leurs **relations** et les **règles** (contraintes) qui permettent de construire des modèles M1 valides. Il en fixe la **syntaxe abstraite** ; c'est la « grammaire » du langage.*

Analogie avec la langue : M2 est la grammaire (sujet, verbe, complément…), M1 est une phrase correcte (« Le feu passe au vert. ») ; « Vert le au passe » n'est pas conforme.

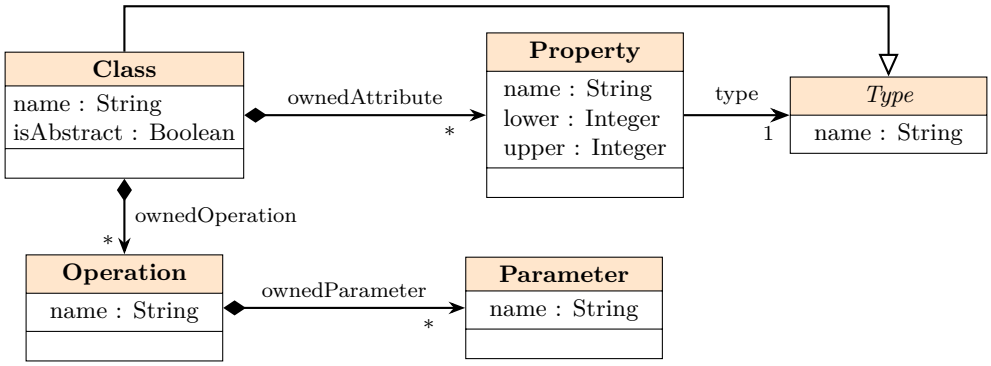


Figure 1.6 – Méta-modèle simplifié du diagramme de classes (M2)

Le méta-modèle de la figure 1.6 dit qu'une **Class** possède des **Property** (avec un nom, une multiplicité et exactement **un** type) et des **Operation** (avec des **Parameter**). Le diagramme du feu est alors lui-même un **ensemble d'objets** de ce méta-modèle : `FeuDeSignalisation` est une instance de `Class`, `dureeVert` une instance de `Property` dont `lower = upper = 1`, etc. C'est ce qui permet à un outil (EMF, Papyrus) de stocker, parcourir et vérifier un diagramme.

Exemple 1.9 – Quatre méta-modèles, quatre langages.

- **Diagramme de classes** (figure 1.6) : `Class`, `Property`, `Operation`, `Parameter`, `Type`.
- **Machine à états** : `StateMachine`, `State`, `Transition`, `Event`, avec des contraintes : exactement **un** état initial par machine ; aucune transition ne **sort** d'un état final ; aucune n'**entre** dans l'état initial.
- **Base relationnelle** (figure 1.9) : `Database`, `Table`, `Column`, `PrimaryKey`, `ForeignKey`, `SQLType`.
- **Cas d'utilisation** : `Actor`, `UseCase`, `Association`, `Include`, `Extend`, `Subject` (exercice 1.4).

Aucun de ces méta-modèles ne contient de concept *métier* (Feu, Livre, Commande) : ils décrivent des **langages**, utilisables pour n'importe quel domaine. □

1.3.4 La relation de conformité

Définition 1.4 (Conformité). *Un modèle est **conforme** à un méta-modèle si (1) **tous** ses éléments sont instances de méta-classes de ce méta-modèle, (2) leurs liens respectent les références et les multiplicités du méta-modèle **et** (3) **toutes** les contraintes additionnelles (OCL) sont respectées.*

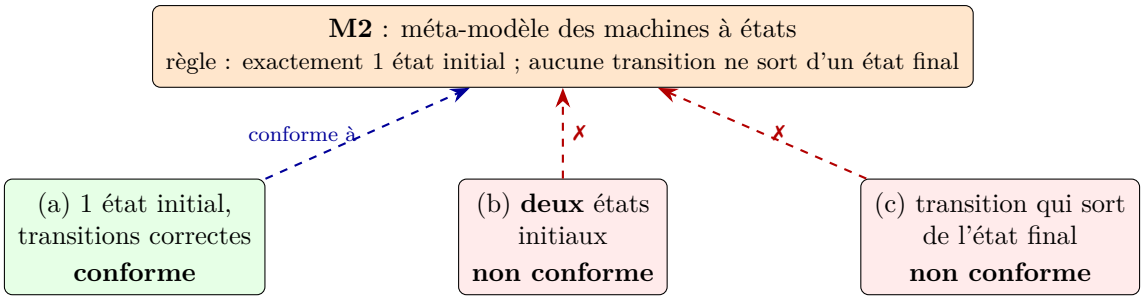


Figure 1.7 – Conformité : exemples et contre-exemples

Les modèles (b) et (c) se *dessinent* très bien sur papier… mais un outil les **refuse**. La vérification devient **automatique**. Les règles fines s'écrivent en **OCL** (chapitre 3) ; avec le méta-modèle de machines à états du chapitre 2 (exemple 2.14) :

```
context StateMachine inv unSeulInitial:
  self.states->select(s | s.kind = StateKind::INITIAL)->size() = 1
```

Exemple 1.10 – Quatre vérifications de conformité.

- **Machine à états** (figure 1.7) : deux états initiaux violent une *contrainte* : non conforme.
- **Document XML** (exemple 1.17) : `<feu id="RBT-012" vert="vingt"/>` viole le *type* `xs:integer` déclaré par le schéma : non conforme.
- **Programme Java** (exemple 1.18) : `clas Feu {}` ne respecte pas la *grammaire* : le compilateur le refuse.
- **Diagramme de classes** (exemple 1.20) : un attribut sans type viole la *multiplicité* `type [1]` : non conforme.

Dans les quatre cas, c'est un **outil** (validateur EMF, validateur XML, compilateur) qui détecte l'erreur, parce que le langage est défini précisément. □

1.3.5 Niveau M3 : le méta-méta-modèle (MOF, Ecore)

- Le niveau M3 est le plus abstrait : il définit les concepts pour écrire des **méta-modèles** (UML, SysML, BPMN…).
- Le standard de l'OMG est le **MOF** (*Meta Object Facility*) ; ses concepts principaux sont Class, Property, Operation, DataType, Package.
- Le MOF est **réflexif** : il est décrit avec le MOF. C'est pourquoi la pile s'arrête à M3 : il n'existe pas de niveau M4.
- Deux variantes : **EMOF** (*Essential MOF*, noyau simple) et **CMOF** (*Complete MOF*).

Attention. « Méta » est une **relation**, pas une nature. Le méta-modèle UML est un *modèle* pour le MOF (il lui est conforme), et un *méta-modèle* pour vos diagrammes.

Ecore. **Ecore** est l'implantation d'EMOF (légèrement simplifiée) dans Eclipse (*Eclipse Modeling Framework*, EMF). C'est le M3 utilisé dans tous les travaux pratiques de ce cours.

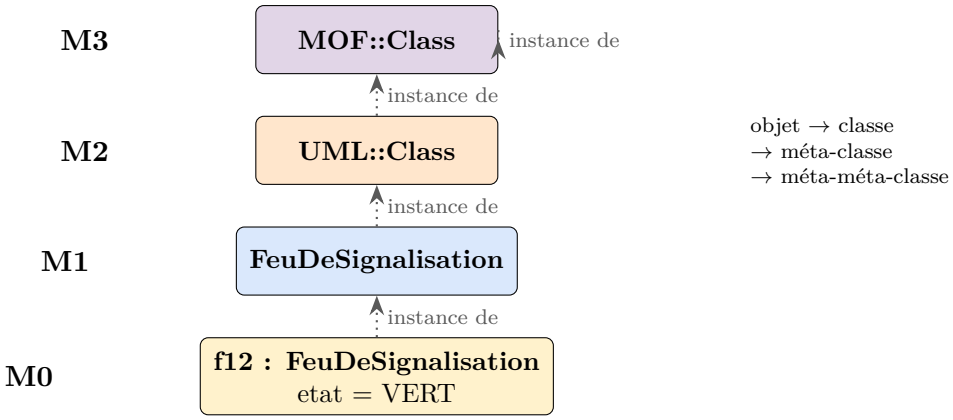


Figure 1.8 – La chaîne complète « instance de » du feu de signalisation

Tableau 1.2 – Correspondance MOF – Ecore et exemples

MOF (M3)	Ecore (M3)	Exemple dans UML (M2)	Exemple feu (M1)
Package	`EPackage`	le paquetage `uml`	`signalisation`
Class	`EClass`	`Class`, `State`	`FeuDeSignalisation`
Property (type simple)	`EAttribute`	`name`, `isAbstract`	`etat`, `dureeVert`
Property (type classe)	`EReference`	`ownedAttribute`, `source`	`feux`
DataType / Enumeration	`EDataType` / `EEnum`	`Boolean`, `VisibilityKind`	`Couleur`
Operation	`EOperation`	`allAttributes()`	`changerEtat()`

Exemple 1.11 – Un mini-langage de machines à états en Ecore, un modèle en XMI. Méta-modèle (M2) : un `Diagramme` contient (composition) des `Etat` ; chaque `Etat` contient des `Transition` sortantes qui pointent vers un état `suivant` ; le `Diagramme` désigne son état `initial`. Le modèle (M1) du feu est enregistré au format **XMI** (*XML Metadata Interchange*) :

```
<?xml version="1.0" encoding="UTF-8"?>
<sm:Diagramme xmi:version="2.0" xmlns:xmi="http://www.omg.org/XMI"
    xmlns:sm="http://ensias.ma/diagramme/1.0" nom="Feu" initial="//@etats.0">
  <etats nom="Rouge">  <sortantes garde="after(25)" suivant="//@etats.1"/> </etats>
  <etats nom="Vert">   <sortantes garde="after(20)" suivant="//@etats.2"/> </etats>
  <etats nom="Orange"> <sortantes garde="after(5)"  suivant="//@etats.0"/> </etats>
</sm:Diagramme>
```

Une balise imbriquée = un objet **contenu** ; un attribut XML porte soit la valeur d'un `EAttribute` (`nom`, `garde`), soit un **chemin** vers l'objet visé par une référence non contenante (`suivant="//@etats.1"` = le 2e état, les indices commençant à 0). Le `xmlns:sm` rattache le fichier à son méta-modèle. □

1.3.6 Le même principe dans plusieurs espaces technologiques

La pile M0–M3 n'est **pas propre à UML** : c'est une façon générale d'organiser langages et modèles (tableau 1.3).

Exemple 1.12 – Un élément de chaque niveau dans quatre domaines.

Tableau 1.3 – Les quatre niveaux dans différents espaces technologiques

Niveau	Monde UML	Monde Java	Monde XML	Monde BD
M3	MOF	EBNF (notation des grammaires)	XSD des XSD (*schema for schemas*)	MOF (ou Ecore)
M2	méta-modèle UML	grammaire de Java	schéma XML (`.xsd`)	méta-modèle relationnel (Table, Column)
M1	diagramme de classes du feu	le programme `Feu.java`	document `.xml`	schéma de la table FEU
M0	le feu n°12 dans la rue	les objets en mémoire	les données du feu n°12	les lignes de la table FEU

	Feu	Bibliothèque	Boutique	GAB
M3	`EClass`	`EClass`	`EReference`	`EAttribute`
M2	`State`	`Class`	`Association`	`UseCase`
M1	état `Vert`	classe `Livre`	association Client–Commande	cas « Retirer de l'argent »
M0	f12 vert à 9h00:25	l'exemplaire n°111	M. Idrissi a passé la commande 4521	Mme Amrani retire 500 DH

□

1.3.7 Méthode : reconnaître le niveau d'un élément

1. L'élément est-il un objet concret, daté, avec des valeurs réelles ? ⇒ **M0**.
2. Décrit-il un système *particulier* (concepts métier : Livre, Feu, Adhérent) ? ⇒ **M1**.
3. Décrit-il un *langage* (Class, State, Table, Transition), sans concept métier ? ⇒ **M2**.
4. Décrit-il les langages de méta-modélisation (EClass, EReference) et se décrit-il lui-même ? ⇒ **M3**.

Tableau 1.4 – Pièges fréquents sur les niveaux de modélisation

Erreur fréquente	Ce qu'il faut dire
« La classe Livre est en M0 car elle représente un vrai livre. »	La classe est en **M1** ; seul **un livre précis** est en M0.
« Le méta-modèle, c'est mon diagramme le plus abstrait. »	Non : il décrit **le langage** et ne contient aucun concept métier.
« Une multiplicité 0..* est une méta-classe. »	Non : ce sont les **valeurs** de `lower`/`upper` d'une **Property**.
« M1 est instance de M0. »	C'est l'inverse : les éléments M0 sont **instances** d'éléments M1 ; M1 est **conforme à** M2.
« Il existe un niveau M4. »	Non : le MOF se décrit lui-même (réflexivité).
« CIM, PIM et PSM sont trois niveaux différents. »	Non : ce sont trois modèles **M1** vus à des niveaux d'*abstraction* différents (chapitre 4).

1.4 Études de cas corrigées

Exemple 1.13 – Le diagramme de cas d'utilisation d'un GAB. Un schéma montre l'acteur *Client* relié aux cas *Retirer de l'argent*, *Consulter le solde* et *Changer le code*, à l'intérieur d'un rectangle *GAB*.

- C'est un **diagramme de cas d'utilisation**, de niveau **M1**.
- C'est un modèle d'**analyse** et de **communication** : il dit *ce que* fait le système, pas *comment*.
- Il est validé par le méta-modèle des cas d'utilisation : méta-classes `Actor`, `UseCase`, `Association`, `Include`, `Extend`, `Subject`.
- Exemple d'élément M0 : « Mme Amrani retire 500 DH au GAB de l'Agdal le 03/10 à 9h14 ».

□

Exemple 1.14 – Méta-modèle d'une base de données relationnelle. Énoncé : une base contient des **tables** ; une table comporte des **colonnes** ; tous ont un **nom**. Une colonne a un **type SQL** : `Integer`, `Text` ou `Varchar(n)`. Une table a des **contraintes** nommées : **Primary Key** (une ou plusieurs colonnes) ou **Foreign Key** (une ou plusieurs colonnes, référence une autre table).

Méthode : les **noms** de l'énoncé deviennent des méta-classes ; « contient / comporte » donne des **compositions** ; « de deux sortes » donne un **héritage** (méta-classe abstraite) ; « se caractérise par » donne des **attributs**. Résultat : figure 1.9. `SQLType` et `Constraint` sont **abstraites**. Les colonnes d'une clé sont des **références simples** vers des colonnes déjà contenues par la table (pas des compositions). □

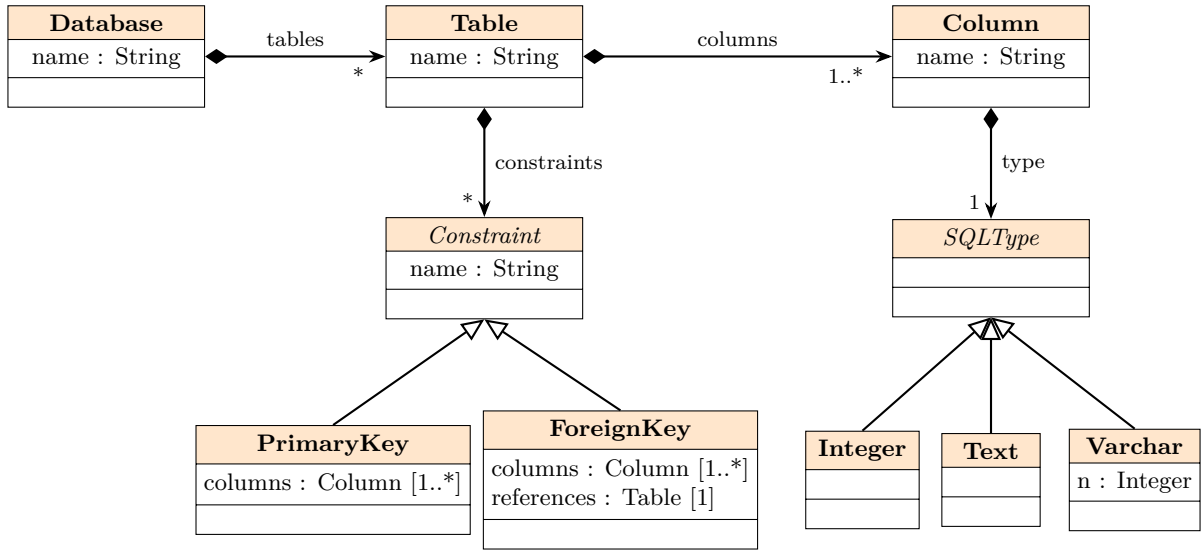


Figure 1.9 – Méta-modèle (M2) simplifié d'une base de données relationnelle

La table `LIVRE` (colonnes `isbn`, `titre`, `auteur`) est alors un **modèle M1** conforme à ce méta-modèle relationnel ; ses **lignes** sont des données réelles (M0).

Exemple 1.15 – Le diagramme de classes d'une bibliothèque. Une bibliothèque gère des **livres** (ISBN, titre, auteur, état : disponible, emprunté, en retard), des **adhérents** (numéro,

nom, e-mail) et des **emprunts** (date de début, date de retour prévue, date de retour effective optionnelle). Un livre n'est emprunté que s'il est disponible ; au plus 5 emprunts *en cours* par adhérent.

- `Adherent`, `Emprunt`, `Livre` → méta-classe **Class** ; `isbn`, `titre...` → **Property** ; `EtatLivre` → **Enumeration** ; `effectue`, `concerne` → **Association** ; `estDisponible()` → **Operation**.
- Le diagramme est en **M1** ; le concept « Class » d'UML en **M2** ; un livre physique en **M0** ; le MOF en **M3**.
- La règle « au plus 5 emprunts *en cours* » ne se dessine pas : une multiplicité `0..5` compterait aussi les emprunts terminés (l'historique) ; on l'exprime par un **invariant OCL** qui ne compte que les emprunts sans date de retour effective.

□

Exemple 1.16 – Le diagramme de classes d'une boutique en ligne. Une boutique gère des `Client` (nom, e-mail), des `Produit` (référence, libellé, prix) et des `Commande` (numéro, date, statut : EN_COURS, PAYEE, LIVREE). Une commande appartient à **un** client et **contient** au moins une `LigneCommande` (quantité) qui **référence** un produit.

- « contient » donne une **composition** `Commande` ♦– `LigneCommande [1..*]` ; « référence » une **association simple** `LigneCommande` – `Produit [1]` : supprimer une commande supprime ses lignes, pas les produits ;
- `statut` est un attribut de type **énumération** `StatutCommande` ;
- la règle « quantité ≥ 1 » et « le total est la somme des lignes » s'écrivent en OCL (chapitre 3).

□

1.5 Exemples complémentaires

Exemple 1.17 – Les quatre niveaux dans le monde XML. Le méta-modèle (M2) est un schéma XSD :

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="carrefour">
    <xs:complexType><xs:sequence>
      <xs:element name="feu" maxOccurs="unbounded">
        <xs:complexType>
          <xs:attribute name="id"   type="xs:string"  use="required"/>
          <xs:attribute name="vert" type="xs:integer" use="required"/>
        </xs:complexType>
      </xs:element>
    </xs:sequence></xs:complexType>
  </xs:element>
</xs:schema>
```

Le modèle (M1) est un document conforme : `<carrefour><feu id="RBT-012" vert="20"/><feu id="RBT-013" vert="25"/></carrefour>`. Le schéma des schémas XSD du W3C, qui définit ce qu'est un schéma, joue le rôle de M3 ; le feu RBT-012 du carrefour réel est en M0. Un

validateur XML vérifie la **conformité** du document au schéma : un attribut `vert="vingt"` serait refusé (type `xs:integer`). □

Exemple 1.18 – Les quatre niveaux dans le monde Java.

- **M3** : la notation EBNF, qui sert à écrire des grammaires ;
- **M2** : la grammaire de Java, par exemple `ClassDecl ::= "class" Identifier "{" {FieldDecl | MethodDecl} "}"` ;
- **M1** : le programme `class Feu { int dureeVert; }`, conforme à la grammaire ;
- **M0** : l'objet créé par `new Feu()` en mémoire pendant l'exécution, avec `dureeVert = 20`.

Un compilateur qui refuse `clas Feu {}` détecte une **non-conformité** au méta-modèle (la grammaire). □

Exemple 1.19 – Un même système, trois modèles pour trois objectifs. Pour un distributeur automatique de billets (GAB) :

- un **diagramme de cas d'utilisation** pour discuter avec la banque (*quoi ?*) ;
- un **diagramme d'états** pour décrire le cycle « attente carte → saisie code → choix montant → distribution » (*comment se comporte-t-il ?*) ;
- un **modèle de files d'attente** pour estimer le temps d'attente aux heures de pointe (*combien ?*).

Aucun n'est « le bon » : chacun est **utile** pour une question précise. □

Exemple 1.20 – Vérifier la conformité d'un diagramme de classes. On reprend le méta-modèle de la figure 1.6, où une `Property` a exactement un `type`. Le diagramme suivant contient une classe `Livre` avec l'attribut `titre` (sans type) et une opération `emprunter()` placée dans une `Property`.

- `titre` viole la multiplicité `type [1]` : non conforme ;
- une `Operation` ne peut pas être contenue par une `Property` (aucune composition ne le permet) : non conforme.

Après correction (`titre : String`, `emprunter()` dans `ownedOperation` de `Livre`), le diagramme est conforme. □

1.6 Exercices

Les corrigés figurent en annexe A.

Exercice 1.1 – Classer par niveau. Indiquer le niveau (M0 à M3) de : (a) le diagramme d'états du feu ; (b) la méta-classe `State` ; (c) `EClass` ; (d) l'adhérent Mme Amrani ; (e) la grammaire de Java ; (f) le fichier `Feu.java` ; (g) le MOF ; (h) le schéma relationnel de la base « bibliothèque » ; (i) la ligne (M1, 'Sara') de la table `MEMBRE`.

Exercice 1.2 – Chaîne « instance de ». Pour le livre physique « Le Petit Prince » de la bibliothèque de l'ENSIAS, donner la chaîne complète des éléments de M0 à M3 et nommer chaque relation.

Exercice 1.3 – Types de modèles. Pour chaque modèle, dire s'il est descriptif ou prescriptif, contemplatif ou productif : (a) le plan d'une maison à construire, dessiné à la main ; (b) un fichier `.ecore` à partir duquel on génère du Java ; (c) le schéma d'une base existante obtenu par rétro-ingénierie et utilisé pour générer une documentation ; (d) un croquis sur un tableau blanc.

Exercice 1.4 – Méta-modèle des cas d'utilisation. Proposer un méta-modèle simplifié (M2) permettant de décrire des diagrammes de cas d'utilisation : un système nommé contient des cas d'utilisation ; des acteurs sont associés à des cas ; un cas peut en inclure ou en étendre un autre.

Exercice 1.5 – Conformité. Avec le méta-modèle de l'exercice 1.4, le modèle suivant est-il conforme ? Un acteur « Client » associé au cas « Retirer » ; le cas « Retirer » inclut le cas « S'authentifier » ; l'acteur « Banque » inclut le cas « Retirer ». Justifier.

Exercice 1.6 – Vrai ou faux ? (a) Un méta-modèle est un modèle. (b) Un modèle M1 peut être conforme à plusieurs méta-modèles. (c) Ecore est un méta-modèle M2. (d) Le niveau M0 contient des classes. (e) En IDM, le code n'est plus jamais écrit à la main.

1.7 Synthèse

Tableau 1.5 – Synthèse du chapitre 1

Modèle	représente un système, le simplifie et sert un objectif.
IDM	le modèle devient productif : on le vérifie, on le transforme, on génère du code.
M0	le réel : objets concrets, datés, avec des valeurs.
M1	le modèle d'un système ; les objets M0 en sont des instances.
M2	le méta-modèle (le langage) ; M1 lui est conforme.
M3	MOF / Ecore : définit les langages et se décrit lui-même.
Conformité	éléments instances des méta-classes, liens et multiplicités respectés, contraintes vérifiées.

1.8 Quiz

Répondez aux questions suivantes. Les corrigés commentés figurent en annexe A.

Q1. Une erreur de conception découverte en production coûte, par rapport à une erreur corrigée pendant l'analyse des exigences, environ :

a) 2 fois plus

b) 10 fois plus

c) 100 fois plus

d) autant

Q2. Un diagramme de classes dessiné au tableau puis effacé à la fin de la réunion est un modèle :

a) productif
b) contemplatif
c) exécutable
d) conforme au MOF

Q3. Un modèle qui décrit un système *à construire* est dit :

a) descriptif
b) prescriptif
c) réflexif
d) exogène

Q4. Le méta-modèle UML est-il un modèle ?

a) Non, c'est un méta-modèle, pas un modèle
b) Oui, c'est un modèle conforme au MOF
c) Oui, c'est un modèle M1
d) Non, il appartient au niveau M3

Q5. Quelle relation relie un élément M0 à un élément M1 ?

a) « conforme à »
b) « instance de »
c) « hérite de »
d) « raffine »

Q6. Pourquoi n'existe-t-il pas de niveau M4 ?

a) Parce que l'OMG l'a interdit
b) Parce que le MOF est réflexif : il se décrit lui-même
c) Parce qu'Ecore remplace M4
d) Parce que M3 est un modèle M1

Q7. À quel niveau se trouve la méta-classe `Transition` d'une machine à états ?

a) M0
b) M1
c) M2
d) M3

Q8. À quel niveau se trouve `EReference` ?

a) M0
b) M1
c) M2
d) M3

Q9. Un fichier `carrefour.xsd` joue le rôle de :

a) modèle M1
b) méta-modèle M2
c) données M0
d) méta-méta-modèle M3

Q10. Une machine à états qui possède deux états initiaux :

a) est conforme car chaque élément est instance d'une méta-classe
b) n'est pas conforme car une contrainte du méta-modèle est violée
c) appartient au niveau M2
d) est un modèle descriptif

Q11. Classer : (a) le langage SQL ; (b) la table LIVRE ; (c) la ligne (111, « Le Petit Prince ») ; (d) l'état `Orange` du diagramme du feu ; (e) le feu n°13.

Q12. Citer deux apports et une limite de l'IDM.

Chapitre 2

Méta-modélisation avec EMF et Ecore

Résumé

*Pour qu'un modèle soit **productif**, il doit être écrit dans un langage défini précisément : son **méta-modèle**. Ce chapitre montre comment construire un méta-modèle avec **Ecore**, le méta-méta-modèle d'Eclipse EMF : paquetages (`EPackage`), classes (`EClass`), attributs (`EAttribute`), références (`EReference` : composition ou non, multiplicités, opposées), types de données, énumérations et héritage. On voit ensuite comment EMF **génère une API Java** à partir du méta-modèle (`.genmodel`), comment on **instancie** un modèle (`.xmi`) et comment on construit un éditeur graphique dédié avec Obeo Designer / Sirius. Chaque choix de modélisation est illustré sur la bibliothèque, la machine à états, la boutique en ligne et les familles. Les contraintes qu'un méta-modèle ne peut pas exprimer seront écrites en OCL au chapitre 3.*

Objectifs d'apprentissage

À l'issue de ce chapitre, vous serez capable de :

- identifier les concepts d'Ecore et leurs propriétés (`lowerBound`, `upperBound`, `containment`, `eOpposite`, `eSuperTypes`) ;
- construire un méta-modèle Ecore à partir d'un énoncé en langue naturelle ;
- choisir entre attribut et référence, composition et simple référence, et fixer les multiplicités ;
- écrire et lire un modèle conforme au format XMI ;
- expliquer le rôle du `.genmodel` et utiliser l'API Java générée ;
- décrire la construction d'un éditeur graphique avec Sirius.

Contenu du chapitre

2.1 EMF et Ecore 19
2.1.1 Les concepts d'Ecore 19
2.1.2 Démarche : de l'énoncé au méta-modèle 20
2.1.3 Modèle et méta-modèle : deux niveaux à ne pas confondre 20
2.1.4 Génération de l'API Java : le `.genmodel` 21
2.1.5 Instancier un modèle : le fichier XMI 23
2.1.6 Un éditeur graphique dédié avec Obeo Designer / Sirius 23
2.2 Les choix de modélisation en détail 24

2.2.1 Attribut ou référence ? . . . 24
2.2.2 Multiplicités . . . 24
2.2.3 Composition ou simple référence ? . . . 25
2.2.4 Énumérations et types de données . . . 25
2.2.5 Héritage et classes abstraites . . . 26
2.2.6 Références opposées . . . 26
2.3 Exemples complets . . . 26
2.4 Exercices . . . 28
2.5 Synthèse . . . 28
2.6 Quiz . . . 29

2.1 EMF et Ecore

EMF (*Eclipse Modeling Framework*) est le socle de la modélisation dans Eclipse. Il fournit : le langage de méta-modélisation **Ecore** (une implantation d'EMOF), la sérialisation **XMI**, un **générateur de code** Java, des éditeurs arborescents et une API **réflexive**. Presque tous les outils IDM d'Eclipse (OCL, Sirius, Acceleo, ATL, Xtext, Papyrus) s'appuient sur EMF.

2.1.1 Les concepts d'Ecore

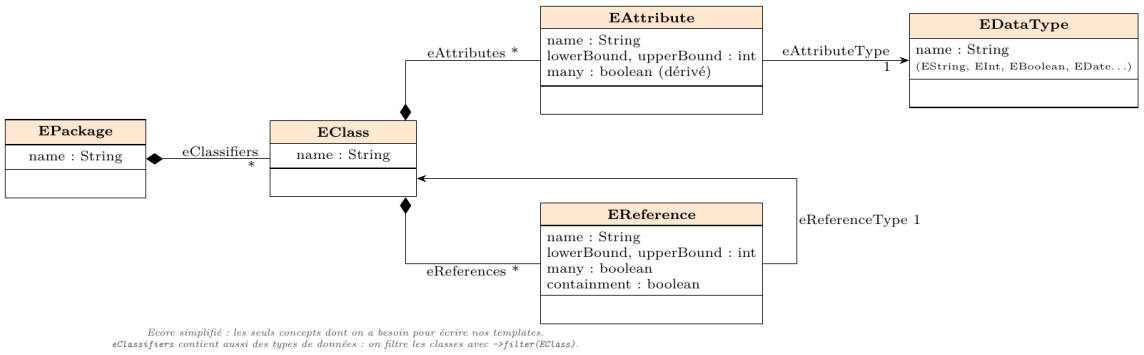


Figure 2.1 – Méta-modèle Ecore simplifié : les concepts utiles dans ce cours

Quelques propriétés dérivées sont très utilisées dans les transformations : `many` vaut vrai si `upperBound` vaut `-1` (noté `*`) ou est supérieure à 1 ; `required` vaut vrai si `lowerBound` ≥ 1. Par défaut, un attribut ou une référence créé dans l'éditeur a `lowerBound = 0` et `upperBound = 1` (`[0..1]`).

À retenir.

- **Composition** (`containment = true`) : l'objet contenu n'existe que dans son conteneur ; en XMI il est écrit *à l'intérieur* de la balise du conteneur. Tout objet d'un modèle EMF a **au plus un** conteneur ; seuls les objets racines d'une ressource n'en ont pas.
- **Référence opposée** (`eOpposite`) : deux références qui se répondent (`Family.father` ↔ `Member.familyFather`). EMF maintient automatiquement les deux côtés.

Tableau 2.1 – Principaux concepts d'Ecore

Concept	Rôle	Propriétés importantes
`EPackage`	conteneur racine d'un méta-modèle	`name`, `nsURI` (identifiant unique), `nsPrefix`, `eClassifiers`
`EClass`	une méta-classe	`name`, `abstract`, `interface`, `eSuperTypes`, `eAttributes`, `eReferences`, `eOperations`
`EAttribute`	propriété de type simple	`name`, `eAttributeType`, `lowerBound`, `upperBound`, `iD`, `defaultValueLiteral`
`EReference`	propriété dont le type est une `EClass`	`eReferenceType`, `containment` (composition), `eOpposite`, bornes
`EDataType`	type de données	`EString`, `EInt`, `EBoolean`, `EDouble`, `EDate`…
`EEnum`	énumération	`eLiterals` (`EEnumLiteral`)
`EOperation`	opération	paramètres (`EParameter`), type de retour

2.1.2 Démarche : de l'énoncé au méta-modèle

1. Créer un projet (*Modeling Project* sous Obeo Designer, ou *Empty EMF Project*).
2. Créer le fichier `.ecore` et son `EPackage` racine : `name`, `nsURI` (ex. `http://fr.obeo.library/1.0`), `nsPrefix`.
3. Ajouter les `EClass` (les **noms** de l'énoncé).
4. Ajouter les `EAttribute` avec leur type et leurs bornes.
5. Ajouter les `EReference` : composition (« contient ») ou simple référence (« concerne »), bornes, opposées.
6. Ajouter l'héritage (`eSuperTypes`) et marquer les classes abstraites.
7. **Valider** le méta-modèle (clic droit *Validate*).

Exemple 2.1 – Le méta-modèle `library.ecore`. Le méta-modèle `library.ecore` de la figure 2.2 est utilisé aux chapitres 5 et 6 :

- `Library` (`name : EString [1]`) contient, par trois compositions `0..*`, des `Book` (référence `book`), des `Member` (référence `member`) et des `Loan` (référence `loan`) ;
- `Book` : `id`, `title`, `isbn`, `author` (`EString [1]`) et `available : EBoolean` (valeur par défaut `false`, bornes `[0..1]`) ;
- `Member` : `memberId`, `name` (obligatoires, `[1]`), `email` (facultatif, `[0..1]`) ;
- `Loan` : `loanDate : EDate [1]`, `returnDate : EDate [0..1]` ; il référence **un** livre (`book : Book [0..1]`) et **un** membre (`member : Member [0..1]`), sans composition.

Les noms des compositions sont au **singulier** (`book`, `member`, `loan`), comme dans la figure : c'est ce nom que l'on retrouve dans le XMI (`<book .../>`) et dans l'API générée (`getBook()`, qui renvoie une liste). □

2.1.3 Modèle et méta-modèle : deux niveaux à ne pas confondre

Selon l'outil, `library.ecore` joue deux rôles différents (figure 2.3) :

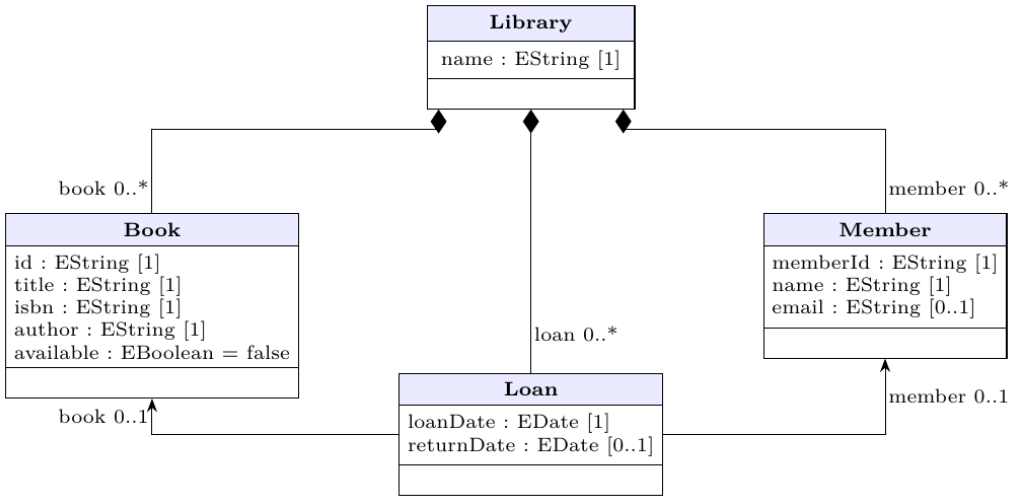


Figure 2.2 – Le méta-modèle `library.ecore`

- pour **EMF**, c'est un **méta-modèle** (M2) : on peut créer des bibliothèques concrètes (`.xmi`) qui lui sont conformes ;
- pour **Acceleo** ou un programme Java qui *lit* ce fichier pour générer du code, c'est un **modèle** (M1) conforme au méta-modèle **Ecore** (qui joue alors le rôle de M2).

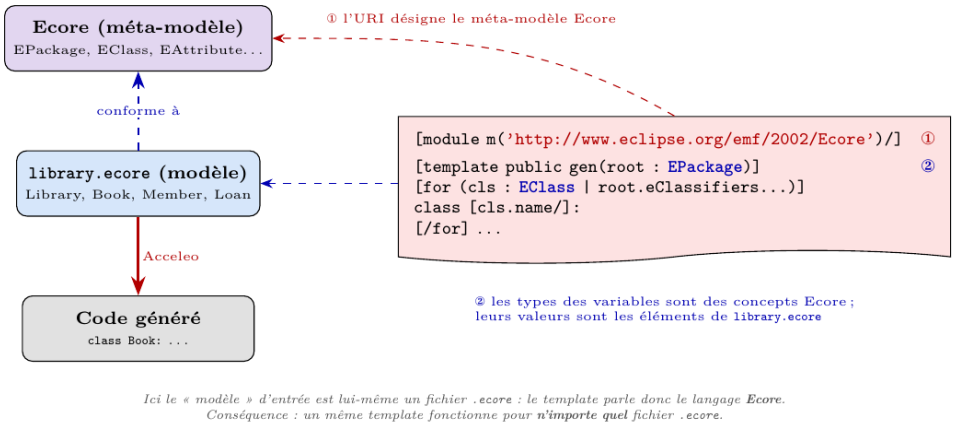


Figure 2.3 – `library.ecore` : méta-modèle pour EMF, modèle pour un générateur

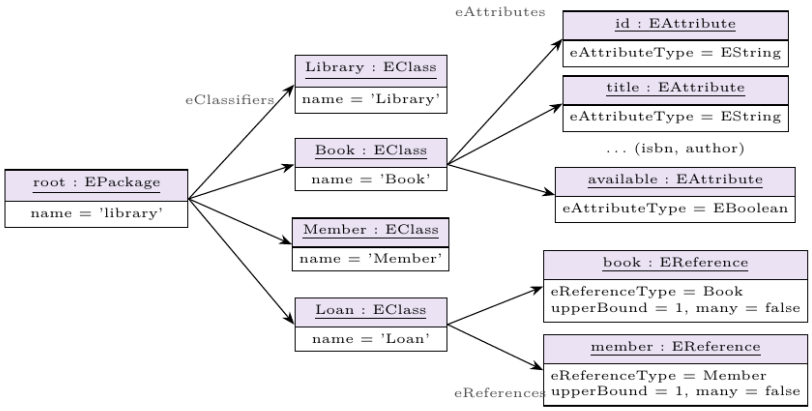
2.1.4 Génération de l'API Java : le `.genmodel`

À partir du méta-modèle, EMF génère une API Java complète :

1. clic droit sur `library.ecore` → *New* → *EMF Generator Model* (fichier `library.genmodel`) ;
2. clic droit sur la racine du `.genmodel` → *Generate Model Code* (et éventuellement *Edit* et *Editor Code*).

Exemple 2.2 – Créer et enregistrer un modèle avec l'API générée.

```
LibraryFactory f = LibraryFactory.eINSTANCE;
```



*Ce que « voit » Acceleo : un **graphe d'objets** Ecore. Le template **navigue** dans ce graphe en suivant les flèches.*

Figure 2.4 – `library.ecore` vu comme un graphe d'objets Ecore (`EPackage`, `EClass`, `EAttribute`, `EReference`)

Tableau 2.2 – Code généré par EMF pour le méta-modèle `library`

Élément généré	Contenu
Interfaces `Book`, `Member`...	getters/setters (`getTitle()`, `setTitle()`, `isAvailable()`), listes `EList<Book> getBook()`
Classes `BookImpl`...	implémentation, notifications, gestion des opposées
`LibraryFactory`	création des objets : `LibraryFactory.eINSTANCE.createBook()`
`LibraryPackage`	méta-données réflexives : `LibraryPackage.Literals.BOOK__TITLE`
`util` (`LibrarySwitch`, `LibraryAdapterFactory`)	*switch* et *adapter factory* ; un `LibraryValidator` est ajouté si le méta-modèle porte des contraintes

```
Library lib = f.createLibrary();   lib.setName("ENSIAS");
Book b = f.createBook();           b.setTitle("Le Petit Prince"); b.setAvailable(false);
Member m = f.createMember();       m.setName("Sara");
lib.getBook().add(b);              lib.getMember().add(m);    // compositions
Loan l = f.createLoan();           l.setBook(b); l.setMember(m);
l.setLoanDate(new Date());
lib.getLoan().add(l);

ResourceSet rs = new ResourceSetImpl();
rs.getResourceFactoryRegistry().getExtensionToFactoryMap()
  .put("xmi", new XMIResourceFactoryImpl());
Resource r = rs.createResource(URI.createFileURI("model/ensias.xmi"));
r.getContents().add(lib);          // la racine
r.save(null);                      // lève IOException : à déclarer ou à capturer
```

Hors d'Eclipse (application Java autonome), il faut aussi enregistrer le paquetage : `LibraryPackage.eINSTANCE.eClass();` avant de charger un fichier `.xmi`. □

2.1.5 Instancier un modèle : le fichier XMI

On peut aussi créer un modèle sans écrire de code : clic droit sur la méta-classe racine dans l'éditeur Ecore → *Create Dynamic Instance*, puis édition dans le *Sample Reflective Ecore Model Editor*. Le fichier produit ressemble à :

```
<?xml version="1.0" encoding="UTF-8"?>
<library:Library xmi:version="2.0" xmlns:xmi="http://www.omg.org/XMI"
    xmlns:library="http://fr.obeo.library/1.0" name="ENSIAS">
  <book id="B1" title="Le Petit Prince" isbn="978-2070612758" author="Saint-Exupéry"/>
  <member memberId="M1" name="Sara" email="sara@um5.ac.ma"/>
  <loan loanDate="2026-11-02" book="//@book.0" member="//@member.0"/>
</library:Library>
```

L'attribut `xmlns:library` doit correspondre au `nsURI` du méta-modèle : c'est lui qui dit « ce modèle est conforme à `library.ecore` ». L'attribut `available` n'apparaît pas : il a sa valeur par défaut (`false`) et EMF n'écrit pas les valeurs par défaut.

2.1.6 Un éditeur graphique dédié avec Obeo Designer / Sirius

Obeo Designer (fondé sur **Sirius**) permet de définir, sans programmer, une **syntaxe concrète graphique** pour un méta-modèle : un fichier `.odesign` décrit un *Viewpoint*, des *Diagram Descriptions*, des *Node Mappings* (une `EClass` → un nœud avec forme, couleur, libellé), des *Edge Mappings* (une `EReference` → un lien) et une *palette* d'outils de création. Le modèle est enregistré en XMI, la représentation dans un fichier `.aird`. On obtient ainsi un véritable **DSL graphique** (langage dédié).

Exemple 2.3 – Trois éditeurs Sirius pour trois méta-modèles.

- **Machine à états** : `State` → nœud arrondi (vert si `INITIAL`, double cercle si `FINAL`) ; `Transition` → flèche étiquetée par `event`.
- **Bibliothèque** : `Book` → rectangle bleu (rouge si `available = false`) ; `Loan` → lien entre le membre et le livre.
- **Familles** : `Family` → conteneur ; `Member` → nœud dont l'icône dépend du rôle (père, mère, fils, fille).

□

2.2 Les choix de modélisation en détail

2.2.1 Attribut ou référence ?

- Un **attribut** (`EAttribute`) porte une **valeur** de type simple : texte, nombre, booléen, date, valeur d'énumération. Il n'a pas d'identité propre.
- Une **référence** (`EReference`) pointe vers un **objet** d'une `EClass`, qui a sa propre identité et ses propres propriétés.

Exemple 2.4 – Auteur : attribut ou classe ?. Dans `library.ecore`, `author : EString` est un attribut : on ne connaît de l'auteur que son nom. Si l'on veut connaître sa nationalité, sa date de naissance et la liste de ses livres, on crée une `EClass Author` et une référence `Book.authors : Author [1..*]`. Le choix dépend des **questions** que le modèle doit permettre de traiter. □

Exemple 2.5 – Trois autres arbitrages attribut / référence.

- **Boutique** : `Commande.nomClient : EString` suffit pour imprimer un bon ; mais pour retrouver « toutes les commandes de M. Idrissi » sans ambiguïté (deux clients homonymes), il faut `Commande.client : Client [1]`.
- **Feu** : la couleur courante est une valeur d'énumération : attribut `etat : Couleur`. Le carrefour, lui, a un nom, une position et plusieurs feux : c'est une classe.
- **Machine à états** : la cible d'une transition doit être *un état du modèle*, pas un texte « Vert » qu'on pourrait mal orthographier : référence `target : State [1]`.

Indice : si deux objets différents peuvent avoir la même valeur sans être « la même chose », ou si l'on veut naviguer vers la chose désignée, c'est une **référence**. □

2.2.2 Multiplicités

Tableau 2.3 – Multiplicités Ecore et leur signification

Notation	`lowerBound`	`upperBound`	Signification	Exemple
`[1]` ou `[1..1]`	1	1	exactement un, obligatoire	`Member.name`
`[0..1]` ou `[?]`	0	1	au plus un, facultatif	`Member.email`, `Loan.book`
`[*]` ou `[0..*]`	0	-1	un nombre quelconque	`Library.book`
`[1..*]` ou `[+]`	1	-1	au moins un	`Commande.lignes`
`[0..3]`	0	3	au plus trois	emprunts simultanés

Exemple 2.6 – Lire des multiplicités dans quatre domaines.

- **Feu** : `Carrefour.feux : FeuDeSignalisation [1..*]` – un carrefour sans feu n'a pas de sens.
- **Familles** : `Family.father : Member [1]`, `Family.sons : Member [0..*]`.

- **Boutique** : `LigneCommande.produit : Produit [1]`, `Client.telephone : EString [0..1]`.
- **Machine à états** : `Transition.event : EString [0..1]` – une transition automatique n'a pas d'événement.

Les multiplicités sont vérifiées par la validation EMF : un `Family` sans père est signalé « *The required feature 'father' must be set* ». □

2.2.3 Composition ou simple référence ?

On choisit une **composition** (`containment = true`) lorsque l'objet cible **fait partie** de l'objet source et n'a pas de sens sans lui : une bibliothèque *contient* ses livres, un diagramme *contient* ses états. On choisit une **simple référence** lorsque l'objet cible existe indépendamment : un emprunt *concerne* un livre, mais ne le contient pas.

Exemple 2.7 – Composition ou référence : quatre cas.

Lien	Choix	Justification
`Library.book` / `Loan.book`	composition / référence	le livre appartient à la bibliothèque ; l'emprunt ne fait que le désigner
`Commande.lignes` / `LigneCommande.produit`	composition / référence	supprimer la commande supprime ses lignes, pas le produit
`Carrefour.feux`	composition	un feu est installé dans un seul carrefour
`StateMachine.states` / `Transition.target`	composition / référence	la transition pointe vers un état déjà contenu par la machine

Test : « si je supprime le conteneur, l'objet doit-il disparaître ? » Oui ⇒ composition. □

Attention. Dans un modèle EMF, un objet qui n'est rattaché à **aucun** conteneur (ni racine de la ressource) n'est pas écrit dans le fichier XMI ; s'il est référencé par un objet enregistré, la sauvegarde échoue (*DanglingHREFException*). Un méta-modèle Ecore a donc presque toujours une **classe racine** qui contient, directement ou non, tous les autres objets.

2.2.4 Énumérations et types de données

Exemple 2.8 – Ajouter une énumération. On veut connaître l'état d'un livre. On crée une `EEnum BookState` avec trois littéraux `AVAILABLE`, `BORROWED`, `LATE`, puis un attribut `Book.state : BookState [1]`. En XMI la valeur s'écrit en clair : `<book title="Le Petit Prince" state="BORROWED"/>`. Dans l'API Java, EMF génère une énumération `BookState` avec `BookState.BORROWED`. □

Exemple 2.9 – Trois autres énumérations. `Couleur` = {ROUGE, ORANGE, VERT} pour le feu ; `StatutCommande` = {EN_COURS, PAYEE, LIVREE, ANNULEE} pour la boutique ; `StateKind` = {INITIAL, NORMAL, FINAL} pour la machine à états (exemple 2.14). Une énumération convient quand la liste des valeurs est **fixe et connue** à la conception ; si les valeurs changent souvent (catégories de produits), on préfère une classe. □

Les types de données prédéfinis les plus utilisés sont `EString`, `EInt`, `EBoolean`, `EDouble`, `EFloat`, `ELong` et `EDate`. On peut définir son propre `EDataType` en l'associant à une classe Java (`instanceClassName`, par exemple `java.math.BigDecimal` pour des montants).

2.2.5 Héritage et classes abstraites

Exemple 2.10 – Factoriser avec l'héritage. Une médiathèque prête des livres et des DVD. Les deux ont un `id` et un `title` ; un livre a un `isbn`, un DVD une `duration`. On crée une `EClass Item` **abstraite** (`abstract = true`) avec `id` et `title`, puis `Book` et `Dvd` dont l'`eSuperTypes` vaut `Item`. La référence `Loan.item : Item [1]` accepte alors un livre *ou* un DVD. En XMI, le type concret est précisé par `xsi:type` (avec la déclaration `xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"`) : `<items xsi:type="media:Dvd" title="Inception"/>`. □

Exemple 2.11 – Trois autres hiérarchies.

- **Familles → Personnes** : `Person` (abstraite, `fullName`) a deux sous-classes `Male` et `Female` (chapitre 7).
- **Relationnel** : `Constraint` (abstraite, `name`) a pour sous-classes `PrimaryKey` et `ForeignKey` (figure 1.9).
- **Boutique** : `Paiement` (abstrait, `montant`, `date`) a pour sous-classes `PaiementCarte` (`quatreDerniersChiffres`) et `Virement` (`reference`).

Dans les trois cas, la classe mère est abstraite : on ne crée jamais « une personne » ou « un paiement » sans préciser lequel. □

2.2.6 Références opposées

Exemple 2.12 – Navigation dans les deux sens. Pour savoir quels emprunts concernent un livre, on ajoute `Book.loans : Loan [0..*]` et on déclare `eOpposite` entre `Loan.book` et `Book.loans`. Quand le programme exécute `loan.setBook(b)`, EMF ajoute automatiquement `loan` à `b.getLoans()` : les deux côtés restent cohérents. En XMI, les deux côtés d'une paire non contenante sont enregistrés ; pour alléger le fichier, on marque souvent l'un d'eux `transient` (il est alors recalculé au chargement). Lorsque l'un des côtés est une composition, seul le côté conteneur est écrit (l'imbrication suffit). □

Exemple 2.13 – Trois autres paires opposées. `Family.father` ↔ `Member.familyFather` (composition ↔ conteneur) ; `State.outgoing` ↔ `Transition.source` ; `Client.commandes` ↔ `Commande.client`. Dans chaque paire, les multiplicités doivent être cohérentes : si `Commande.client` vaut `[1]`, `Client.commandes` vaut `[0..*]`. □

2.3 Exemples complets

Exemple 2.14 – Méta-modèle d'un langage de machines à états. Énoncé : « Une machine possède un nom et contient des états et des transitions. Un état a un nom ; il est initial, final ou normal. Une transition relie un état source à un état cible et porte un événement (texte). »

EClass	Propriétés
`StateMachine`	`name : EString [1]` ; `states : State [0..*]` (containment) ; `transitions : Transition [0..*]` (containment)
`State`	`name : EString [1]` ; `kind : StateKind [1]` ; `outgoing : Transition [0..*]` (opposée de `source`)
`Transition`	`event : EString [0..1]` ; `source : State [1]` (opposée de `outgoing`) ; `target : State [1]`
`StateKind` (EEnum)	`INITIAL`, `NORMAL`, `FINAL`

Le modèle du feu de signalisation conforme à ce méta-modèle :

```
<sm:StateMachine xmi:version="2.0" xmlns:xmi="http://www.omg.org/XMI"
    xmlns:sm="http://ensias.ma/sm/1.0" name="Feu">
  <states name="Rouge"  kind="INITIAL" outgoing="//@transitions.0"/>
  <states name="Vert"   kind="NORMAL"  outgoing="//@transitions.1"/>
  <states name="Orange" kind="NORMAL"  outgoing="//@transitions.2"/>
  <transitions event="after(25s)" source="//@states.0" target="//@states.1"/>
  <transitions event="after(20s)" source="//@states.1" target="//@states.2"/>
  <transitions event="after(5s)"  source="//@states.2" target="//@states.0"/>
</sm:StateMachine>
```

La règle « exactement un état initial » ne peut pas être exprimée par les multiplicités : elle sera écrite en OCL (chapitre 3). □

Exemple 2.15 – Un DSL de listes de tâches. Énoncé : « Un projet contient des listes ; une liste contient des tâches. Une tâche a un titre, une priorité (BASSE, NORMALE, HAUTE), une échéance facultative, un état terminé ou non, et peut dépendre d'autres tâches. »

- `Project` (`name`) ♦→ `lists : TaskList [0..*]` ;
- `TaskList` (`title`) ♦→ `tasks : Task [0..*]` ;
- `Task : title : EString [1]`, `priority : Priority [1]`, `dueDate : EDate [0..1]`, `done : EBoolean [1]`, `dependsOn : Task [0..*]` (simple référence, réflexive) ;
- `Priority` (EEnum) : `BASSE`, `NORMALE`, `HAUTE`.

`dependsOn` n'est **pas** une composition : une tâche peut dépendre d'une tâche d'une autre liste sans la contenir. □

Exemple 2.16 – Le méta-modèle d'une boutique en ligne.

EClass	Propriétés
`Boutique`	`nom : EString [1]` ; ♦ `clients : Client [0..*]` ; ♦ `produits : Produit [0..*]`
`Client`	`email : EString [1] (iD = true)` ; `nom : EString [1]` ; ♦ `commandes : Commande [0..*]`
`Commande`	`numero : EInt [1]` ; `date : EDate [1]` ; `statut : StatutCommande [1]` ; ♦ `lignes : LigneCommande [1..*]`
`LigneCommande`	`quantite : EInt [1]` ; `produit : Produit [1]` (simple référence)
`Produit`	`reference : EString [1]` ; `libelle : EString [1]` ; `prix : EDouble [1]`

Ici la commande est contenue par son client (`Client.commandes`) : son conteneur unique est donc le client, et `Commande.client` peut être obtenu par `eContainer()` ou déclaré comme opposée. □

Exemple 2.17 – Lire un modèle avec l'API réflexive. Sans code généré, l'API réflexive d'EMF lit n'importe quel modèle :

```
EObject racine = resource.getContents().get(0);
EClass c = racine.eClass();                                    // la méta-classe
System.out.println("Racine de type " + c.getName());
for (EAttribute a : c.getEAllAttributes())
    System.out.println(a.getName() + " = " + racine.eGet(a)); // valeur de l'attribut
```

Sur `ensias.xmi`, on obtient « Racine de type Library » puis « name = ENSIAS ». C'est cette API, générique, qu'utilisent les outils comme Acceleo, ATL ou Sirius. □

2.4 Exercices

Les corrigés figurent en annexe A.

Exercice 2.1 – Gestion de cours. Une école propose des `Cours` (code, intitulé, nombre de crédits). Chaque cours est assuré par un `Enseignant` (nom, grade : ASSISTANT, PROFESSEUR) et suivi par des `Etudiant` (CNE, nom). Un étudiant obtient une `Note` (valeur réelle) par cours suivi. Construire le méta-modèle Ecore (classe racine `Ecole`), en précisant pour chaque référence sa multiplicité et si c'est une composition.

Exercice 2.2 – Multiplicités. Donner `lowerBound` et `upperBound` pour : (a) un livre a au moins un auteur ; (b) un adhérent a au plus 3 emprunts ; (c) une personne a un numéro de téléphone facultatif ; (d) une commande a au moins une ligne.

Exercice 2.3 – XMI. Écrire le fichier XMI d'une bibliothèque « ENSIAS » conforme à `library.ecore`, avec deux livres, un membre et un emprunt du second livre par ce membre.

Exercice 2.4 – Trouver les erreurs. Le fichier suivant se veut conforme à `library.ecore`. Trouver les trois erreurs principales.

```
<library:Library xmlns:library="http://fr.obeo.library/2.0" name="ENSIAS">
  <book id="B1" title="Le Petit Prince" available="oui"/>
  <loan loanDate="2026-11-02" book="//@book.3"/>
</library:Library>
```

Exercice 2.5 – API générée. À l'aide de l'API générée (`LibraryFactory`), écrire le code Java qui crée un membre « Sara », un livre « Le Petit Prince » et un emprunt qui les relie, les range dans une bibliothèque `lib`, puis affiche le nombre de livres de la bibliothèque.

Exercice 2.6 – Héritage. Dans le méta-modèle de l'exercice 2.1, on distingue les étudiants `Licence` et `Master` (ce dernier a un sujet de mémoire). Modifier le méta-modèle.

2.5 Synthèse

Tableau 2.4 – Synthèse du chapitre 2

Ecore	M3 d'Eclipse : `EPackage`, `EClass`, `EAttribute`, `EReference`, `EDataType`, `EEnum`.
Attribut / référence	valeur simple / lien vers un objet ayant une identité.
Composition	`containment = true` ; tout objet a au plus un conteneur ; une classe racine.
Multiplicités	`lowerBound`, `upperBound` (`-1` = *).
`.ecore` / `.xmi`	méta-modèle / modèle conforme, liés par le `nsURI`.
`.genmodel`	génère l'API Java (interfaces, `Impl`, `Factory`, `Package`).
Sirius	syntaxe graphique (`.odesign`, `.aird`).

2.6 Quiz

Répondez aux questions suivantes. Les corrigés commentés figurent en annexe A.

Q1. Dans Ecore, une propriété dont le type est une `EClass` est :

a) un `EAttribute`

b) une `EReference`

c) un `EDataType`

d) une `EOperation`

Q2. Que signifie `containment = true` sur une `EReference` ?

a) La référence est obligatoire

b) La référence est une composition : l'objet cible est contenu

c) La référence est multiple

d) La référence a une opposée

Q3. Quelle propriété de l'`EPackage` relie un fichier `.xmi` à son méta-modèle ?

a) `name`

b) `nsPrefix`

c) `nsURI`

d) `eClassifiers`

Q4. Le fichier `.genmodel` sert à :

a) dessiner le diagramme du méta-modèle

b) générer l'API Java du méta-modèle

c) valider les contraintes OCL

d) stocker les instances

Q5. Quelle valeur de `upperBound` représente « * » ?

a) 0

b) 1

c) -1

d) 999

Q6. Un objet d'un modèle EMF qui n'est contenu par aucune composition et n'est pas racine :

a) est enregistré à la racine
b) n'est pas enregistré dans le fichier XMI
c) provoque une erreur de compilation
d) devient une référence opposée

Q7. Dans l'API générée, comment crée-t-on un livre ?

a) `new Book()`
b) `LibraryFactory.eINSTANCE.createBook()`
c) `LibraryPackage.createBook()`
d) `EcoreUtil.create("Book")`

Q8. Que permet `eOpposite` ?

a) d'inverser une composition
b) de maintenir automatiquement deux références qui se répondent
c) de rendre une référence obligatoire
d) d'hériter d'une classe

Q9. Dans un XMI, comment indique-t-on qu'un élément d'une référence de type `Item` est en réalité un `Dvd` ?

a) `type="Dvd"`
b) `xsi:type="media:Dvd"`
c) `class="Dvd"`
d) c'est impossible

Q10. Pour représenter l'état d'un livre (disponible, emprunté, en retard), quel concept Ecore utiliser ?

Q11. Citer deux différences entre un attribut `author : EString` et une classe `Author`.

Chapitre 3

Le langage de contraintes OCL

Résumé

*Un diagramme de classes décrit la structure d'un système, mais il ne peut pas exprimer toutes les règles métier : « un employé a au moins 18 ans », « le chef d'un département en est membre », « le budget d'un département ne dépasse pas le capital de l'entreprise »… **OCL** (*Object Constraint Language*) est le langage textuel, formel et sans effet de bord de l'OMG qui complète UML et les méta-modèles MOF/Ecore. Ce chapitre présente OCL pas à pas sur un modèle simple (une entreprise, ses départements et ses employés) et sur un diagramme d'objets dont toutes les valeurs sont calculées : pour chaque concept – attribut, énumération, opération, association de multiplicité 1, 0..1 ou *, association réflexive, composition, héritage – on donne l'expression OCL et sa **valeur**. Chaque construction est ensuite reprise sur d'autres modèles (bibliothèque, boutique en ligne, feu de signalisation). On étudie enfin les invariants, les pré/post-conditions, les définitions (`def`, `let`, `derive`, `init`, `body`), les collections et l'utilisation d'OCL au niveau méta (M2) et dans les outils Eclipse.*

Objectifs d'apprentissage

À l'issue de ce chapitre, vous serez capable de :

- expliquer pourquoi un diagramme de classes doit être complété par des contraintes ;
- écrire une expression OCL dans un **contexte** et utiliser `self` ;
- naviguer vers un attribut, une énumération, une opération, une association (1, 0..1, *), une association réflexive et une composition ;
- tester et convertir un type (`oclIsKindOf`, `oclIsTypeOf`, `oclAsType`, `oclIsUndefined`) ;
- écrire des invariants, des pré/post-conditions (`@pre`, `result`) et des définitions ;
- manipuler les collections (`select`, `reject`, `collect`, `forAll`, `exists`, `iterate`…) et **calculer** la valeur d'une expression sur un diagramme d'objets.

Contenu du chapitre

3.1 Pourquoi OCL ? . . . 32
3.1.1 Les limites d'un diagramme de classes . . . 32
3.1.2 Où utilise-t-on OCL ? . . . 33
3.2 Le modèle d'étude . . . 33
3.2.1 Le diagramme d'objets d'étude . . . 33
3.2.2 Trois autres modèles pour s'entraîner . . . 35

3.3 Contexte et `self` . . . 35
3.4 Accéder aux éléments du modèle . . . 36
3.4.1 Les attributs . . . 36
3.4.2 Les énumérations . . . 36
3.4.3 Les opérations . . . 37
3.4.4 Les associations de multiplicité 1 . . . 37
3.4.5 Les associations de multiplicité 0..1 . . . 38
3.4.6 Les associations de multiplicité * . . . 38
3.4.7 Navigation enchaînée et `collect` implicite . . . 39
3.4.8 Les associations réflexives . . . 39
3.4.9 Les compositions . . . 40
3.4.10 L'héritage et les opérations sur les types . . . 41
3.5 Les types de base et les expressions . . . 42
3.6 Les collections . . . 42
3.6.1 Les quatre types de collections . . . 42
3.6.2 Opérations de base . . . 43
3.6.3 Opérations itératives (*itérateurs*) . . . 43
3.6.4 L'itérateur général `iterate` . . . 44
3.6.5 Opérations ensemblistes . . . 44
3.6.6 `allInstances` . . . 45
3.7 Spécifier des opérations : pré- et post-conditions . . . 45
3.8 Définitions : `def`, `body`, `derive`, `init` . . . 46
3.9 OCL au niveau méta et dans les outils . . . 46
3.10 Méthode et pièges fréquents . . . 47
3.11 Exercices . . . 48
3.12 Synthèse . . . 49
3.13 Quiz . . . 49

3.1 Pourquoi OCL ?

3.1.1 Les limites d'un diagramme de classes

Un diagramme de classes fixe les classes, leurs attributs, leurs associations et des multiplicités. Il ne dit rien, par exemple, sur :

- les **valeurs permises** d'un attribut (« l'âge est compris entre 18 et 65 ») ;
- les **relations entre associations** (« le chef d'un département fait partie de ses membres ») ;
- les règles **d'agrégation** (« la somme des budgets ne dépasse pas le capital ») ;
- l'**effet** d'une opération (« après `augmenterSalaire(t)`, le salaire a augmenté de t %») ;

- les **valeurs calculées** (« le revenu d'un cadre est son salaire plus sa prime »).

Écrites en langue naturelle, ces règles sont ambiguës et ne peuvent pas être vérifiées par un outil. OCL permet de les écrire de façon **précise** et **vérifiable**.

Exemple 3.1 – Ce que le diagramme ne dit pas, dans trois domaines.

- **Bibliothèque** : « un adhérent a au plus 3 emprunts *en cours* » (une multiplicité 0..3 compterait aussi l'historique).
- **Boutique** : « le montant total d'une commande est la somme de ses lignes ».
- **Feu** : « la durée du vert est comprise entre 10 et 90 secondes ».

Chacune de ces règles devient, plus loin dans ce chapitre, une expression OCL d'une ligne. □

Définition 3.1 (OCL – Object Constraint Language). *Langage normalisé par l'OMG (version 2.4, 2014) pour écrire des* ***contraintes*** *et des* ***requêtes*** *sur des modèles UML ou sur tout modèle conforme à un méta-modèle MOF/Ecore. OCL est* ***typé***, ***déclaratif*** *et* ***sans effet de bord*** *: l'évaluation d'une expression ne modifie jamais le modèle, elle renvoie seulement une valeur.*

3.1.2 Où utilise-t-on OCL ?

- **Au niveau M1** : règles métier d'un modèle (invariants, pré/post-conditions) ;
- **au niveau M2** : règles de bonne formation d'un méta-modèle (« une machine à états a un seul état initial ») ;
- **dans les outils IDM** : OCLinEcore (contraintes dans un `.ecore`), requêtes dans Sirius, expressions des templates **Acceleo** (chapitre 5) et des règles **ATL** (chapitre 7).

3.2 Le modèle d'étude

La présentation pas à pas s'appuie sur le modèle de la figure 3.1.

- Une `Entreprise` (`nom`, `capital`) **emploie** des `Personne` (rôles `employeur` 1 / `employes` 0..*) et **contient** (composition) au moins un `Departement`.
- Un `Departement` (`nom`, `budget`) a des `membres` (0..*) et au plus un `chef` (0..1).
- Une `Personne` (`nom`, `prenom`, `age`, `salaire`, `sexe`) appartient à au plus un `departement` ; elle a au plus un `manager` et des `subordonnes` (association **réflexive**) et au plus un `conjoint` (association réflexive).
- Un `Cadre` est une `Personne` qui reçoit une `prime` (**héritage**).
- `Sexe` est une **énumération** {HOMME, FEMME}.

3.2.1 Le diagramme d'objets d'étude

Toutes les valeurs données dans ce chapitre sont calculées sur le diagramme d'objets suivant (tableau 3.1) : une entreprise `e1` « TechMaroc » (capital 1 000 000), deux départements et cinq personnes.

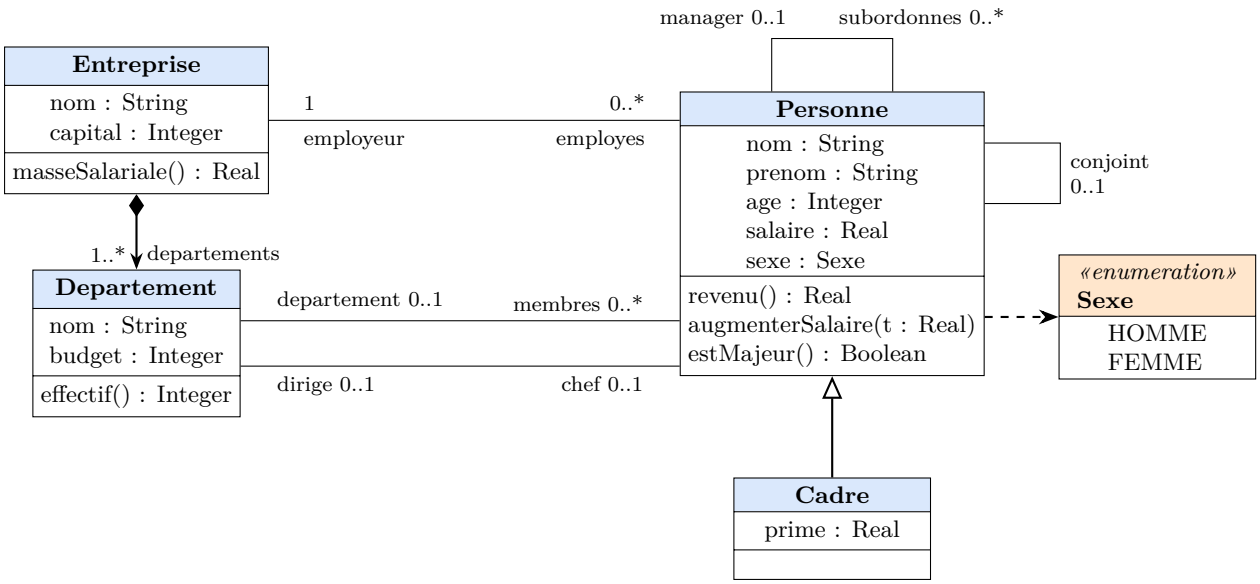


Figure 3.1 – Modèle d'étude OCL : entreprise, départements et employés

Tableau 3.1 – Diagramme d'objets d'étude (niveau M0)

Objet	Classe	Prénom Nom	Âge	Sexe	Salaire	Prime	Dépt	Manager	Conjoint
`p1`	Cadre	Sara Amrani	35	F	15000	3000	R&D	–	–
`p2`	Personne	Youssef Bennani	28	H	9000	–	R&D	p1	–
`p3`	Cadre	Nadia Alaoui	42	F	12000	2000	Ventes	–	–
`p4`	Personne	Omar Tazi	23	H	6000	–	Ventes	p3	p5
`p5`	Personne	Salma Tazi	24	F	7000	–	R&D	p1	p4

Objet	Département	Budget	Membres	Chef
`d1`	R&D	500000	p1, p2, p5	p1
`d2`	Ventes	200000	p3, p4	p3

`e1.employes` = {p1, p2, p3, p4, p5} ; `e1.departements` = {d1, d2}.

3.2.2 Trois autres modèles pour s'entraîner

Les mêmes constructions sont reprises sur trois petits modèles :

- **Bibliothèque** : `Adherent(nom, age)` – `emprunts : Emprunt [0..*]` ; `Emprunt(dateDebut, dateRetour [0..1])` – `livre : Livre [1]` ; `Livre(isbn, titre, disponible : Boolean)` ; `Bibliotheque` contient `livres` et `adherents`.
- **Boutique** : `Client(nom, email)` – `commandes : Commande [0..*]` ; `Commande(numero, statut : StatutCommande, total : Real)` contient `lignes : LigneCommande [1..*]` ; `LigneCommande(quantite)` – `produit : Produit [1]` ; `Produit(reference, prix : Real, stock : Integer)`.
- **Feu** : `Carrefour(nom)` contient `feux : Feu [1..*]` ; `Feu(id, etat : Couleur, dureeVert, dureeOrange, dureeRouge : Integer)`.

3.3 Contexte et `self`

Toute expression OCL est écrite dans un **contexte**, qui indique sur quel type (ou quelle opération) elle porte. Dans l'expression, `self` désigne l'**instance courante** du contexte. Une contrainte peut recevoir un **nom**, utile dans les messages d'erreur des outils.

```
context Personne inv ageValide:      -- contexte : la classe Personne ; nom : ageValide
  self.age >= 18                     -- self = la personne évaluée

context Personne inv:                -- contrainte sans nom ; self est implicite
  age >= 18
```

Tableau 3.2 – Les formes de contexte OCL

Forme	Usage	Exemple
`context C inv`	invariant de la classe C	`context Personne inv: age >= 18`
`context C::op(...) pre/post`	pré/post-condition d'une opération	`Personne::augmenterSalaire(t)`
`context C::op(...) body`	corps d'une opération de requête	`context Personne::revenu() : Real body: ...`
`context C::attr : T init/derive`	valeur initiale / dérivée d'un attribut	`context Departement::masse : Real derive: ...`
`context C def`	définir un attribut ou une opération auxiliaire	`context Personne def: nomComplet : String = ...`

Exemple 3.2 – Choisir le bon contexte.

- « Le budget d'un département est positif » parle d'*un* département : `context Departement inv: self.budget > 0`.
- « Une commande contient au moins une ligne » : `context Commande inv: self.lignes->notEmpty()`.
- « Un feu reste au vert au moins 10 s » : `context Feu inv: self.dureeVert >= 10`.
- « Les ISBN des livres d'une bibliothèque sont uniques » parle de *la collection* des livres : le contexte naturel est `Bibliotheque`, pas `Livre`.

□

3.4 Accéder aux éléments du modèle

3.4.1 Les attributs

Un attribut se lit avec le point : `self.attribut`. Le résultat a le type de l'attribut.

Exemple 3.3 – Lire des attributs.

```
p2.age                          -- 28           (Integer)
p2.prenom                       -- 'Youssef'    (String)
p1.salaire                      -- 15000.0      (Real)
p2.prenom + ' ' + p2.nom        -- 'Youssef Bennani'
d1.budget > 300000              -- true
```

Contraintes sur les attributs :

```
context Personne    inv ageLegal:     self.age >= 18 and self.age <= 65
context Personne    inv salairePos:   self.salaire > 0
context Personne    inv nomNonVide:   self.nom.size() > 0
context Entreprise  inv capitalMin:   self.capital >= 10000
```

Sur le diagramme d'étude, tous ces invariants sont vrais. □

Exemple 3.4 – Invariants sur les attributs dans les autres modèles.

```
context Feu           inv vertBorne:   self.dureeVert >= 10 and self.dureeVert <= 90
context Feu           inv cycleMax:    self.dureeVert + self.dureeOrange + self.dureeRouge <=
    180
context LigneCommande inv qtePositive: self.quantite >= 1
context Produit       inv prixPositif: self.prix > 0 and self.stock >= 0
```

Pour un feu (vert 20, orange 5, rouge 25) : `vertBorne` et `cycleMax` ($50 \leq 180$) sont vrais. □

3.4.2 Les énumérations

Une valeur d'énumération s'écrit `NomEnum::LITTERAL`. On compare avec `=` ou `<>`.

Exemple 3.5 – Utiliser l'énumération `Sexe`.

```
p1.sexe                                         -- Sexe::FEMME
p2.sexe = Sexe::HOMME                           -- true
p4.sexe <> Sexe::FEMME                          -- true
e1.employes->select(p | p.sexe = Sexe::FEMME)->size()      -- 3

-- Chaque département compte au moins une femme et au moins un homme
context Departement inv mixite:
  self.membres->exists(p | p.sexe = Sexe::FEMME) and
  self.membres->exists(p | p.sexe = Sexe::HOMME)
```

`mixite` est vraie pour `d1` (p1, p5 / p2) et pour `d2` (p3 / p4). □

Exemple 3.6 – Énumérations dans les autres modèles.

```
-- Une commande livrée a un total strictement positif
context Commande inv livreeComplete:
```

```
  self.statut = StatutCommande::LIVREE implies self.total > 0

-- Au plus un feu vert à la fois dans un carrefour
context Carrefour inv unSeulVert:
  self.feux->select(f | f.etat = Couleur::VERT)->size() <= 1
```

□

3.4.3 Les opérations

On peut appeler dans une expression OCL les opérations **de requête** du modèle (celles qui ne modifient pas l'état, marquées `{query}` en UML). Les opérations qui modifient l'état (`augmenterSalaire`) ne peuvent pas être appelées dans une contrainte.

Exemple 3.7 – Appeler des opérations de requête.

```
p1.revenu()                    -- 18000.0  (salaire + prime d'un cadre)
p2.revenu()                    -- 9000.0
p4.estMajeur()                 -- true
d1.effectif()                  -- 3
e1.masseSalariale()            -- 49000.0

context Personne inv revenuPositif: self.revenu() > 0
```

□

3.4.4 Les associations de multiplicité 1

On navigue d'un objet vers un autre en utilisant le **nom de rôle** de l'extrémité opposée de l'association. Si la multiplicité est **1**, le résultat est **un objet**.

Exemple 3.8 – Naviguer vers l'employeur.

```
p3.employeur                   -- e1
p3.employeur.nom               -- 'TechMaroc'
p3.employeur.capital           -- 1000000

-- Le salaire d'un employé est au plus égal à 2 % du capital de son employeur
context Personne inv salaireRaisonnable:
  self.salaire <= self.employeur.capital * 0.02
```

Pour p1 : $15000 \leq 20000$, l'invariant est vrai. □

Exemple 3.9 – Multiplicité 1 dans les autres modèles.

```
-- Le sous-total d'une ligne : quantité * prix du produit référencé
context LigneCommande def: sousTotal : Real = self.quantite * self.produit.prix
-- On ne commande pas plus que le stock
context LigneCommande inv stockSuffisant: self.quantite <= self.produit.stock
-- Le titre du livre d'un emprunt n'est jamais vide
context Emprunt inv titreConnu: self.livre.titre.size() > 0
```

□

À retenir. Si l'extrémité d'association n'a pas de nom de rôle, OCL utilise le **nom de la classe cible en minuscule** (`self.entreprise`). Il est préférable de toujours nommer les rôles.

3.4.5 Les associations de multiplicité 0..1

Si la multiplicité est **0..1**, le résultat est un objet **ou** une valeur **indéfinie** (`null`). On teste l'absence avec `oclIsUndefined()` et on protège la navigation avec `implies` ou `if`.

Exemple 3.10 – Manager, département, conjoint.

```
p2.manager                      -- p1
p2.manager.prenom               -- 'Sara'
p1.manager                      -- null (indéfini)
p1.manager.oclIsUndefined()     -- true
p4.conjoint.prenom              -- 'Salma'
p5.departement.nom              -- 'R&D'

-- Un employé a le même employeur que son manager (s'il en a un)
context Personne inv memeEmployeur:
  not self.manager.oclIsUndefined() implies self.manager.employeur = self.employeur

-- Le conjoint de mon conjoint, c'est moi
context Personne inv conjointSymetrique:
  not self.conjoint.oclIsUndefined() implies self.conjoint.conjoint = self
```

□

Exemple 3.11 – Multiplicité 0..1 dans les autres modèles.

```
-- Un emprunt rendu l'est après avoir commencé
context Emprunt inv retourApresDebut:
  not self.dateRetour.oclIsUndefined() implies self.dateRetour >= self.dateDebut

-- Un client sans téléphone a au moins un e-mail renseigné
context Client inv contact:
  self.telephone.oclIsUndefined() implies self.email.size() > 0
```

□

Attention. Sans garde (`implies`), l'expression `self.manager.employeur` évaluée sur `p1`, qui n'a pas de manager, donne une valeur *invalide* : l'invariant ne peut pas être vérifié.

3.4.6 Les associations de multiplicité *

Si la multiplicité est supérieure à 1, le résultat est une **collection** : un `Set` par défaut, un `OrderedSet` si l'extrémité est marquée `{ordered}`. On lui applique des opérations avec la **flèche** `->`.

Exemple 3.12 – Employés, membres, subordonnés.

```
e1.employes                     -- Set{p1, p2, p3, p4, p5}
e1.employes->size()             -- 5
d1.membres                      -- Set{p1, p2, p5}
p1.subordonnes                  -- Set{p2, p5}
p2.subordonnes->isEmpty()       -- true
```

```
d2.membres->includes(p4)        -- true

context Departement inv auMoinsUnMembre:   self.membres->notEmpty()
context Entreprise  inv auPlus1000:        self.employes->size() <= 1000
```

□

Exemple 3.13 – Multiplicité * dans les autres modèles.

```
-- Au plus 3 emprunts EN COURS (sans date de retour) par adhérent
context Adherent inv quota:
  self.emprunts->select(e | e.dateRetour.oclIsUndefined())->size() <= 3

-- Un carrefour a entre 2 et 8 feux
context Carrefour inv nbFeux: self.feux->size() >= 2 and self.feux->size() <= 8
```

□

3.4.7 Navigation enchaînée et `collect` implicite

On peut enchaîner les navigations. Quand on navigue **à partir d'une collection**, OCL applique implicitement un `collect` : le résultat est un `Bag` (doublons possibles).

Exemple 3.14 – Chaînes de navigation.

```
p2.manager.departement.nom      -- 'R&D'
p4.manager.employeur.nom        -- 'TechMaroc'
d1.membres.salaire              -- Bag{15000, 9000, 7000}
                                --  = d1.membres->collect(m | m.salaire)
d1.membres.salaire->sum()       -- 31000
e1.departements.membres         -- Bag{p1, p2, p5, p3, p4} (aplati)
e1.employes.departement.nom     -- Bag{'R&D', 'R&D', 'Ventes', 'Ventes', 'R&D'}
e1.employes.departement.nom->asSet()   -- Set{'R&D', 'Ventes'}
```

□

Exemple 3.15 – Navigation enchaînée dans la boutique et la bibliothèque.

```
-- Le total d'une commande est la somme des sous-totaux de ses lignes
context Commande inv totalCoherent:
  self.total = self.lignes->collect(l | l.quantite * l.produit.prix)->sum()

-- Les titres des livres empruntés par un adhérent
a.emprunts.livre.titre                      -- Bag de String
-- Tous les produits jamais commandés par un client
c.commandes.lignes.produit->asSet()         -- Set(Produit)
```

Pour une commande de 2 cahiers à 15 DH et 1 stylo à 10 DH : total attendu `40.0`. □

3.4.8 Les associations réflexives

Une association réflexive relie une classe à elle-même (`manager`/`subordonnes`, `conjoint`). Les rôles distinguent les deux extrémités.

Exemple 3.16 – Hiérarchie et conjoints.

```
p5.manager.subordonnes          -- Set{p2, p5}
p5.manager.subordonnes->excluding(p5)     -- Set{p2}  (les collègues de p5)

-- On n'est pas son propre manager
context Personne inv pasSonManager:   self.manager <> self

-- On n'est pas son propre conjoint, et un conjoint est majeur
context Personne inv conjointValide:
  not self.conjoint.oclIsUndefined() implies
      (self.conjoint <> self and self.conjoint.age >= 18)

-- Un subordonné gagne moins que son manager
context Personne inv salaireHierarchie:
  self.subordonnes->forAll(s | s.salaire < self.salaire)
```

□

Exemple 3.17 – Réflexivité ailleurs : tâches et transitions.

```
-- Une tâche ne dépend pas d'elle-même (DSL de tâches, chapitre 2)
context Task inv pasAutoDependance: self.dependsOn->excludes(self)
-- Une tâche terminée ne dépend que de tâches terminées
context Task inv dependancesFinies: self.done implies self.dependsOn->forAll(t | t.done)
-- Pas de boucle immédiate entre deux états (A -> B et B -> A sur le même événement)
context Transition inv pasAllerRetour:
  self.target.outgoing->forAll(t | t.target = self.source implies t.event <> self.event)
```

□

3.4.9 Les compositions

Une composition se navigue comme une association ordinaire. Les invariants permettent d'exprimer les liens entre la composition et les autres associations.

Exemple 3.18 – Départements de l'entreprise.

```
e1.departements                 -- Set{d1, d2}
e1.departements.budget->sum()   -- 700000

-- La somme des budgets ne dépasse pas le capital
context Entreprise inv budgetsCouverts:
  self.departements.budget->sum() <= self.capital

-- Les membres d'un département sont des employés de l'entreprise qui le contient
context Entreprise inv membresEmployes:
  self.departements->forAll(d | self.employes->includesAll(d.membres))

-- Le chef d'un département en est membre
context Departement inv chefMembre:
  not self.chef.oclIsUndefined() implies self.membres->includes(self.chef)

-- Deux départements d'une entreprise ont des noms différents
context Entreprise inv nomsDeptUniques:
  self.departements->isUnique(d | d.nom)
```

□

Exemple 3.19 – Compositions dans les autres modèles.

```
context Bibliotheque inv isbnUniques:   self.livres->isUnique(l | l.isbn)
context Commande     inv produitUneFois: self.lignes->isUnique(l | l.produit)
context Carrefour    inv idsUniques:    self.feux->isUnique(f | f.id)
```

Chaque fois, la règle d'unicité est posée sur le **conteneur**, qui voit toute la collection. □

3.4.10 L'héritage et les opérations sur les types

Une instance de `Cadre` est aussi une `Personne` : elle possède tous ses attributs et associations. Pour accéder à `prime` depuis le type `Personne`, il faut tester puis convertir le type.

Tableau 3.3 – Opérations sur les types

Opération	Signification	Sur p1 (Cadre)	Sur p2
`oclIsTypeOf(Personne)`	type **exact**	false	true
`oclIsKindOf(Personne)`	type ou sous-type	true	true
`oclIsKindOf(Cadre)`	est un cadre ?	true	false
`oclAsType(Cadre).prime`	conversion puis accès	3000	invalide
`oclIsUndefined()`	valeur absente ?	false	false

Exemple 3.20 – Contraintes avec héritage.

```
context Cadre inv primePositive:        self.prime > 0       -- prime visible directement
context Cadre inv cadreAge:             self.age >= 25       -- age hérité de Personne

-- Le chef d'un département est un cadre
context Departement inv chefCadre:
  not self.chef.oclIsUndefined() implies self.chef.oclIsKindOf(Cadre)

-- Les cadres de l'entreprise
e1.employes->select(p | p.oclIsKindOf(Cadre))                  -- Set{p1, p3}
e1.employes->select(p | p.oclIsKindOf(Cadre))
          ->collect(p | p.oclAsType(Cadre).prime)->sum()        -- 5000
Cadre.allInstances()->size()                                    -- 2
```

□

Exemple 3.21 – Héritage dans les autres modèles.

```
-- Médiathèque : un DVD emprunté dure au moins 1 minute
context Loan inv dvdValide:
  self.item.oclIsKindOf(Dvd) implies self.item.oclAsType(Dvd).duration > 0
-- Boutique : un paiement par carte porte 4 chiffres
context PaiementCarte inv quatreChiffres: self.quatreDerniersChiffres.size() = 4
-- Machine à états : un état final n'a aucune transition sortante
context State inv finalSansSortie:
  self.kind = StateKind::FINAL implies self.outgoing->isEmpty()
```

□

Tableau 3.4 – Types de base d'OCL et exemples évalués

Type	Opérations	Exemples (résultat)
`Boolean`	`and or xor not implies, =, <>`	`true implies false` (false) ; `false implies x` (true)
`Integer`	`+ - * /, div, mod, abs(), max(n), min(n)`	`7 div 2` (3) ; `7 mod 2` (1) ; `7 / 2` (3.5) ; `(-4).abs()` (4)
`Real`	`+ - * /, floor(), round(), max, min`	`3.7.floor()` (3) ; `3.5.round()` (4)
`String`	`size(), concat(s), +, substring(i,j), toUpper(), toLower(), indexOf(s)`	`'Sara'.size()` (4) ; `'Sara'.substring(1,2)` ('Sa', indices à partir de 1) ; `'ocl'.toUpper()` ('OCL')

3.5 Les types de base et les expressions

L'implication. `a implies b` est faux seulement si `a` est vrai et `b` faux. C'est l'opérateur naturel des règles conditionnelles : « *si* la personne a un conjoint, *alors* ... ».

La conditionnelle. `if c then e1 else e2 endif` est une **expression** (elle renvoie une valeur) : la branche `else` est **obligatoire**.

Exemple 3.22 – Expressions conditionnelles.

```
if p4.age < 25 then 'junior' else 'senior' endif           -- 'junior'
if p1.oclIsKindOf(Cadre) then p1.oclAsType(Cadre).prime
                         else 0 endif                       -- 3000
-- Frais de port : gratuits au-delà de 500 DH
if c.total >= 500 then 0 else 30 endif                      -- commande de 40 DH : 30

-- Les mineurs n'ont pas de conjoint
context Personne inv: self.age < 18 implies self.conjoint.oclIsUndefined()
```

□

La variable locale `let`. `let x : T = e1 in e2` nomme une sous-expression pour la réutiliser.

```
context Departement inv budgetSuffisant:
  let masse : Real = self.membres.salaire->sum() * 12 in
  masse <= self.budget
-- d1 : 31000 * 12 = 372000 <= 500000  -> vrai
-- d2 : 18000 * 12 = 216000 >  200000  -> FAUX : d2 viole l'invariant
```

Ce contre-exemple montre l'intérêt d'évaluer une contrainte sur des données : la règle est raisonnable, mais le diagramme d'étude ne la respecte pas pour les Ventes.

3.6 Les collections

3.6.1 Les quatre types de collections

Littéraux : `Set{1, 2, 3}`, `Sequence{1..5}`, `Bag{'a', 'a', 'b'}`. Conversions : `asSet()`, `asBag()`, `asSequence()`, `asOrderedSet()`.

Tableau 3.5 – Types de collections OCL

Type	Doublons	Ordonné	Obtenu par
`Set`	non	non	navigation vers une extrémité *
`OrderedSet`	non	oui	extrémité * `{ordered}`
`Bag`	oui	non	navigation à partir d'une collection, `collect`
`Sequence`	oui	oui	`collect` sur un ensemble ordonné, `sortedBy`

3.6.2 Opérations de base

Exemple 3.23 – Opérations de base sur `e1.employes`.

```
e1.employes->size()                    -- 5
e1.employes->isEmpty()                 -- false
e1.employes->notEmpty()                -- true
e1.employes->includes(p3)              -- true
d1.membres->excludes(p3)               -- true
e1.employes->includesAll(d2.membres)   -- true
e1.employes.salaire->sum()             -- 49000
e1.employes.age->max()                 -- 42
e1.employes.age->min()                 -- 23
e1.employes.sexe->count(Sexe::FEMME)   -- 3
Sequence{3, 1, 2}->first()             -- 3
Sequence{3, 1, 2}->at(2)               -- 1
```

□

3.6.3 Opérations itératives (*itérateurs*)

Un itérateur parcourt les éléments d'une collection avec une variable : `coll->op(x | expr)`. La variable peut être omise (`coll->select(age > 30)`), mais la nommer rend l'expression plus claire.

Tableau 3.6 – Itérateurs OCL évalués sur le diagramme d'étude

Expression	Signification	Résultat
`e1.employes->select(p | p.age > 30)`	filtre	`Set{p1, p3}`
`e1.employes->reject(p | p.age > 30)`	filtre inverse	`Set{p2, p4, p5}`
`e1.employes->collect(p | p.prenom)`	image	`Bag{'Sara', ..., 'Salma'}`
`e1.employes->forAll(p | p.age >= 18)`	tous vérifient	`true`
`e1.employes->exists(p | p.salaire > 14000)`	au moins un	`true`
`e1.employes->one(p | p.age > 40)`	exactement un	`true` (p3)
`e1.employes->any(p | p.sexe = Sexe::HOMME)`	un élément quelconque	p2 ou p4
`e1.employes->isUnique(p | p.nom)`	unicité	`false` (deux Tazi)
`e1.employes->sortedBy(p | p.age)`	tri	`Sequence{p4, p5, p2, p1, p3}`
`e1.employes->select(p | p.salaire < 8000)->size()`	composition	`2`

Exemple 3.24 – Itérateurs dans la boutique et la bibliothèque.

```
-- Les commandes payées d'un client
```

```
c.commandes->select(o | o.statut = StatutCommande::PAYEE)
-- Le chiffre d'affaires d'un client (commandes livrées)
c.commandes->select(o | o.statut = StatutCommande::LIVREE).total->sum()
-- Tous les livres empruntés (en cours) sont marqués indisponibles
context Bibliotheque inv dispoCoherente:
  self.adherents.emprunts->select(e | e.dateRetour.oclIsUndefined())
      ->forAll(e | not e.livre.disponible)
-- Existe-t-il un produit en rupture ?
b.produits->exists(x | x.stock = 0)          -- b : la boutique
```

□

Exemple 3.25 – `forAll` à deux variables : unicité.

```
-- Deux employés différents n'ont pas le même couple (prénom, nom)
context Entreprise inv identiteUnique:
  self.employes->forAll(x, y | x <> y implies
        (x.prenom <> y.prenom or x.nom <> y.nom))
```

Sur `e1`, l'invariant est vrai : Omar Tazi et Salma Tazi ont le même nom mais des prénoms différents. (On évite d'appeler les variables `p1`, `p2` pour ne pas les confondre avec les objets du diagramme.) □

3.6.4 L'itérateur général `iterate`

Tous les itérateurs peuvent s'écrire avec `iterate`, qui accumule un résultat : `coll->iterate(x ; acc : T = init | expr)`.

Exemple 3.26 – Calculer avec `iterate`.

```
-- Somme des salaires (équivaut à e1.employes.salaire->sum())
e1.employes->iterate(p ; s : Real = 0 | s + p.salaire)              -- 49000

-- Nombre de femmes (équivaut à select(...)->size())
e1.employes->iterate(p ; n : Integer = 0 |
      if p.sexe = Sexe::FEMME then n + 1 else n endif)               -- 3

-- Total d'une commande (équivaut à collect(...)->sum())
o.lignes->iterate(l ; t : Real = 0 | t + l.quantite * l.produit.prix)
```

□

3.6.5 Opérations ensemblistes

Exemple 3.27 – Union, intersection, ajout, retrait.

```
d1.membres->union(d2.membres)              -- Set{p1, p2, p5, p3, p4}
d1.membres->intersection(p1.subordonnes)   -- Set{p2, p5}
d1.membres->including(p4)                  -- Set{p1, p2, p5, p4}
d1.membres->excluding(p1)                  -- Set{p2, p5}
d1.membres - p1.subordonnes                -- Set{p1}  (différence de Set)
Set{1, 2} = Set{2, 1}                      -- true
```

□

3.6.6 `allInstances`

`T.allInstances()` renvoie l'ensemble de toutes les instances **existantes** du type `T`. Il est utile pour exprimer des règles globales, mais coûteux : on lui préfère une navigation depuis un objet racine quand c'est possible.

```
Personne.allInstances()->size()           -- 5
Departement.allInstances()->select(d | d.budget > 300000)    -- Set{d1}
context Personne inv nomCompletUnique:
  Personne.allInstances()->isUnique(p | p.prenom + ' ' + p.nom)
```

3.7 Spécifier des opérations : pré- et post-conditions

Une opération se spécifie par contrat :

- la **pré-condition** (`pre`) doit être vraie **avant** l'appel : c'est une obligation de l'appelant ;
- la **post-condition** (`post`) doit être vraie **après** l'appel : c'est une garantie de l'opération ;
- dans une post-condition, `x@pre` désigne la valeur de `x` **avant** l'appel et `result` la valeur renvoyée.

Exemple 3.28 – Contrat de `augmenterSalaire`.

```
context Personne::augmenterSalaire(t : Real)
  pre  tauxValide:   t > 0 and t <= 20
  post salaireAugm:  self.salaire = self.salaire@pre * (1 + t / 100)
  post ageInchange:  self.age = self.age@pre
```

Si l'on appelle `p2.augmenterSalaire(10)`, la pré-condition est vraie ($0 < 10 \leq 20$) et la post-condition impose `p2.salaire = 9900`. □

Exemple 3.29 – Opération qui renvoie une valeur.

```
context Departement::effectif() : Integer
  post: result = self.membres->size()          -- d1 : result = 3

context Entreprise::embaucher(p : Personne)
  pre  pasDejaEmploye: self.employes->excludes(p)
  post estEmploye:     self.employes = self.employes@pre->including(p)
  post unDePlus:       self.employes->size() = self.employes@pre->size() + 1
```

□

Exemple 3.30 – Contrats dans la bibliothèque, la boutique et le feu.

```
context Adherent::emprunter(l : Livre)
  pre  dispo:   l.disponible
  pre  quota:   self.emprunts->select(e | e.dateRetour.oclIsUndefined())->size() < 3
  post indispo: not l.disponible
  post unDePlus: self.emprunts->size() = self.emprunts@pre->size() + 1

context Commande::payer()
  pre  enCours: self.statut = StatutCommande::EN_COURS
  post payee:   self.statut = StatutCommande::PAYEE
```

```
context Feu::changerEtat()
  post cycle: (self.etat@pre = Couleur::ROUGE  implies self.etat = Couleur::VERT) and
              (self.etat@pre = Couleur::VERT   implies self.etat = Couleur::ORANGE) and
              (self.etat@pre = Couleur::ORANGE implies self.etat = Couleur::ROUGE)
```

Le contrat de `changerEtat` décrit exactement la machine à états du feu (figure 1.5). □

3.8 Définitions : `def`, `body`, `derive`, `init`

Exemple 3.31 – Compléter le modèle par des définitions.

```
-- body : corps d'une opération de requête
context Personne::revenu() : Real
  body: if self.oclIsKindOf(Cadre) then self.salaire + self.oclAsType(Cadre).prime
        else self.salaire endif                          -- p1 : 18000 ; p2 : 9000

-- derive : attribut dérivé /masse (calculé, non stocké), ajouté à Departement
context Departement::masse : Real
  derive: self.membres.salaire->sum()                    -- d1 : 31000 ; d2 : 18000

-- init : valeur initiale d'un attribut à la création
context Personne::salaire : Real
  init: 3000

-- def : attribut ou opération auxiliaire, réutilisable dans d'autres contraintes
context Personne def: nomComplet : String = self.prenom + ' ' + self.nom
context Personne def: collegues() : Set(Personne) =
  if self.departement.oclIsUndefined() then Set{}
  else self.departement.membres->excluding(self) endif

p4.nomComplet                    -- 'Omar Tazi'
p2.collegues()                   -- Set{p1, p5}
```

□

Exemple 3.32 – Définitions dans les autres modèles.

```
context Commande::total : Real
  derive: self.lignes->collect(l | l.quantite * l.produit.prix)->sum()
context Commande::statut : StatutCommande
  init: StatutCommande::EN_COURS
context Feu def: dureeCycle : Integer = self.dureeVert + self.dureeOrange + self.dureeRouge
context Adherent def: empruntsEnCours : Set(Emprunt) =
  self.emprunts->select(e | e.dateRetour.oclIsUndefined())
```

Avec `total` dérivé, l'invariant `totalCoherent` devient inutile : la valeur ne peut plus être incohérente. Avec `empruntsEnCours`, l'invariant `quota` s'écrit `self.empruntsEnCours->size() <= 3`. □

3.9 OCL au niveau méta et dans les outils

Les mêmes constructions s'appliquent à un **méta-modèle** (M2) : elles définissent alors des règles de bonne formation que tous les modèles conformes doivent respecter.

Exemple 3.33 – Contraintes de bonne formation.

```
-- Machine à états (méta-modèle du chapitre 2) : exactement un état initial ;
-- pas de transition sortant d'un état final ; pas de transition entrant dans l'initial
context StateMachine inv unSeulInitial:
  self.states->select(s | s.kind = StateKind::INITIAL)->size() = 1
context State inv finalSansSortie:
  self.kind = StateKind::FINAL implies self.outgoing->isEmpty()
context Transition inv pasVersInitial:
  self.target.kind <> StateKind::INITIAL

-- Méta-modèle library.ecore : ISBN uniques ; un livre emprunté (non rendu) n'est pas
    disponible
context Library inv isbnUnique:      self.book->isUnique(b | b.isbn)
context Loan    inv livreIndispo:
  (self.returnDate.oclIsUndefined() and not self.book.oclIsUndefined())
      implies not self.book.available

-- Méta-modèle Ecore lui-même : deux attributs d'une classe ont des noms différents
context EClass inv nomsAttributsUniques: self.eAttributes->isUnique(a | a.name)
```

Attention, sur la machine à états du feu, `pasVersInitial` est **violée** : la transition Orange → Rouge entre dans l'état initial. C'est une règle de certains langages (UML impose un pseudo-état initial distinct de Rouge) ; dans notre petit langage, on ne l'ajoute que si l'on veut interdire ce cas. □

Dans Eclipse. **OCLinEcore** permet d'écrire les invariants directement dans le fichier `.ecore` (éditeur texte *OCLinEcore*) ; la validation EMF (*Validate*) les vérifie sur chaque modèle `.xmi`. La **console OCL** (*Show Xtext OCL Console*) évalue une expression sur l'objet sélectionné dans un éditeur : c'est l'outil idéal pour tester les exemples de ce chapitre.

```
class Personne {
    attribute nom : String[1];
    attribute age : ecore::EInt[1];
    property manager : Personne[?];
    invariant ageLegal: age >= 18 and age <= 65;
    invariant pasSonManager: manager <> self;
}
```

Listing 3.1 – Invariant dans un fichier .ecore (syntaxe OCLinEcore)

3.10 Méthode et pièges fréquents

Pour écrire une contrainte :

1. choisir le **contexte** le plus naturel (la classe dont la règle parle) ;
2. traduire la règle en navigation : quels objets, quels attributs ?
3. repérer les quantificateurs : « tous » → `forAll`, « au moins un » → `exists`, « ceux qui » → `select`, « le nombre de » → `size()` ;
4. protéger les navigations 0..1 (`implies`, `oclIsUndefined()`) ;
5. **tester** la contrainte sur un diagramme d'objets valide et sur un contre-exemple.

Tableau 3.7 – Pièges fréquents en OCL

Erreur	Correction
`self.employes.size()`	`self.employes->size()` (flèche sur une collection)
`if c then a endif`	`if c then a else b endif` (le `else` est obligatoire)
`p.nom = "Tazi"`	`p.nom = 'Tazi'` (apostrophes)
`self.manager.salaire > 0` sans garde	`not self.manager.oclIsUndefined() implies ...`
`self.prime > 0` dans le contexte `Personne`	contexte `Cadre`, ou `oclAsType(Cadre).prime`
`post: self.salaire = self.salaire * 1.1`	`self.salaire = self.salaire@pre * 1.1`
`sexe = FEMME`	`sexe = Sexe::FEMME`
`self.emprunts->size() <= 3` pour « 3 en cours »	filtrer d'abord les emprunts en cours (`select`)

3.11 Exercices

Les exercices 3.1 à 3.5 portent sur le modèle de la figure 3.1 et le diagramme d'objets du tableau 3.1. Les corrigés figurent en annexe A.

Exercice 3.1 – Évaluer des expressions. Donner la valeur de : (a) `p5.manager.nom` ; (b) `d2.membres.age->sum()` ; (c) `p3.subordonnes->size()` ; (d) `e1.employes->select(p | p.sexe = Sexe::HOMME).prenom` ; (e) `e1.departements->select(d | d.membres->size() > 2).nom` ; (f) `p4.conjoint.manager.prenom` ; (g) `e1.employes->exists(p | p.age < 20)`.

Exercice 3.2 – Invariants sur les attributs et les associations. Écrire en OCL : (a) le budget d'un département est strictement positif ; (b) le nom d'une entreprise comporte au moins 2 caractères ; (c) une personne n'est membre que d'un département de son employeur ; (d) le manager d'une personne appartient au même département qu'elle. Lesquels sont violés par le diagramme d'étude ?

Exercice 3.3 – Collections. Écrire les expressions donnant : (a) le salaire moyen des employés de `e1` ; (b) les noms des départements sans chef ; (c) les employés qui n'ont ni manager ni subordonnés ; (d) le plus jeune employé ; (e) le nombre de couples mariés dans l'entreprise.

Exercice 3.4 – Héritage. (a) Écrire un invariant imposant que la prime d'un cadre ne dépasse pas 30 % de son salaire. (b) Écrire une expression qui donne la somme des primes des cadres de `e1`. (c) Que vaut `p2.oclIsTypeOf(Cadre)` ?

Exercice 3.5 – Contrats d'opérations. Spécifier par pré/post-conditions : (a) `Departement::ajouterMembre(p : Personne)` ; (b) `Personne::changerManager(m : Personne)` (on ne peut pas devenir son propre manager) ; (c) `Entreprise::licencier(p : Personne)`.

Exercice 3.6 – Traduire des règles métier. Une bibliothèque a des `Livre` (`isbn`, `titre`, `disponible : Boolean`) et des `Adherent` (`nom`, `age`) qui ont des `emprunts : Livre [0..*]` (les livres actuellement empruntés). Écrire : (a) un adhérent emprunte au plus 3 livres ; (b) un

livre emprunté n'est pas disponible ; (c) un mineur emprunte au plus 1 livre ; (d) les ISBN sont uniques.

3.12 Synthèse

Tableau 3.8 – Synthèse du chapitre OCL

Contexte	`context C inv`, `context C::op() pre/post/body`, `context C::a derive/init`, `def`.
Navigation	attribut `self.a` ; rôle 1 → objet ; 0..1 → objet ou indéfini ; * → collection.
Types	`oclIsKindOf`, `oclIsTypeOf`, `oclAsType`, `oclIsUndefined`, `Enum::LIT`.
Collections	`Set`, `OrderedSet`, `Bag`, `Sequence` ; flèche `->`.
Itérateurs	`select`, `reject`, `collect`, `forAll`, `exists`, `one`, `any`, `isUnique`, `sortedBy`, `iterate`.
Contrats	`pre`, `post`, `@pre`, `result`.

3.13 Quiz

Répondez aux questions suivantes. Les corrigés commentés figurent en annexe A.

Q1. Une expression OCL peut-elle modifier le modèle ?

a) Oui, dans une post-condition

b) Oui, avec `def:`

c) Non, OCL est sans effet de bord

d) Oui, avec `@pre`

Q2. Que vaut `p2.manager.prenom` sur le diagramme d'étude ?

a) `'Youssef'`

b) `'Sara'`

c) `'Nadia'`

d) indéfini

Q3. Quel est le type de `e1.employes` ?

a) `Personne`

b) `Set(Personne)`

c) `Bag(Personne)`

d) `Sequence(Personne)`

Q4. Quel est le type de `d1.membres.salaire` ?

a) `Real`

b) `Set(Real)`

c) `Bag(Real)`

d) `Sequence(Real)`

Q5. Comment écrit-on la valeur FEMME de l'énumération `Sexe` ?

a) `FEMME`
b) `'FEMME'`
c) `Sexe::FEMME`
d) `#FEMME`

Q6. Que vaut `p1.oclIsTypeOf(Personne)` (p1 est un `Cadre`) ?

a) true
b) false
c) indéfini
d) une erreur

Q7. Pour exprimer « tous les employés sont majeurs », on utilise :

a) `exists`
b) `forAll`
c) `collect`
d) `select`

Q8. Que vaut `e1.employes->select(p | p.age > 30)->size()` ?

a) 1
b) 2
c) 3
d) 5

Q9. Dans une post-condition, `self.salaire@pre` désigne :

a) le salaire initial à la création
b) le salaire avant l'exécution de l'opération
c) la pré-condition
d) le résultat de l'opération

Q10. Quelle opération teste qu'une association 0..1 n'est pas renseignée ?

a) `isEmpty()`
b) `oclIsUndefined()`
c) `oclIsKindOf()`
d) `excludes()`

Q11. Que vaut `e1.employes->isUnique(p | p.nom)` ? Pourquoi ?

Q12. Écrire un invariant imposant qu'un département ait au plus 50 membres.

Chapitre 4

L'approche MDA (Model Driven Architecture)

Résumé

*Le **métier** d'une application (prêter des livres à des adhérents) change peu, alors que les **plateformes** techniques (Java EE, .NET, PHP/SQL, mobile, cloud…) se succèdent sans fin. **MDA**, démarche IDM proposée par l'OMG en 2001, sépare les spécifications **fonctionnelles** des spécifications d'**implantation**. Le système est décrit par une suite de modèles, du plus abstrait au plus technique : **CIM** (besoins), **PIM** (solution indépendante de la plateforme), **PDM** (description de la plateforme) et **PSM** (solution projetée sur une plateforme), jusqu'au **code**. On passe d'un modèle à l'autre par des **transformations** (raffinement, projection, génération, rétro-ingénierie), que l'on classe selon plusieurs critères (endogène/exogène, horizontale/verticale, M2M/M2T/T2M). Trois grandes approches permettent de les écrire : par **template** (XSLT, Acceleo), par **programmation** (Java + EMF) et par **modélisation** (ATL, QVT). La bibliothèque est suivie du besoin jusqu'au code Java et SQL ; la boutique en ligne, le feu de signalisation et la gestion des notes illustrent les mêmes notions.*

Objectifs d'apprentissage

À l'issue de ce chapitre, vous serez capable de :

- expliquer l'idée de MDA : séparer le métier de la plateforme ;
- distinguer CIM, PIM, PDM et PSM et dire ce que chacun contient ;
- retenir la formule **PIM + PDM ⇒ PSM ⇒ Code** ;
- classer une transformation (sens, endogène/exogène, horizontale/verticale, M2M/M2T) ;
- écrire une transformation simple par template (XSLT), par programmation (Java) ou par modélisation (ATL) ;
- choisir une approche de transformation selon le besoin.

Contenu du chapitre

4.1 Pourquoi MDA ? 52
4.2 Les modèles de MDA 54
4.2.1 CIM – Computation Independent Model 54
4.2.2 PIM – Platform Independent Model 54
4.2.3 PDM – Platform Description Model 55

4.2.4 PSM – Platform Specific Model 55
4.2.5 Le code 56
4.3 Les transformations de modèles 57
4.3.1 Les transformations de MDA 58
4.3.2 Taxonomie des transformations 59
4.4 Les approches de transformation 60
4.4.1 Transformation par template 61
4.4.2 Transformation par programmation 62
4.4.3 Transformation par modélisation 64
4.5 Études de cas corrigées 65
4.6 Exemples complémentaires 66
4.7 Exercices 67
4.8 Quiz 68

4.1 Pourquoi MDA ?

Le génie logiciel fait face à plusieurs tendances : complexité croissante des logiciels, séparation des préoccupations (métier, sécurité, persistance, IHM), multiplicité des besoins (front et back office) et surtout **multiplicité et évolution permanente des plateformes**. Faut-il continuer à baser les développements sur le code ?

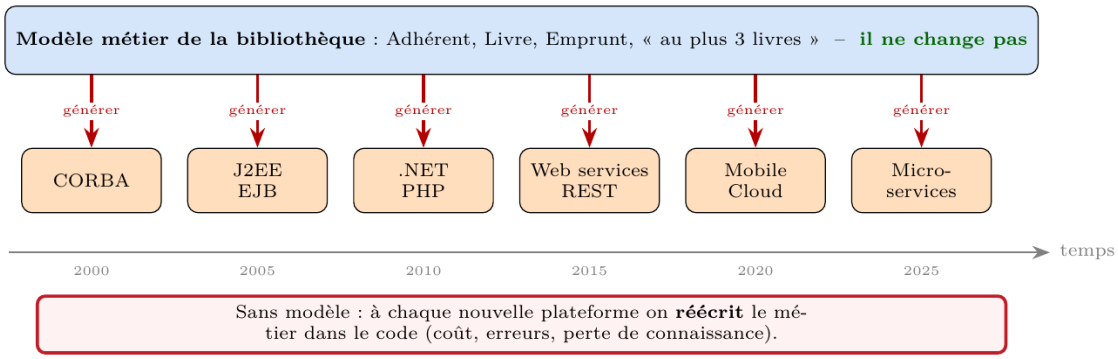


Figure 4.1 – Le métier est stable, la plateforme change : sans MDA, on réécrit le métier à chaque technologie

À retenir. Idée de MDA : écrire le métier **une seule fois** dans un modèle, puis **générer** la version de chaque plateforme. Formule : **PIM + PDM ⇒ PSM ⇒ Code**.

MDA s'appuie sur les standards de l'OMG : **UML** (modèles), **OCL** (contraintes), **MOF** (méta-modèles), **XMI** (échange) et **QVT** (transformations). Le développement procède par **transformations** et **enrichissements successifs** (raffinement) des modèles d'analyse, puis de conception, jusqu'au code.

Exemple 4.1 – Analogie : la recette de cuisine.

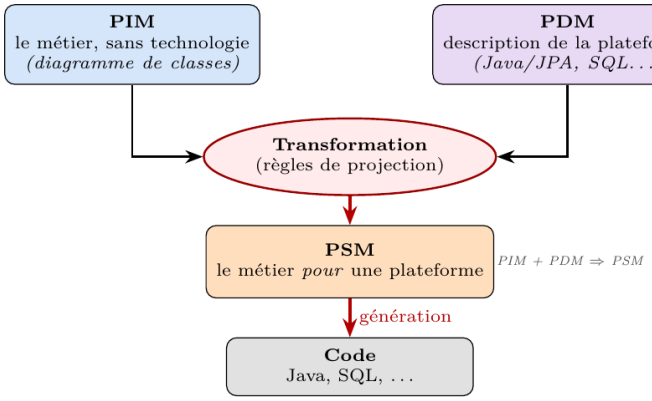


Figure 4.2 – Le principe de MDA (cycle « en Y ») : le PIM et le PDM se rejoignent dans le PSM

MDA	Cuisine	Bibliothèque
CIM (besoin)	« Un gâteau pour 8 personnes, sans gluten. »	« Les adhérents empruntent au plus 3 livres. »
PIM (solution)	La recette : ingrédients, étapes, cuisson.	Diagramme de classes Adherent, Livre + OCL.
PDM (plateforme)	Le four : électrique, à gaz, micro-ondes.	Java/JPA, SQL, .NET.
PSM (solution adaptée)	La recette adaptée au four à gaz (th. 6).	Classes Java `@Entity`, tables SQL.
Code	Le gâteau !	`Adherent.java`, `bibliotheque.sql`

La recette (PIM) ne change pas quand on change de four : seule l'adaptation (PSM) change. □

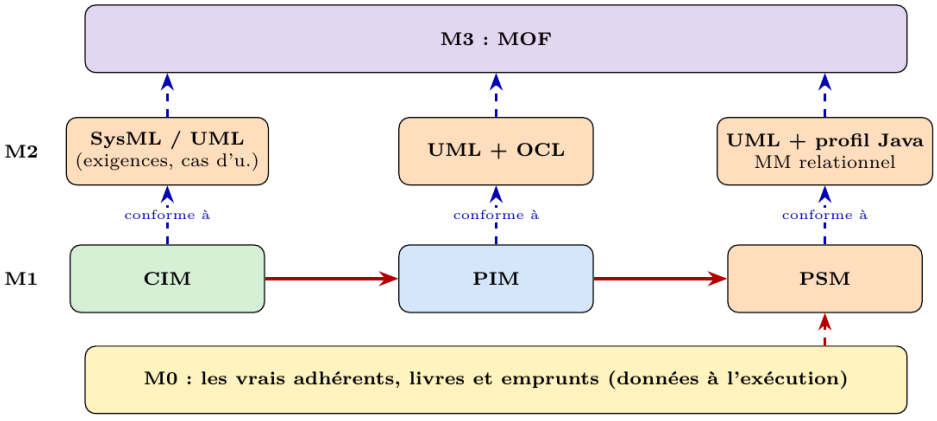


Figure 4.3 – MDA et les niveaux M0 à M3 : CIM, PIM et PSM sont tous des modèles M1

Attention. CIM, PIM et PSM sont **tous des modèles M1** du même système. Ils diffèrent par leur **point de vue** (plus ou moins proche de la technique), pas par leur niveau méta. Le PDM, lui, décrit une plateforme ; il prend souvent la forme d'un méta-modèle ou d'un profil (niveau M2) accompagné de règles.

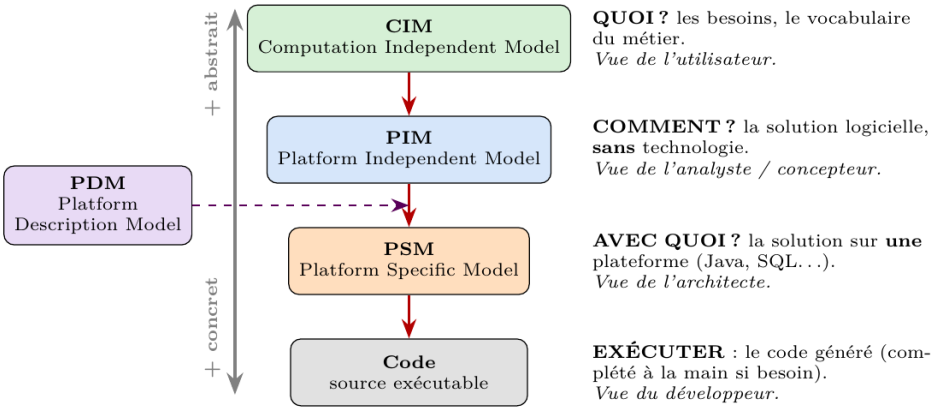


Figure 4.4 – Les modèles de MDA, du plus abstrait au plus concret

4.2 Les modèles de MDA

Le besoin de la bibliothèque : « Notre bibliothèque prête des livres à ses **adhérents**. Un adhérent peut **rechercher** un livre, l'**emprunter** et le **rendre**. Il emprunte **au plus 3 livres** en même temps, pour **15 jours** au maximum. Un livre emprunté n'est plus disponible. Le **bibliothécaire** gère le catalogue. » Les **noms** donnent les futurs concepts (Adhérent, Livre, Emprunt), les **verbes** les futures fonctions (emprunter, rendre...).

4.2.1 CIM – Computation Independent Model

Modèle des **exigences**, du **métier** ou du **domaine**. Il décrit *ce que* le système doit faire (acteurs, flux, actions), **sans aucune notion informatique** : on peut le montrer au client. Notations : cas d'utilisation, diagramme d'exigences SysML, activités métier. Public : client, expert métier, analyste.

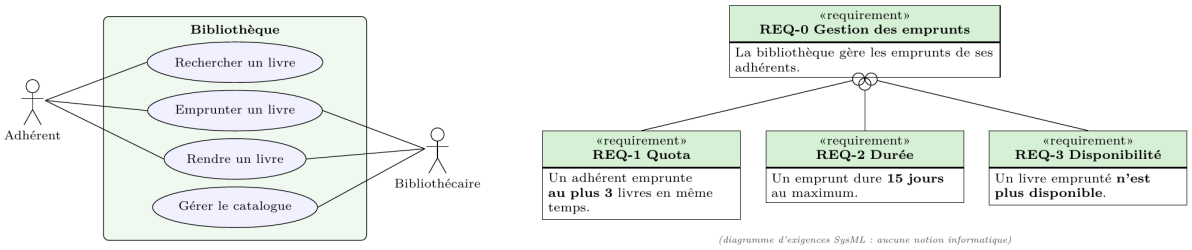


Figure 4.5 – CIM de la bibliothèque : cas d'utilisation et exigences SysML numérotées

Chaque exigence est **numérotée** (REQ-1 quota, REQ-2 durée, REQ-3 disponibilité) : on pourra vérifier qu'elle est prise en compte par le PIM, le PSM et le code (**traçabilité**).

4.2.2 PIM – Platform Independent Model

Modèle d'**analyse et de conception** : il décrit la solution logicielle (structure et comportement) **indépendamment de toute plateforme**. Types abstraits (`String`, `Integer`, `Boolean`, `Date`) ; les règles métier deviennent des **contraintes OCL** (REQ-1 ⇒ « au plus 3 »).

À retenir. Test simple : si le mot « Java », « SQL » ou « table » apparaît, ce n'est plus un PIM.

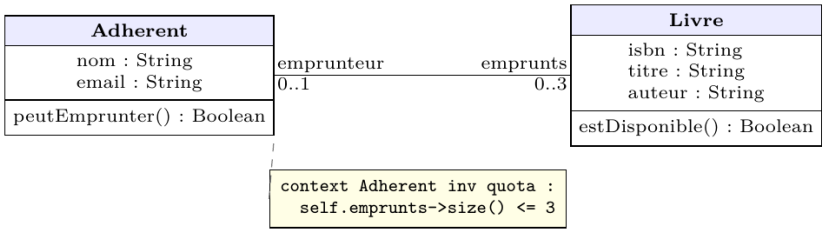


Figure 4.6 – PIM de la bibliothèque : diagramme de classes et contrainte OCL

Tableau 4.1 – Ce que le PIM contient et ne contient pas

Le PIM contient	Le PIM ne contient pas
les concepts métier (Adherent, Livre)	d'identifiant technique (id auto-incrémenté)
leurs attributs avec des types abstraits	de type de plateforme : `VARCHAR`, `List<...>`, `LocalDate`
associations et multiplicités (0..1, 0..3)	d'annotation : `@Entity`, `@Id`
opérations métier : `peutEmprunter()`	de détail d'IHM, de framework, de base de données
contraintes OCL	

4.2.3 PDM – Platform Description Model

Il décrit la **structure** et les **services techniques** d'une plateforme d'exécution (Java EE, .NET, SQL, CORBA…) et sert de référence aux **règles de passage** du PIM vers cette plateforme. En pratique : un **profil UML**, un **méta-modèle** de la plateforme et des **règles** écrites en QVT, ATL, Acceleo ou Java.

Concept du PIM (UML)	Plateforme Java / JPA	Plateforme SQL
Classe `C`	classe `C` annotée `@Entity`	table `C`
Identité d'un objet (implicite)	attribut `@Id Long id`	colonne `id PRIMARY KEY`
Attribut `String`	`private String` + get/set	colonne `VARCHAR(255)`
Attribut `Integer` / `Boolean`	`int` / `boolean`	`INTEGER` / `BOOLEAN`
Attribut `Date`	`java.time.LocalDate`	`DATE`
Association `0..1` – `0..*`	`List<B>` `@OneToMany` / `A` `@ManyToOne`	clé étrangère côté « plusieurs »
Association `*` – `*`	`List` des deux côtés `@ManyToMany`	table de jointure
Opération	méthode `public`	(pas de traduction)

Figure 4.7 – PDM : correspondances entre concepts du PIM et des plateformes Java/JPA et SQL

4.2.4 PSM – Platform Specific Model

Modèle **technique** : le PIM **projeté** sur une plateforme particulière, souvent avec un **profil UML** (UML Profile for Java/EJB, CORBA, .NET, SQL…). Par rapport au PIM, le PSM Java ajoute un `id` technique, des types Java (`List<Livre>`) et des stéréotypes (`«Entity»`, `«Id»`, `«OneToMany»`).

Dans le PSM relationnel, chaque classe est devenue une **table**, chaque attribut une **colonne** ; l'association 0..1 – 0..3 est devenue une **clé étrangère** `adherent_id` dans `LIVRE` (côté « plusieurs »). La contrainte « au plus 3 » n'a pas d'équivalent direct en SQL : elle sera vérifiée par le code (ou un *trigger*).

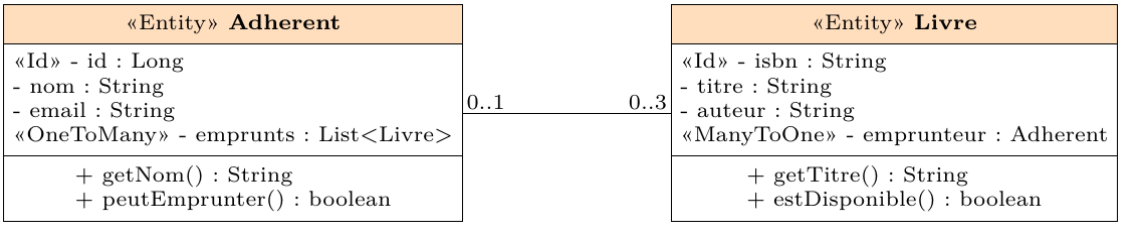


Figure 4.8 – PSM Java/JPA de la bibliothèque

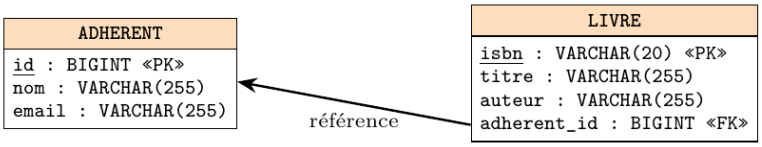


Figure 4.9 – PSM relationnel (SQL) de la bibliothèque

4.2.5 Le code

Exemple 4.2 – Code généré à partir des PSM.

```
@Entity
public class Adherent {
    @Id @GeneratedValue
    private Long id;
    private String nom;
    private String email;
    @OneToMany(mappedBy = "emprunteur")
    private List<Livre> emprunts = new ArrayList<>();

    public boolean peutEmprunter() {     // REQ-1 / contrainte OCL
        return emprunts.size() < 3;
    }
}
```

```
CREATE TABLE ADHERENT (id BIGINT PRIMARY KEY, nom VARCHAR(255), email VARCHAR(255));
CREATE TABLE LIVRE (
  isbn VARCHAR(255) PRIMARY KEY, titre VARCHAR(255),
  adherent_id BIGINT REFERENCES ADHERENT(id));
```

Le code est obtenu par génération automatique (totale ou partielle), puis complété à la main (corps des méthodes métier). Le côté `Livre` porte le champ `@ManyToOne Adherent emprunteur` correspondant à `mappedBy`. □

Exemple 4.3 – CIM, PIM, PSM et code dans trois autres domaines.

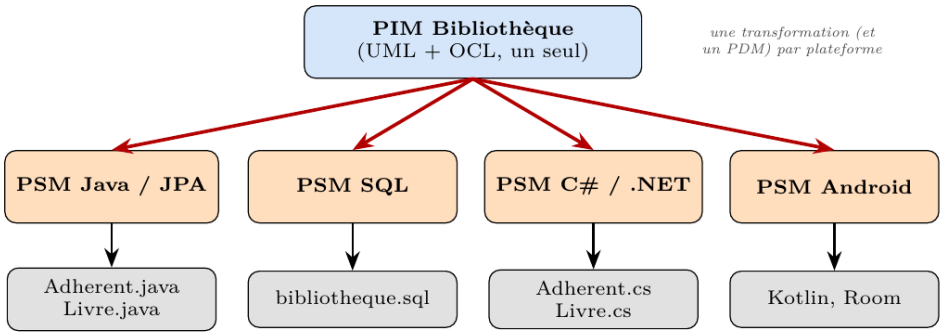


Figure 4.10 – Un seul PIM, plusieurs PSM : le métier n'est pas réécrit

	Boutique en ligne	**Feu de signalisation**	**Hôpital**
CIM	« Le client passe une commande d'au moins un produit et la paie en ligne. »	« Le feu alterne rouge, vert, orange ; le vert dure 20 s. »	« Un patient prend rendez-vous avec son médecin référent. »
PIM	`Client`, `Commande`, `LigneCommande`, `Produit` ; OCL `lignes->notEmpty()`	machine à états Rouge/Vert/Orange, `after(20 s)`	`Patient`, `Medecin`, `RendezVous(date)`
PDM	Java/Spring + PostgreSQL	microcontrôleur C (timer matériel)	.NET + SQL Server
PSM	`«Entity» Commande`, `«OneToMany» lignes`	`enum Etat`, `switch`, tâche périodique 1 s	`[Table] Patient`, `DbSet<Patient>`
Code	`Commande.java`, `schema.sql`	`feu.c`	`Patient.cs`

Le même PIM du feu pourrait aussi être projeté vers un automate programmable ou un simulateur Java : c'est le PDM qui change, pas le PIM. □

Tableau 4.2 – Synthèse des modèles de MDA

Modèle	Question	Contenu	Bibliothèque	Notations
CIM	Quoi ?	besoins, acteurs, règles métier	« au plus 3 livres »	cas d'util., SysML
PIM	Comment, sans techno ?	classes, comportement, contraintes	Adherent, Livre, OCL	UML + OCL
PDM	Sur quelle plateforme ?	concepts et règles de la plateforme	Java/JPA, SQL	profil, méta-modèle
PSM	Comment, avec la techno ?	PIM projeté	`@Entity`, tables	UML + profil
Code	Exécuter	source généré + complété	`.java`, `.sql`	langage cible

4.3 Les transformations de modèles

Définition 4.1 (Transformation de modèles). *Une **transformation** produit un modèle **cible** M_b à partir d'un modèle **source** M_a. Elle est définie par des **règles** écrites sur les **méta-modèles** (MM_a, MM_b), et non sur un modèle particulier : la même transformation sert donc pour **tous** les modèles conformes à MM_a. Un **moteur** exécute les règles.*

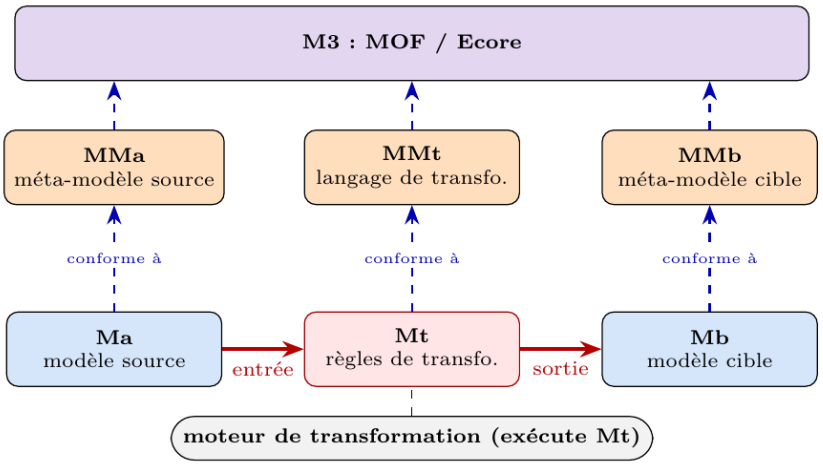


Figure 4.11 – Schéma général d'une transformation : la transformation est elle-même un modèle

4.3.1 Les transformations de MDA

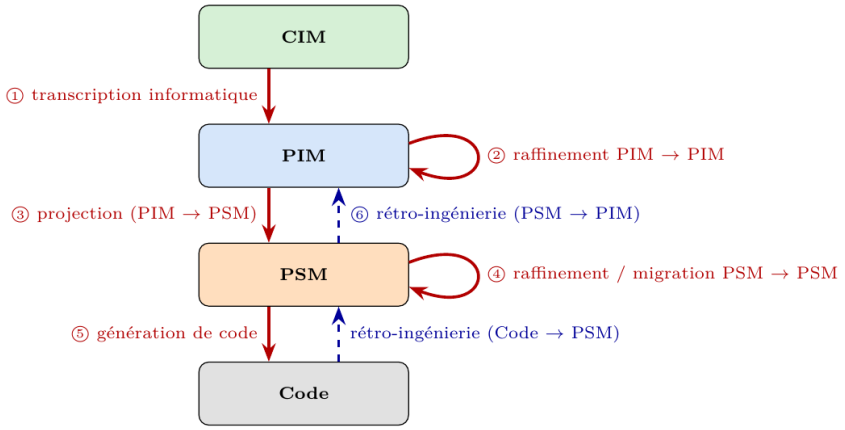


Figure 4.12 – Les six transformations de MDA

① **CIM → PIM** : transcription informatique, souvent **à la main** ;

② **PIM → PIM** : **raffinement** – on ajoute de l'information non liée à une plateforme (ex. réifier une association en classe `Emprunt`) ;

③ **PIM → PSM** : **projection** vers une technologie décrite par un PDM ;

④ **PSM → PSM** : raffinement (Java → Java + Spring) ou **migration** (.NET → Java) ;

⑤ **PSM → Code** : **génération de code** ;

⑥ **PSM → PIM** : **rétro-ingénierie** (*reverse engineering*), pour intégrer ou moderniser des applications existantes.

Exemple 4.4 – Les six transformations sur trois domaines.

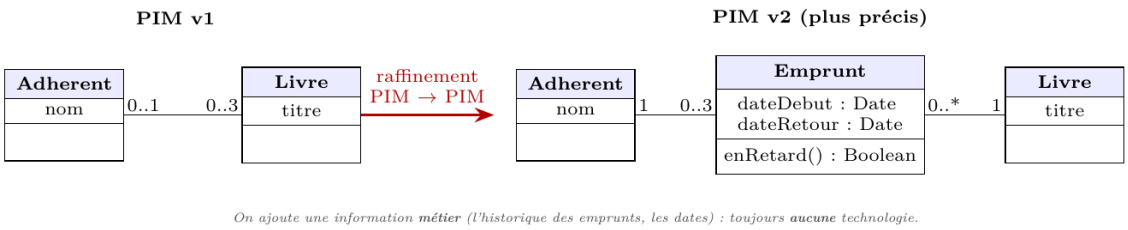


Figure 4.13 – Raffinement PIM → PIM : réification de l'association en classe Emprunt

	Bibliothèque	**Boutique**	**Feu**
CIM→PIM	cas « Emprunter » → classes Adherent, Livre	cas « Commander » → Commande, Ligne	« cycle 25/20/5 s » → machine à états
PIM→PIM	association → classe `Emprunt`	ajout de `Paiement` abstrait	ajout d'un état `Clignotant` (mode nuit)
PIM→PSM	classes → `@Entity`	classes → tables PostgreSQL	états → `enum Etat` C
PSM→PSM	JPA → JPA + Spring Data	MySQL → PostgreSQL	C → C + RTOS
PSM→Code	`Adherent.java`	`schema.sql`	`feu.c`
PSM→PIM	ancienne base `LIVRE` → classe Livre	schéma PHP existant → PIM	code C existant → machine à états

□

Exemple 4.5 – Règles de projection PIM → Java/JPA.

- pour chaque **classe** C → une classe Java C annotée `@Entity` ;
- si C n'a pas d'identifiant → ajouter `@Id @GeneratedValue Long id` ;
- pour chaque **attribut** → un champ `private` + `get/set` ;
- pour chaque association 1 – * → côté « un » : `List<...>` + `@OneToMany(mappedBy)` ; côté « plusieurs » : un champ + `@ManyToOne`.

Ces règles sont **généralisées** et **capitalisées** dans un langage de transformation : on les réutilise pour d'autres projets. □

Exemple 4.6 – Modernisation d'une application. Ancienne base SQL $\xrightarrow{(6)}$ PIM $\xrightarrow{(3)}$ PSM Java/JPA $\xrightarrow{(5)}$ code neuf : le métier est **récupéré** puis **régénéré** sur une nouvelle plateforme. (Le schéma SQL est d'abord lu pour obtenir un PSM relationnel : c'est une étape T2M.) □

4.3.2 Taxonomie des transformations

- **Endogène** : source et cible ont le **même** méta-modèle (refactoring, raffinement) ; **exogène** : méta-modèles **différents** (UML → relationnel).
- **Verticale** : le niveau d'abstraction change (raffinement, projection, rétro-ingénierie) ; **horizontale** : il reste le même (migration, restructuration, renommage).
- **Unidirectionnelle** (un seul sens, cas d'ATL) ou **bidirectionnelle** (QVT Relations).

Les deux premiers critères sont indépendants : on trouve les quatre combinaisons (exemple 4.7).

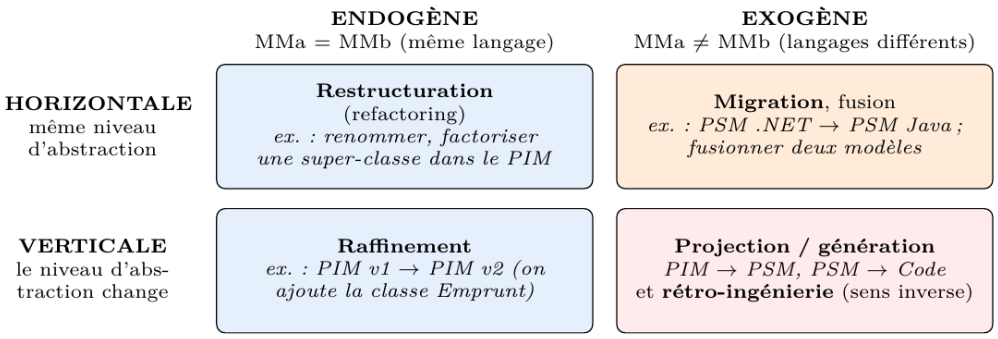


Figure 4.14 – Taxonomie : endogène/exogène, horizontale/verticale

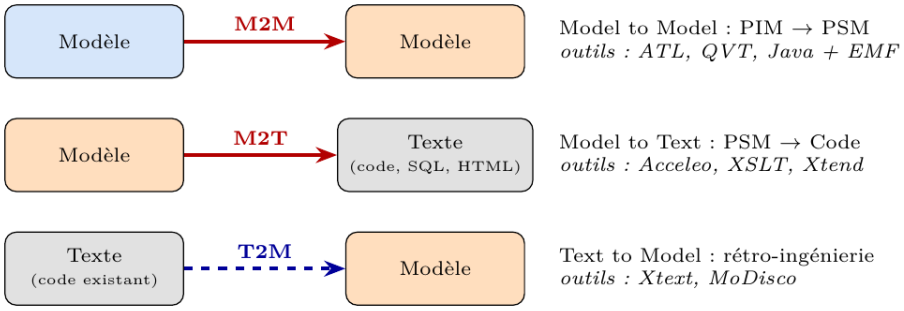


Figure 4.15 – Selon la nature de la cible : M2M, M2T, T2M

Dans la chaîne MDA, PIM → PSM est en général une transformation **M2M** (*model to model*) ; PSM → Code est une transformation **M2T** (*model to text*) ; la lecture d'un code ou d'un schéma existant pour en faire un modèle est une transformation **T2M**.

Exemple 4.7 – Classer huit transformations réelles.

Transformation	MDA	Endo/exo, sens	Nature
diagramme de classes → schéma relationnel	PIM→PSM	exogène, verticale	M2M
schéma relationnel → script `CREATE TABLE`	PSM→Code	exogène, verticale	M2T
fusion de deux diagrammes de classes	PIM→PIM	endogène, horizontale	M2M
ajout des getters/setters dans un PSM Java	PSM→PSM	endogène, verticale (raffinement)	M2M
renommer « Adherent » en « Membre »	PIM→PIM	endogène, horizontale	M2M
code Java → diagramme de classes	Code→PSM (puis PIM)	exogène, verticale	T2M
modèle UML 1.4 → UML 2.5	PIM→PIM	exogène, horizontale	M2M
Families → Persons (chapitre 7)	–	exogène, horizontale	M2M

□

4.4 Les approches de transformation

Tableau 4.3 – Les trois grandes approches de transformation

	Par template	**Par programmation**	**Par modélisation**
La transformation est…	un texte à trous	un programme	un modèle
Cible typique	texte (M2T)	modèle ou texte	modèle (M2M)
Outils	XSLT, Acceleo, Velocity, Xtend	Java + EMF, PyEcore	ATL, QVT
Points forts	simple, lisible	puissant, flexible	déclaratif, réutilisable, traçable
Points faibles	logique complexe difficile	verbeux, règles « cachées »	langage à apprendre
Bibliothèque	PSM → `Adherent.java`	PIM → schéma relationnel	PIM → schéma relationnel

4.4.1 Transformation par template

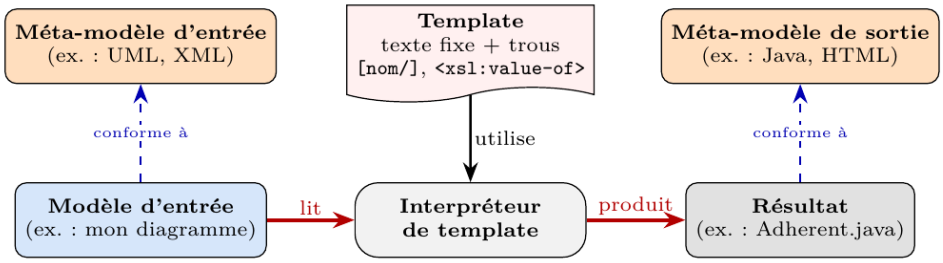


Figure 4.16 – Approche par template : du texte fixe et des trous remplis par le modèle

Le template contient du texte **fixe** et des **trous** ; un interpréteur parcourt le modèle et remplit les trous. C'est l'approche idéale pour les transformations M2T. Le chapitre 5 détaille Acceleo ; voici des exemples avec **XSLT**, qui transforme un document XML.

Exemple 4.8 – XML → texte avec XSLT. Le modèle `article.xml` :

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="article_txt.xsl"?>
<Article>
  <Title>Mon article</Title>
  <Authors> <Author>M. Foo</Author> <Author>M. Bar</Author> </Authors>
</Article>
```

Le template `article_txt.xsl` :

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:output method="text" encoding="UTF-8"/>
  <xsl:template match="/">
    <xsl:text>Article - </xsl:text>
    <xsl:value-of select="/Article/Title"/>
    <xsl:text>&#10;Auteurs :&#10;</xsl:text>
    <xsl:for-each select="/Article/Authors/Author">
      <xsl:text>  - </xsl:text><xsl:value-of select="."/><xsl:text>&#10;</xsl:text>
    </xsl:for-each>
  </xsl:template>
</xsl:stylesheet>
```

Résultat : « Article - Mon article », puis « Auteurs : » et un tiret par auteur. Avec un autre

template (`method="html"`, balises `<h1>`, `<ul>`, `<li>`), le **même modèle** donne une page HTML : c'est exactement le principe « un modèle, plusieurs cibles ». □



Figure 4.17 – Un document XML, deux feuilles XSLT, deux résultats

Tableau 4.4 – Lire un template XSLT

Instruction	Rôle	Dans l'exemple
`xsl:template match="/"`	règle appliquée à la racine du document	point de départ
`xsl:value-of select="..."`	insère une valeur du modèle (un trou)	le titre
`xsl:for-each select="..."`	boucle sur des éléments	un tiret par auteur
`xsl:if test="..."`	texte conditionnel	–
`xsl:output method="text"`	type de sortie : text, html, xml	texte brut

Exemple 4.9 – XSLT sur le carrefour : un rapport texte. À partir du document `<carrefour><feu id="RBT-012" vert="20"/><feu id="RBT-013" vert="45"/></carrefour>` (exemple 1.17) :

```
<xsl:template match="/">
  <xsl:for-each select="/carrefour/feu">
    <xsl:value-of select="@id"/><xsl:text> : vert </xsl:text>
    <xsl:value-of select="@vert"/><xsl:text> s</xsl:text>
    <xsl:if test="@vert > 30"><xsl:text> (long)</xsl:text></xsl:if>
    <xsl:text>&#10;</xsl:text>
  </xsl:for-each>
</xsl:template>
```

Résultat : « RBT-012 : vert 20 s » puis « RBT-013 : vert 45 s (long) ». □

4.4.2 Transformation par programmation

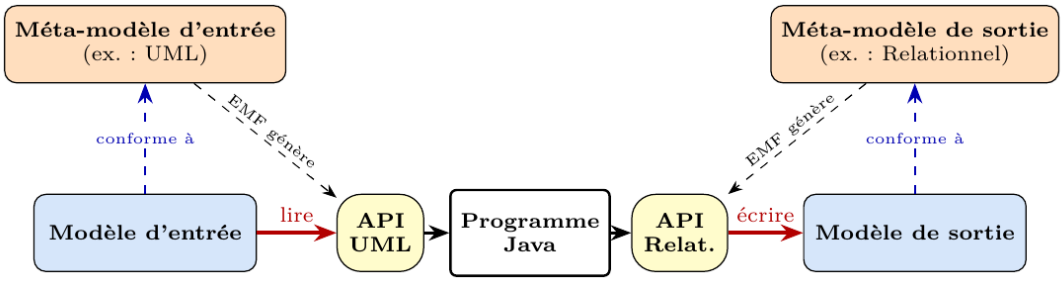


Figure 4.18 – Approche par programmation : un programme lit et écrit les modèles via des API

La transformation est un **programme** (Java) qui lit le modèle source et écrit le modèle cible au moyen d'**API** générées à partir des méta-modèles (EMF : `RelFactory`, `getClasses()`...).

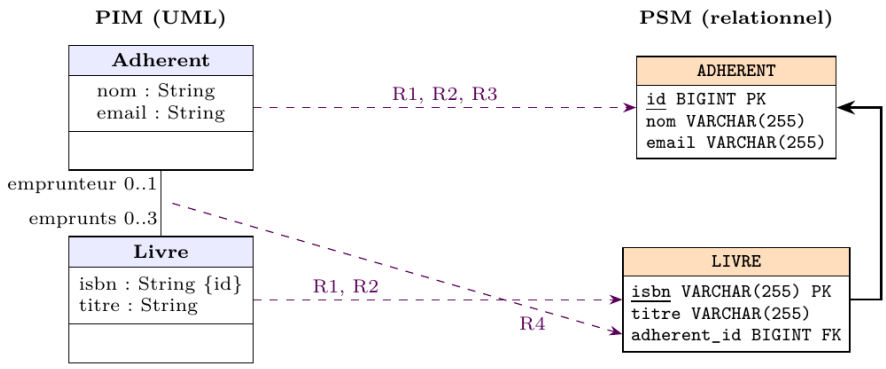


Figure 4.19 – Règles R1 à R4 de la transformation PIM UML → relationnel

Tableau 4.5 – Règles de la transformation PIM → relationnel

Règle	Source (PIM)	Cible (relationnel)
R1	Classe C	Table nommée C en majuscules
R2	Attribut a de type T	Colonne a de type SQL(T) ; clé primaire si a est {id}
R3	Classe sans attribut {id}	Colonne `id BIGINT` clé primaire
R4	Association 0..1 (A) – * (B)	Dans B : colonne `a_id` du type de la clé de A + clé étrangère vers A

La correspondance des types (le **PDM** SQL) : `String` → `VARCHAR(255)`, `Integer` → `INTEGER`, `Real` → `DECIMAL(10,2)`, `Boolean` → `BOOLEAN`, `Date` → `DATE`.

Exemple 4.10 – Le programme Java (API EMF).

```
private final Map<Classe, Table> tables = new HashMap<>();   // trace Classe -> Table
private final RelFactory f = RelFactory.eINSTANCE;           // API générée par EMF

public Schema transformer(Modele pim) {
  Schema s = f.createSchema();  s.setNom(pim.getNom());
  for (Classe c : pim.getClasses()) {                          // R1
    Table t = f.createTable();
    t.setNom(c.getNom().toUpperCase());
    for (Attribut a : c.getAttributs())                        // R2
      t.getColonnes().add(colonne(a.getNom(), sqlType(a.getType()), a.isEstIdentifiant()));
    if (c.getAttributs().stream().noneMatch(Attribut::isEstIdentifiant))   // R3
      t.getColonnes().add(0, colonne("id", "BIGINT", true));
    s.getTables().add(t);
    tables.put(c, t);
  }
  for (Association as : pim.getAssociations()) regleR4(as);    // après R1 : la trace est
    complète
  return s;
}

void regleR4(Association as) {                                 // R4 : association -> FK
  Table un = tables.get(as.getDe());  Table plusieurs = tables.get(as.getVers());
  Colonne pk = cle(un);                                        // la colonne clé de A
  Colonne fk = colonne(un.getNom().toLowerCase() + "_id", pk.getType(), false);
  plusieurs.getColonnes().add(fk);
  CleEtrangere ce = f.createCleEtrangere();
  ce.setColonne(fk);  ce.setReference(un);
  plusieurs.getCleEtrangeres().add(ce);
}
```

```
Colonne colonne(String nom, String type, boolean estCle) {
  Colonne k = f.createColonne();  k.setNom(nom);  k.setType(type);  k.setEstCle(estCle);
  return k;
}
Colonne cle(Table t) {
  return t.getColonnes().stream().filter(Colonne::isEstCle).findFirst().orElseThrow();
}
String sqlType(TypePrimitif t) {                                  // le PDM SQL
  switch (t) {
    case STRING:  return "VARCHAR(255)";   case INTEGER: return "INTEGER";
    case REAL:    return "DECIMAL(10,2)";  case BOOLEAN: return "BOOLEAN";
    case DATE:    return "DATE";
    default: throw new IllegalArgumentException("Type non prévu par le PDM : " + t);
  }
}
```

La `Map tables` garde la **trace** « quelle classe a donné quelle table » : elle sert pour R4. La méthode `sqlType()` code le **PDM** ; un type non prévu provoque une erreur explicite plutôt qu'une traduction silencieuse et fausse. Sur le PIM de la figure 4.19, on obtient `ADHERENT(id BIGINT PK, nom, email)` et `LIVRE(isbn PK, titre, adherent_id BIGINT FK)`. □

4.4.3 Transformation par modélisation

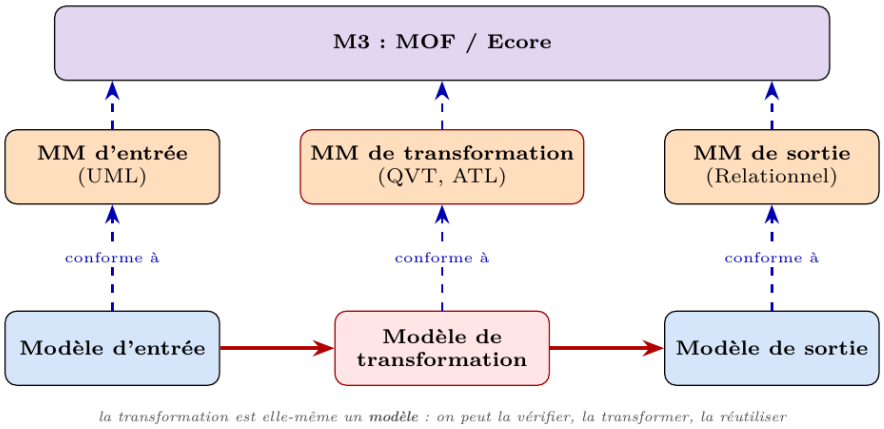


Figure 4.20 – Approche par modélisation : la transformation est un modèle conforme au méta-modèle d'un langage de transformation

La transformation est un **modèle** conforme au méta-modèle d'un langage de transformation (QVT de l'OMG, ATL d'Eclipse). Les règles sont **déclaratives** (« tout X devient un Y ») ; le moteur s'occupe du parcours et des liens (chapitre 7).

Exemple 4.11 – Les règles R1 et R2 en ATL.

```
module PIM2Relationnel;
create OUT : Rel from IN : Pim;

helper def : sqlType(t : Pim!TypePrimitif) : String =
  if t = #STRING then 'VARCHAR(255)'
  else if t = #INTEGER then 'INTEGER'
  else if t = #REAL then 'DECIMAL(10,2)'
  else if t = #BOOLEAN then 'BOOLEAN' else 'DATE' endif endif endif endif;

rule Classe2Table {                          -- R1
```

```
  from c : Pim!Classe
  to   t : Rel!Table ( nom <- c.nom.toUpper(), colonnes <- c.attributs )
}
rule Attribut2Colonne {                    -- R2
  from a : Pim!Attribut
  to   k : Rel!Colonne ( nom <- a.nom, type <- thisModule.sqlType(a.type),
                         estCle <- a.estIdentifiant )
}
```

`colonnes <- c.attributs` : ATL utilise **automatiquement** la règle `Attribut2Colonne` pour relier les colonnes à leur table. Une quinzaine de lignes ATL remplacent la vingtaine de lignes Java de R1 et R2. Les règles R3 et R4 demandent des éléments créés « en plus » ; elles sont traitées au chapitre 7 (règles à plusieurs éléments cibles). □

4.5 Études de cas corrigées

Exemple 4.12 – XML → tableau HTML. Transformer le document suivant en une page HTML contenant un tableau (N°, Nom, Type, Prix) :

```
<test>
  <jeu id="1"><nom>Guild Wars</nom><type>Jeu de rôle en ligne</type><prix>Environ 20 €</prix
    ></jeu>
  <jeu id="2"><nom>Super Mario Galaxy</nom><type>Jeu de plate-forme</type><prix>30-50 €</prix
    ></jeu>
  <jeu id="3"><nom>Mario Sokoban</nom><type>Jeu de réflexion</type><prix>Gratuit</prix></jeu>
</test>
```

Correction.

```
<xsl:template match="/">
 <html><body><table border="1">
   <tr><th>N°</th><th>Nom</th><th>Type</th><th>Prix</th></tr>
   <xsl:for-each select="/test/jeu">
    <tr>
     <td><xsl:value-of select="@id"/></td>   <td><xsl:value-of select="nom"/></td>
     <td><xsl:value-of select="type"/></td>  <td><xsl:value-of select="prix"/></td>
    </tr>
   </xsl:for-each>
 </table></body></html>
</xsl:template>
```

`@id` lit un **attribut** XML ; `nom` lit un **élément** fils. □

Exemple 4.13 – Machine à états → XML par programmation. Avec le méta-modèle de machines à états du chapitre 2 (exemple 2.14) :

```
public static String versXml(StateMachine sm) {
  StringBuilder sb = new StringBuilder();
  sb.append("<stateMachine name=\"").append(sm.getName()).append("\">\n");
  for (State s : sm.getStates())
    sb.append("  <state name=\"").append(s.getName()).append("\"")
      .append(s.getKind() == StateKind.INITIAL ? " initial=\"true\"" : "").append("/>\n");
  for (Transition t : sm.getTransitions())
    sb.append("  <transition from=\"").append(t.getSource().getName())
      .append("\" to=\"").append(t.getTarget().getName())
```

```
      .append("\" event=\"").append(t.getEvent() == null ? "" : t.getEvent()).append("\"/>\n")
    ;
  return sb.append("</stateMachine>\n").toString();
}
```

C'est une transformation **M2T par programmation** : le texte est construit « à la main ». Pour le feu, la première ligne d'état produite est `<state name="Rouge" initial="true"/>`. □

Exemple 4.14 – Modernisation PHP/MySQL → Java/JPA. (1) Schéma MySQL → PSM relationnel : **T2M**, rétro-ingénierie. (2) PSM → PIM : rétro-ingénierie, exogène, verticale, M2M. (3) PIM → PIM : raffinement éventuel (endogène). (4) PIM → PSM Java/JPA : projection, exogène, verticale, M2M. (5) PSM → code Java : génération, M2T. □

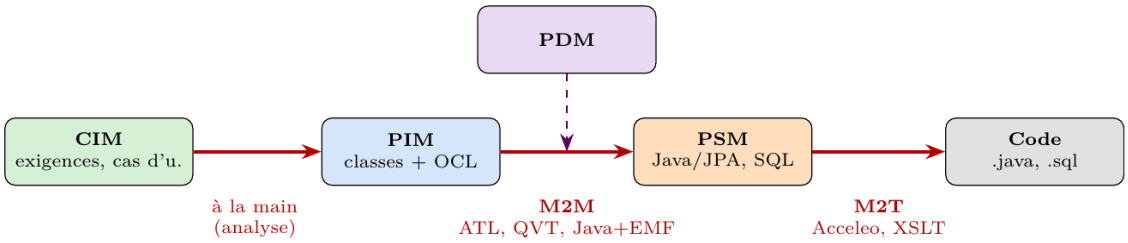


Figure 4.21 – Récapitulatif de la chaîne MDA

4.6 Exemples complémentaires

Exemple 4.15 – Chaîne MDA complète : la gestion des notes. **CIM.** « L'école enregistre les notes des étudiants. Chaque étudiant suit plusieurs modules ; une note est comprise entre 0 et 20 ; un module est validé si la note est au moins 12. La moyenne générale est pondérée par les coefficients des modules. » Cas d'utilisation : *Saisir une note* (enseignant), *Consulter son relevé* (étudiant).

PIM.

- classes `Etudiant(cne : String, nom : String)` et `Module(code : String, coefficient : Integer)` ;
- classe `Note(valeur : Real)` reliant un étudiant (`etudiant [1]`, opposé `notes [0..*]`) et un module (`module [1]`) ;
- opération `Etudiant::moyenne() : Real` et contraintes OCL :

```
context Note inv bornes: self.valeur >= 0 and self.valeur <= 20
context Etudiant inv uneNoteParModule: self.notes->isUnique(n | n.module)
context Etudiant::moyenne() : Real
  pre:  self.notes->notEmpty()
  body: self.notes->collect(n | n.valeur * n.module.coefficient)->sum()
        / self.notes.module.coefficient->sum()
```

PDM. Plateforme Java/JPA + PostgreSQL : `String` → `VARCHAR`, `Real` → `NUMERIC(4,2)`, classe → `@Entity`, association 1–* → clé étrangère.

PSM. `«Entity» Etudiant` avec `«Id» cne`, `«Entity» Note` avec `«ManyToOne» etudiant` et `«ManyToOne» module`.

Code.

```
CREATE TABLE ETUDIANT (cne VARCHAR(20) PRIMARY KEY, nom VARCHAR(100));
CREATE TABLE MODULE   (code VARCHAR(10) PRIMARY KEY, coefficient INTEGER);
CREATE TABLE NOTE (id BIGINT PRIMARY KEY, valeur NUMERIC(4,2) CHECK (valeur BETWEEN 0 AND 20),
  etudiant_cne VARCHAR(20) REFERENCES ETUDIANT(cne), module_code VARCHAR(10) REFERENCES MODULE
    (code),
  UNIQUE (etudiant_cne, module_code));
```

Les contraintes OCL `bornes` et `uneNoteParModule` ont été traduites en clauses `CHECK` et `UNIQUE` : une partie des règles du PIM peut être conservée jusqu'au code. □

Exemple 4.16 – Un PIM, deux PSM : Java/JPA et REST. À partir de la classe `Livre(isbn, titre)` du PIM :

- le PSM **Java/JPA** produit une entité `@Entity class Livre` et un dépôt `LivreRepository` ;
- le PSM **REST** produit une ressource `/livres` avec les opérations `GET /livres`, `GET /livres/{isbn}`, `POST /livres` et un schéma JSON `{"isbn": "string", "titre": "string"}`.

Les deux PSM sont obtenus par deux transformations différentes appliquées au **même** PIM. □

4.7 Exercices

Les corrigés figurent en annexe A.

Exercice 4.1 – Classer des modèles. Pour un hôpital, classer en CIM, PIM, PSM ou code : (a) « un patient est suivi par un médecin référent » ; (b) la classe `Patient(nom, dateNaissance : Date)` ; (c) la table `PATIENT(id BIGINT, nom VARCHAR(80))` ; (d) l'interface `PatientRepository extends JpaRepository` ; (e) le cas d'utilisation « Prendre rendez-vous » ; (f) la classe `«Entity» Patient` avec `«Id» id : Long`.

Exercice 4.2 – Du PIM au PSM relationnel. Appliquer les règles R1 à R4 (tableau 4.5) au PIM suivant et donner le schéma SQL obtenu : `Client(email {id} : String, nom : String)` 0..1 — * `Commande(date : Date, montant : Real)`.

Exercice 4.3 – Taxonomie. Classer (sens MDA, endogène/exogène, horizontale/verticale, M2M/M2T/T2M) : (a) générer une documentation HTML à partir d'un PIM ; (b) supprimer les attributs inutilisés d'un PIM ; (c) lire un fichier de configuration XML pour construire un modèle ; (d) transformer un modèle BPMN en diagramme d'activités UML.

Exercice 4.4 – XSLT. Écrire une feuille XSLT qui transforme le document suivant en une liste HTML « Titre (année) » triée par année (`<xsl:sort select="@annee"/>`).

```
<films>
  <film annee="2010"><titre>Inception</titre></film>
  <film annee="1999"><titre>Matrix</titre></film>
</films>
```

Exercice 4.5 – Règles de projection. Écrire, en français puis en ATL, les règles qui transforment un PIM en PSM Java : classe → classe Java ; attribut → champ privé avec getter ; extrémité d'association * → champ `List<...>`.

Exercice 4.6 – Modernisation. Une application COBOL gère des contrats d'assurance dans des fichiers. On veut une application Java avec une base PostgreSQL et une API REST. Proposer la chaîne de transformations MDA et dire quels modèles on peut **réutiliser** si l'on veut plus tard une application mobile.

4.8 Quiz

Répondez aux questions suivantes. Les corrigés commentés figurent en annexe A.

Q1. MDA a été proposée par :

a) Eclipse
b) l'OMG
c) le W3C
d) l'ISO

Q2. Quel modèle décrit la solution logicielle sans référence à une technologie ?

a) CIM
b) PIM
c) PSM
d) PDM

Q3. Quelle formule résume MDA ?

a) CIM + PSM ⇒ PIM
b) PIM + PDM ⇒ PSM ⇒ Code
c) PSM + PDM ⇒ PIM
d) Code ⇒ PIM ⇒ CIM

Q4. La classe `Livre` annotée `«Entity»` avec un attribut `«Id» isbn` appartient au :

a) CIM
b) PIM
c) PSM
d) PDM

Q5. Le PIM de la bibliothèque est, du point de vue des niveaux OMG :

a) un méta-modèle M2
b) un modèle M1
c) des données M0

d) un méta-méta-modèle M3

Q6. Renommer « Adherent » en « Membre » dans le PIM est une transformation :

a) exogène et verticale
b) endogène et horizontale
c) exogène et horizontale
d) endogène et verticale

Q7. Passer d'un schéma SQL existant à un diagramme de classes s'appelle :

a) projection
b) raffinement
c) rétro-ingénierie
d) génération

Q8. PSM → Code est typiquement une transformation :

a) M2M
b) M2T
c) T2M
d) endogène

Q9. XSLT et Acceleo relèvent de l'approche :

a) par template
b) par programmation
c) par modélisation
d) par rétro-ingénierie

Q10. Dans la transformation Java PIM → relationnel, à quoi sert la `Map<Classe, Table>` ?

a) à stocker le PDM
b) à garder la trace source → cible pour relier les éléments (règle R4)
c) à trier les tables
d) à valider le PIM

Q11. Classer en CIM, PIM, PSM ou Code : (a) « un adhérent emprunte au plus 3 livres » ; (b) la classe Livre avec `isbn : String` ; (c) la table LIVRE avec `adherent_id BIGINT` ; (d) le fichier `Livre.java` ; (e) le cas d'utilisation « Rendre un livre ».

Q12. Quelle approche de transformation choisir pour : (a) générer 40 pages HTML régulières ; (b) une transformation PIM → PSM réutilisable sur plusieurs projets ?

Chapitre 5

Transformation M2T par template avec Acceleo

Résumé

*Écrire quatre classes Java, c'est répéter quatre fois la même forme : seuls les noms et les types changent, et ils sont **déjà écrits dans le modèle**. La génération de code par **template** résout ce problème : un template est un **texte à trous** ; un moteur parcourt le modèle et remplit les trous. **Acceleo**, implantation Eclipse du standard **MOFM2T** de l'OMG, lit tout modèle EMF et produit n'importe quel texte. Ce chapitre présente la structure d'un module Acceleo (`.mtl`), les expressions OCL de navigation dans un modèle Ecore, et les instructions `[for]`, `[if]`, `[let]`, `[query]`, `[file]`, `[protected]` et `[comment]`, chacune illustrée par plusieurs exemples. Un cas complet génère, à partir de `library.ecore`, des classes **Python**, un script **SQL** (clés primaires et étrangères) et des classes **Java** (constructeurs, accesseurs, zones protégées).*

Objectifs d'apprentissage

À l'issue de ce chapitre, vous serez capable de :

- expliquer ce qu'est une transformation **M2T par template** et sa place dans MDA ;
- créer un projet Acceleo, écrire un module et le lancer sur un modèle ;
- naviguer dans un modèle Ecore en OCL (`root.eClassifiers`, `cls.eAttributes`, `->filter(EClass)`) ;
- utiliser `[for]`, `[if]`, `[let]`, `[file]`, `[query]`, `[protected]` ;
- générer du Python, du SQL et du Java à partir de `library.ecore` ;
- diagnostiquer les erreurs fréquentes et appliquer les bonnes pratiques.

Contenu du chapitre

5.1 Pourquoi générer du code ? . . . 71
5.1.1 Acceleo en bref . . . 71
5.2 Premier projet Acceleo . . . 72
5.2.1 Le modèle d'entrée . . . 72
5.2.2 Installation et création du projet . . . 72
5.2.3 Anatomie d'un module . . . 73
5.3 Le langage pas à pas . . . 74

5.3.1 Texte statique et expressions . . . 74
5.3.2 Naviguer dans le modèle avec OCL . . . 74
5.3.3 La boucle [for] . . . 74
5.3.4 La conditionnelle [if] . . . 76
5.3.5 La variable locale [let] . . . 77
5.3.6 Les [query] . . . 77
5.3.7 Un fichier par élément : [file] . . . 78
5.3.8 Zones protégées et commentaires . . . 79
5.4 Cas complet : library.ecore vers Python, SQL et Java . . . 79
5.4.1 Génération Python . . . 79
5.4.2 Génération SQL . . . 80
5.4.3 Génération Java . . . 81
5.5 Bonnes pratiques et erreurs fréquentes . . . 83
5.6 Exemples complémentaires . . . 83
5.7 Exercices . . . 85
5.8 Quiz . . . 86

5.1 Pourquoi générer du code ?

Exemple 5.1 – Le code « mécanique ».

```
public class Book {
    private String title;
    public String getTitle() {
        return this.title; }
    public void setTitle(String title) {
        this.title = title; }
}
```

```
public class Member {
    private String name;
    public String getName() {
        return this.name; }
    public void setName(String name) {
        this.name = name; }
}
```

Les deux classes ont exactement la même forme : seuls les **noms** et les **types** changent. Or ces informations sont déjà dans `library.ecore`. Pourquoi les retaper ? □

Générer le code apporte : la **productivité** (un template écrit une fois produit le code de 4, 40 ou 400 classes) ; la **qualité et la cohérence** (pas de faute de frappe, mêmes conventions partout) ; **plusieurs cibles** (Python, SQL, Java, HTML : un template par cible) ; le **suivi des évolutions** (le modèle change, on régénère).

À retenir. En IDM, le **modèle** est la source de vérité ; le code en est **dérivé** automatiquement.

Analogie : le **publipostage**. Une lettre type (le template) et une liste de destinataires (le modèle) donnent autant de lettres que de lignes.

5.1.1 Acceleo en bref

- Générateur open source de la fondation **Eclipse**, créé par la société Obeo.

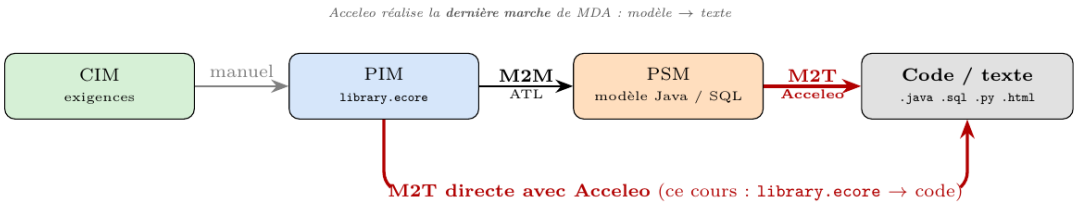


Figure 5.1 – Place d'Acceleo dans MDA : la dernière marche, du modèle au texte

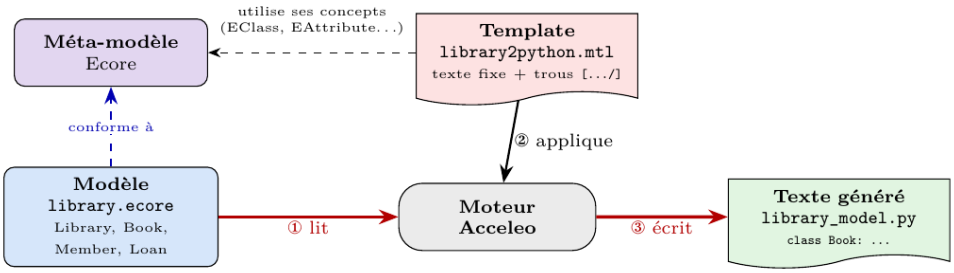


Figure 5.2 – Principe de la génération par template : modèle + template + moteur

- Implémente le standard **MOFM2T** de l'OMG (*MOF Model to Text Transformation Language*).
- Lit tout modèle **EMF** (`.ecore`, `.xmi`, `.uml`) et produit n'importe quel texte.
- Expressions en **OCL** (Acceleo 3, utilisé dans ce cours) ou en **AQL** (Acceleo 4).

5.2 Premier projet Acceleo

5.2.1 Le modèle d'entrée

Le modèle d'entrée est `library.ecore` (figure 2.2, chapitre 2). Pour Acceleo, c'est un **modèle** conforme au méta-modèle **Ecore** (figure 2.3) : le template parcourt des objets `EPackage`, `EClass`, `EAttribute` et `EReference`. Un template peut aussi prendre en entrée un modèle `.xmi` : il faut alors déclarer, dans `[module]`, le `nsURI` du méta-modèle de ce modèle (par exemple `http://fr.obeo.library/1.0`) et le template parcourt des `Library`, `Book`... (exemple 5.16).

5.2.2 Installation et création du projet

1. Prérequis : Eclipse Modeling Tools et JDK 17+.
2. *Help* → *Install New Software* ; site Acceleo 3.7 : `https://download.eclipse.org/acceleo/updates/releases/3.7` ; cocher Acceleo, installer, redémarrer.
3. *File* → *New* → *Acceleo Project* ; nom du module ; méta-modèle `http://www.eclipse.org/emf/2002/Ecore` ; type de l'élément racine : `EPackage` ; cocher *Generate file* et *Main template*.
4. Copier `library.ecore` dans un dossier `model/` ; créer un dossier `gen/`.

Attention. Le site `.../releases/latest` installe **Acceleo 4** (syntaxe AQL). Pour suivre ce cours tel quel, installer **Acceleo 3.7**.

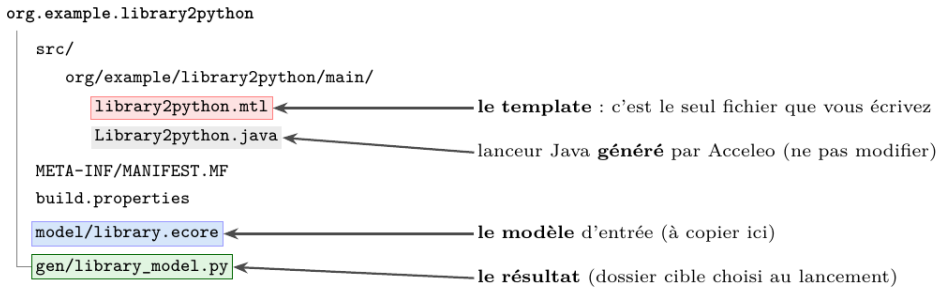


Figure 5.3 – Structure d'un projet Acceleo

5.2.3 Anatomie d'un module

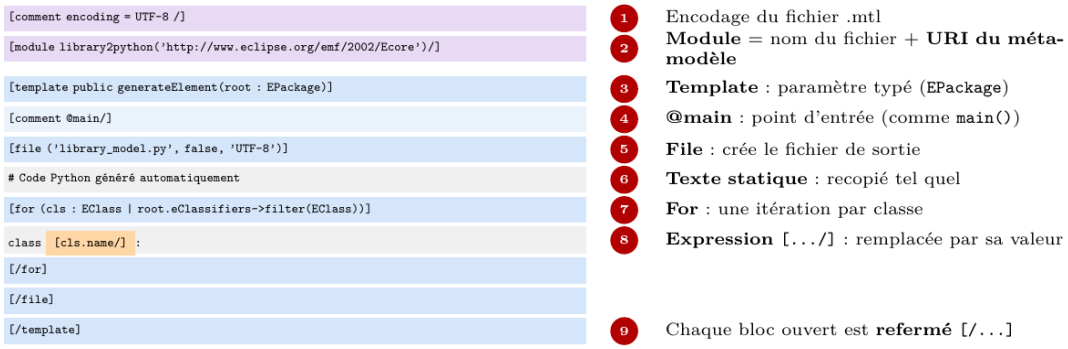


Figure 5.4 – Anatomie d'un module Acceleo (`.mtl`)

Exemple 5.2 – Étape 1 : le point d'entrée.

```
[comment encoding = UTF-8 /]
[module library2python('http://www.eclipse.org/emf/2002/Ecore')/]

[template public generateElement(root : EPackage)]
[comment @main/]
[file ('library_model.py', false, 'UTF-8')]
# -*- coding: utf-8 -*-
# Code Python généré automatiquement à partir du modèle Ecore
[/file]
[/template]
```

- `[module nom('URI')/]` : nom du module (identique au nom du fichier `.mtl`) et méta-modèle d'entrée (ici Ecore) ;
- `[template public nom(param : Type)]` : un template ; `[comment @main/]` en fait le **point d'entrée** ;
- `[file (chemin, false, 'UTF-8')]` : fichier de sortie, relatif au dossier cible ; `false` = écraser, `true` = ajouter à la fin.

On lance par *Run As* → *Launch Acceleo Application* (Model : `library.ecore`, Target : `gen`). Ici, tout est statique : on obtient seulement l'en-tête. □

Exemple 5.3 – Étape 2 : une classe Python par EClass.

```
[file ('library_model.py', false, 'UTF-8')]
# -*- coding: utf-8 -*-
[for (cls : EClass | root.eClassifiers->filter(EClass))]

class [cls.name/]:
    pass
[/for]
[/file]
```

Résultat : `class Library:`, `class Book:`, `class Member:`, `class Loan:` (chacune avec `pass`). `root.eClassifiers` contient les classes **et** les types ; `->filter(EClass)` ne garde que les `EClass`. Si l'on ajoute une classe `Author` au modèle, **rien** n'est à modifier dans le template. □

5.3 Le langage pas à pas

5.3.1 Texte statique et expressions

Tout ce qui n'est pas entre crochets est **recopié tel quel**. `[expression/]` évalue l'expression sur le modèle et écrit le résultat (le « `/` » ferme l'expression). Pour écrire un crochet dans le résultat, on écrit une chaîne : `['[']/]` et `[']']/]`.

Exemple 5.4 – Quatre expressions sur `library.ecore`.

Template	Résultat (pour Book)
`Classe [cls.name/]`	`Classe Book`
`[cls.name.toUpper()/]`	`BOOK`
`[cls.eAttributes->size()/] attributs`	`5 attributs`
`tab['[']0[']']/]`	`tab[0]`

□

5.3.2 Naviguer dans le modèle avec OCL

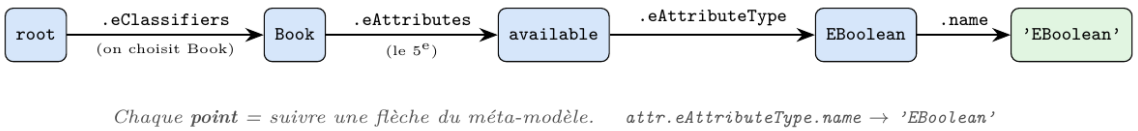


Figure 5.5 – Le point suit une flèche du méta-modèle Ecore

5.3.3 La boucle `[for]`

```
[for (variable : Type | collection) separator(', ')]
    ... texte répété ...
[/for]
```

Le `separator` se place **entre** les éléments, jamais après le dernier. Dans le constructeur Python, la virgule est placée *devant* chaque attribut (après `self`) : pas besoin de séparateur.

Tableau 5.1 – Expressions de navigation dans un modèle Ecore

Expression	Signification	Valeur sur library.ecore
`root.name`	nom du paquetage	`'library'`
`root.eClassifiers->filter(EClass)`	les classes	Library, Book, Member, Loan
`cls.eAttributes`	attributs d'une classe	Book : id, title, isbn, author, available
`cls.eReferences`	références	Loan : book, member
`attr.eAttributeType.name`	nom du type Ecore	`'EString'`, `'EBoolean'`, `'EDate'`
`ref.eReferenceType.name`	classe cible	Loan.book → `'Book'`
`ref.many`, `ref.containment`	multiple ? composition ?	Library.book : true, true

Tableau 5.2 – Opérations OCL utiles dans les templates

Opération	Exemple	Résultat
`->size()`	`cls.eAttributes->size()`	Book → 5
`->select(x | c)`	`cls.eAttributes->select(a | a.lowerBound = 1)`	attributs obligatoires
`->exists(x | c)`	`cls.eReferences->exists(r | r.many)`	vrai pour Library
`->first()`, `->isEmpty()`	`cls.eAttributes->first().name`	Book → `'id'`
`toUpperFirst()`	`attr.name.toUpperFirst()`	`'title'` → `'Title'`
`+`, `concat()`	`'get' + attr.name.toUpperFirst()`	`'getTitle'`
`endsWith()`	`attr.name.endsWith('Id')`	vrai pour `memberId`

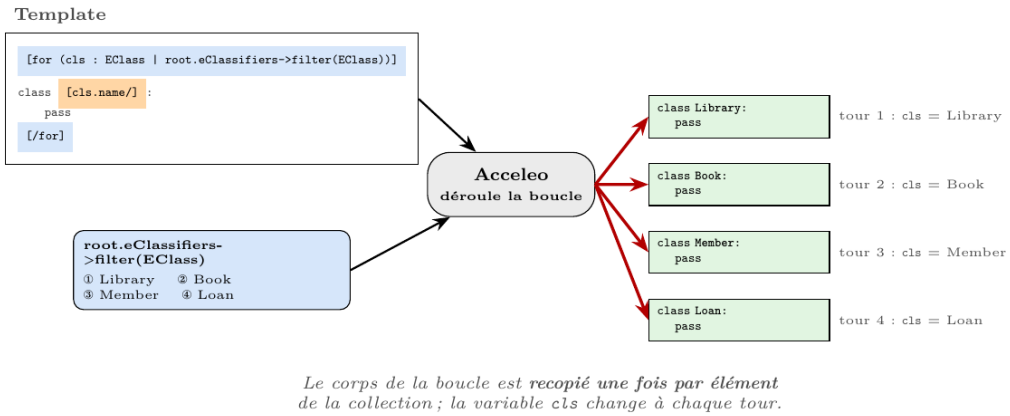


Figure 5.6 – Fonctionnement de la boucle `[for]`

```
def __init__(self[for (attr : EAttribute | cls.eAttributes)], [attr.name/]=None[/for]):
```

Résultat pour Book : `def __init__(self, id=None, title=None, isbn=None, author=None, available=None):`

Exemple 5.5 – Trois boucles, trois formes de sortie.

```
[comment 1. Liste en ligne avec séparateur : en-tête CSV /]
[for (a : EAttribute | cls.eAttributes) separator(',')][a.name/][/for]

[comment 2. Une ligne par élément : énumération Java /]
public enum [e.name/] { [for (l : EEnumLiteral | e.eLiterals) separator(', ')][l.name/][/for] }


[comment 3. Boucles imbriquées : rapport des références /]
[for (c : EClass | root.eClassifiers->filter(EClass))]
[for (r : EReference | c.eReferences)]
[c.name/].[r.name/] -> [r.eReferenceType.name/]
[/for]
[/for]
```

Résultats : `memberId,name,email` pour Member ; `public enum StateKind { INITIAL, NORMAL, FINAL }` pour le méta-modèle de machines à états ; `Library.book -> Book`, …, `Loan.member -> Member` pour la bibliothèque. □

5.3.4 La conditionnelle `[if]`

```
[for (attr : EAttribute | cls.eAttributes)]
[if (attr.eAttributeType.name = 'EBoolean')]
    # [attr.name/] est un booléen
[elseif (attr.lowerBound = 1)]
    # [attr.name/] est obligatoire
[else]
    # [attr.name/] est facultatif
[/if]
[/for]
```

Pour `Member` : « memberId est obligatoire », « name est obligatoire », « email est facultatif ». Opérateurs : `=`, `<>`, `<`, `>`, `and`, `or`, `not`.

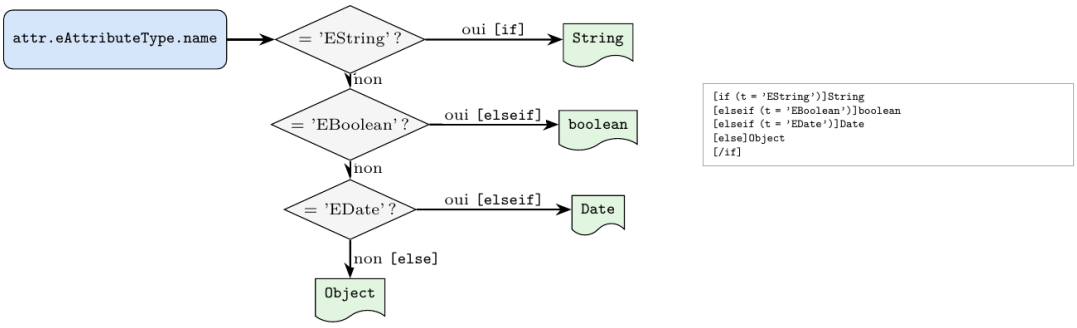


Figure 5.7 – La conditionnelle `[if]` appliquée aux attributs

Exemple 5.6 – Trois usages typiques de `[if]`.

- **Pluriel** : `[n/] attribut[if (n > 1)]s[/if]` → « 1 attribut », « 5 attributs ».
- **Mot-clé selon la nature** : `public [if (cls.abstract)]abstract [/if]class [cls.name/]` → `public abstract class Item` pour la médiathèque.
- **Héritage optionnel** : `class [cls.name/][if (cls.eSuperTypes->notEmpty())] extends [cls.eSuperTypes->first().name/][/if]` → `class Dvd extends Item`, mais `class Loan`.

□

5.3.5 La variable locale `[let]`

```
[let n : Integer = cls.eAttributes->size()]
# [cls.name/] : [n/] attribut[if (n > 1)]s[/if]
[/let]
```

La variable n'existe qu'entre `[let]` et `[/let]` et n'est **pas modifiable**.

Attention. En Acceleo 3, `[let]` est un **bloc** : il doit être refermé par `[/let]`.

Exemple 5.7 – `[let]` pour éviter de recalculer.

```
[let oblig : OrderedSet(EAttribute) = cls.eAttributes->select(a | a.lowerBound > 0)]
# [cls.name/] : [oblig->size()/] champ(s) obligatoire(s) : [for (a : EAttribute | oblig)
    separator(', ')][a.name/][/for]
[/let]
```

Pour Loan : « Loan : 1 champ(s) obligatoire(s) : loanDate ». □

5.3.6 Les `[query]`

Une **query** est une petite fonction qui **calcule une valeur** (un template, lui, produit du texte). Elle sert notamment à coder le **PDM** : la correspondance des types.

```
[query public toSQLType(typeName : String) : String =
    if typeName = 'EString' then 'VARCHAR(100)'
    else if typeName = 'EInt' then 'INT'
    else if typeName = 'EBoolean' then 'BOOLEAN'
    else if typeName = 'EDate' then 'DATE'
    else 'VARCHAR(255)'
    endif endif endif endif
/]
```

Appel : `[attr.eAttributeType.name.toSQLType()/]` ; `'EDate'.toSQLType()` donne `'DATE'`.

Exemple 5.8 – Trois autres queries.

```
[query public isRequired(f : EStructuralFeature) : Boolean = f.lowerBound > 0 /]
[query public plural(s : String) : String =
    if s.endsWith('y') then s.substring(1, s.size() - 1) + 'ies' else s + 's' endif /]
[query public nbObligatoires(c : EClass) : Integer =
    c.eAttributes->select(a | a.isRequired())->size() /]
```

`'Library'.plural()` donne `'Libraries'` ; `'Book'.plural()` donne `'Books'` ; `Member.nbObligatoires()` vaut 2. □

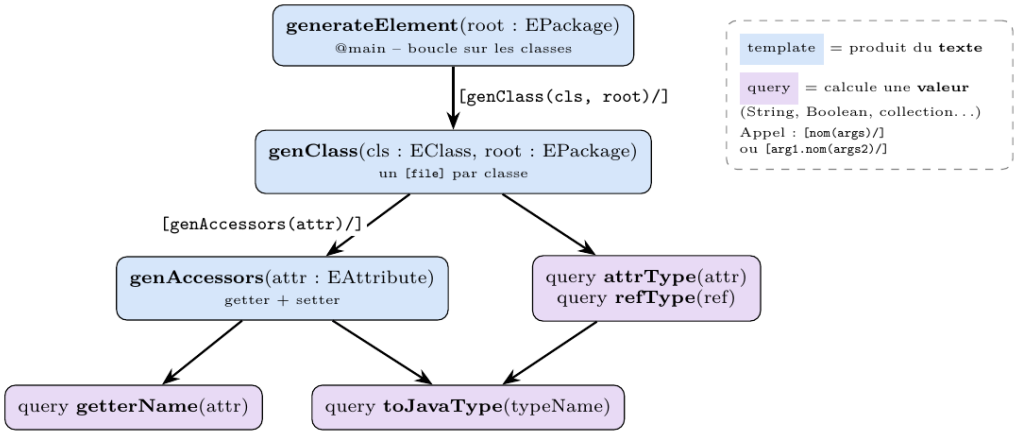


Figure 5.8 – Templates et queries : découper un générateur en morceaux

Un template peut en appeler un autre : `[genClass(cls, root)/]`, ou de façon équivalente `[cls.genClass(root)/]` (le premier argument devant le point).

5.3.7 Un fichier par élément : `[file]`

```
[file (root.name + '/' + cls.name + '.java', false, 'UTF-8')] ... [/file]
```

Un `[file]` placé **dans une boucle** crée **un fichier par tour** de boucle.

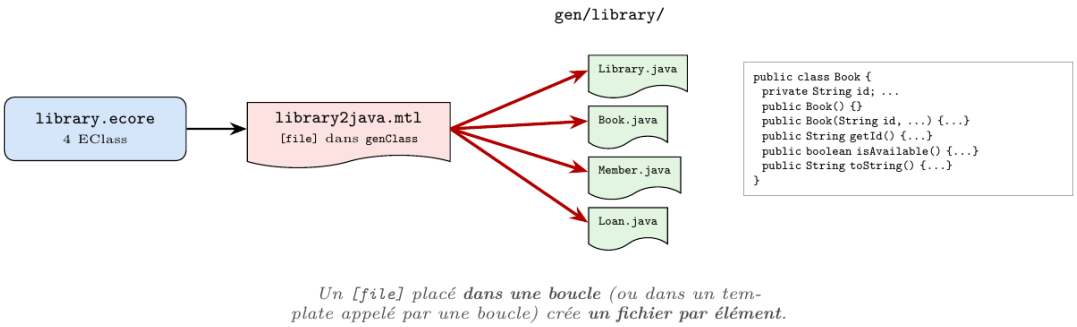


Figure 5.9 – Un `[file]` dans une boucle : un fichier Java par EClass

Exemple 5.9 – Trois stratégies de fichiers.

- **Un seul fichier** pour tout le modèle : `[file ('schema.sql', false, 'UTF-8')]` hors de la boucle (génération SQL).
- **Un fichier par classe** : `[file ('library/' + cls.name + '.java', ...)]` dans la boucle (`library/Book.java`, `library/Loan.java`...).
- **Un fichier qui s'allonge** : `[file ('journal.txt', true, 'UTF-8')]` ajoute une ligne à chaque génération sans effacer les précédentes.

□

5.3.8 Zones protégées et commentaires

```
    // [protected ('methodes ' + cls.name)]
    // Ajoutez ici votre code : il sera conservé
    // [/protected]
```

Dans le fichier généré, la zone devient `// Start of user code methodes Book ... // End of user code`. Le code écrit **à l'intérieur** est conservé lors d'une régénération ; tout code écrit **en dehors** est écrasé. L'identifiant (`'methodes Book'`) doit être **unique** dans le fichier.

`[comment texte /]` est ignoré par le moteur ; `[comment @main/]` marque le point d'entrée ; `[comment encoding = UTF-8 /]` fixe l'encodage du `.mtl`. Un commentaire destiné au code produit s'écrit comme du texte statique (`#`, `//`, `-`).

Attention. **Espaces et retours à la ligne** sont la cause n°1 des « résultats bizarres » : une ligne vide ou une indentation dans une boucle est reproduite à *chaque tour*. Indentez le **texte** comme dans le fichier final, pas les balises.

5.4 Cas complet : `library.ecore` vers Python, SQL et Java

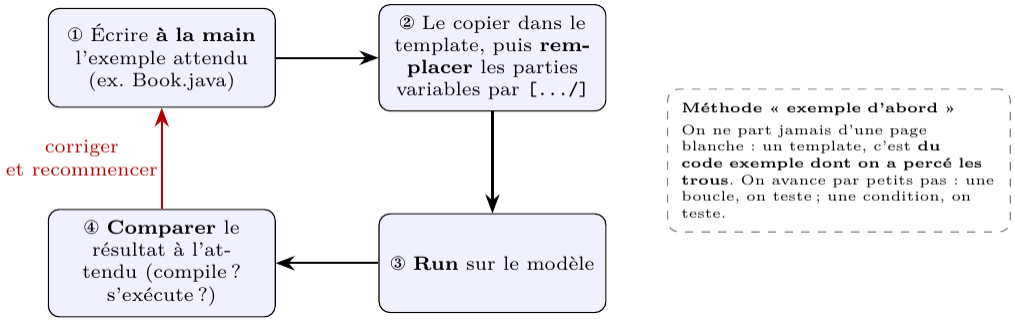


Figure 5.10 – Méthode « l'exemple d'abord » pour écrire un template

5.4.1 Génération Python

Exemple 5.10 – `library2python.mtl`.

```
[file ('library_model.py', false, 'UTF-8')]
# -*- coding: utf-8 -*-
[for (cls : EClass | root.eClassifiers->filter(EClass))]


class [cls.name/]:
    def __init__(self[for (attr : EAttribute | cls.eAttributes)], [attr.name/]=None[/for]):
[for (attr : EAttribute | cls.eAttributes)]
        self.[attr.name/] = [attr.name/]
[/for]
[for (ref : EReference | cls.eReferences)]
        self.[ref.name/] = [ref.initValue()/]
[/for]
[if (cls.eAttributes->isEmpty() and cls.eReferences->isEmpty())]
        pass
[/if]
```

```
[/for]
[/file]

[query public initValue(ref : EReference) : String =
    if ref.many then '[]' else 'None' endif /]
```

Une référence multiple (`0..*`) devient une liste vide `[]`, une référence simple (`0..1`) devient `None` : on respecte ainsi la multiplicité. Le `[if]` final évite un corps vide (erreur Python) pour une classe sans propriété.

```
class Loan:
    def __init__(self, loanDate=None, returnDate=None):
        self.loanDate = loanDate
        self.returnDate = returnDate
        self.book = None
        self.member = None
```

□

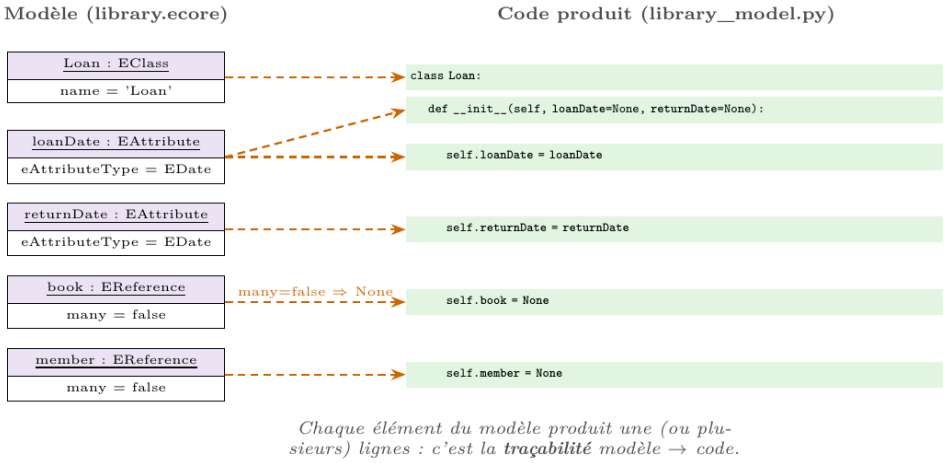


Figure 5.11 – Traçabilité modèle → code Python : d'où vient chaque ligne ?

5.4.2 Génération SQL

Règles : une table par classe ; une colonne par attribut (`NOT NULL` si `lowerBound > 0`) ; clé primaire = attribut nommé `id` ou se terminant par `Id`, sinon colonne technique `id INT` ; chaque référence simple et non contenante devient une **clé étrangère**.

Exemple 5.11 – `library2sql.mtl` : corps et queries.

```
[for (cls : EClass | root.eClassifiers->filter(EClass))]

CREATE TABLE [cls.name/] (
[if (not cls.hasPK())]
    id INT NOT NULL,
[/if]
[for (attr : EAttribute | cls.eAttributes)]
    [attr.name/] [attr.eAttributeType.name.toSQLType()/][if (attr.lowerBound > 0)] NOT NULL[/
    if],
[/for]
[for (ref : EReference | cls.eReferences->select(r | not r.containment and not r.many))]
```

```
    [ref.name/]_id [ref.eReferenceType.pkType()/],
[/for]
    PRIMARY KEY ([cls.pkName()/])[for (ref : EReference | cls.eReferences->select(r | not r.
    containment and not r.many))],
    FOREIGN KEY ([ref.name/]_id) REFERENCES [ref.eReferenceType.name/]([ref.eReferenceType.
    pkName()/])[/for]
);
[/for]

[query public isPK(attr : EAttribute) : Boolean = attr.name = 'id' or attr.name.endsWith('Id')
     /]
[query public hasPK(cls : EClass) : Boolean = cls.eAttributes->exists(a | a.isPK()) /]
[query public pkName(cls : EClass) : String =
    if cls.hasPK() then cls.eAttributes->select(a | a.isPK())->first().name else 'id' endif /]
[query public pkType(cls : EClass) : String =
    if cls.hasPK() then cls.eAttributes->select(a | a.isPK())->first().eAttributeType.name.
    toSQLType()
    else 'INT' endif /]
```

Astuce « anti-virgule » : `PRIMARY KEY` est toujours la dernière ligne ; chaque ligne précédente se termine par une virgule et chaque `FOREIGN KEY` *commence* par une virgule. Résultat :

```
CREATE TABLE Loan (
    id INT NOT NULL,
    loanDate DATE NOT NULL,
    returnDate DATE,
    book_id VARCHAR(100),
    member_id VARCHAR(100),
    PRIMARY KEY (id),
    FOREIGN KEY (book_id) REFERENCES Book(id),
    FOREIGN KEY (member_id) REFERENCES Member(memberId)
);
```

Limite volontaire : les compositions (`Library.book...`) ne produisent aucune colonne ; pour garder le lien « ce livre appartient à cette bibliothèque », il faudrait ajouter dans `Book` une clé étrangère `library_id` (côté contenu) ; voir le corrigé de l'exercice 5.3. □

5.4.3 Génération Java

Exemple 5.12 – `library2java.mtl` : un fichier par classe.

```
[template public generateElement(root : EPackage)]
[comment @main/]
[for (cls : EClass | root.eClassifiers->filter(EClass))]
[genClass(cls, root)/]
[/for]
[/template]

[template public genClass(cls : EClass, root : EPackage)]
[file (root.name + '/' + cls.name + '.java', false, 'UTF-8')]
package [root.name/];

import java.util.ArrayList;
import java.util.Date;
import java.util.List;

public class [cls.name/] {

[for (attr : EAttribute | cls.eAttributes)]
```

```
    private [attr.attrType()/] [attr.name/];
[/for]
[for (ref : EReference | cls.eReferences)]
    private [ref.refType()/] [ref.name/][if (ref.many)] = new ArrayList<>()[/if];
[/for]

    public [cls.name/]() {
    }

    public [cls.name/]([for (attr : EAttribute | cls.eAttributes) separator(', ')][attr.
    attrType()/] [attr.name/][/for]) {
[for (attr : EAttribute | cls.eAttributes)]
        this.[attr.name/] = [attr.name/];
[/for]
    }
[for (attr : EAttribute | cls.eAttributes)]
[genAccessors(attr)/]
[/for]

    // [protected ('methodes ' + cls.name)]
    // Ajoutez ici votre code : il sera conservé
    // [/protected]
}
[/file]
[/template]

[template public genAccessors(attr : EAttribute)]

    public [attr.attrType()/] [attr.getterName()/]() {
        return this.[attr.name/];
    }

    public void set[attr.name.toUpperFirst()/]([attr.attrType()/] [attr.name/]) {
        this.[attr.name/] = [attr.name/];
    }
[/template]
```

□

```
[query public toJavaType(typeName : String) : String =
    if typeName = 'EString' then 'String'
    else if typeName = 'EInt' then 'int'
    else if typeName = 'EBoolean' then 'boolean'
    else if typeName = 'EDouble' then 'double'
    else if typeName = 'EDate' then 'Date'
    else 'Object' endif endif endif endif endif /]
[query public attrType(attr : EAttribute) : String = attr.eAttributeType.name.toJavaType() /]
[query public refType(ref : EReference) : String =
    if ref.many then 'List<' + ref.eReferenceType.name + '>' else ref.eReferenceType.name
    endif /]
[query public getterName(attr : EAttribute) : String =
    (if attr.eAttributeType.name = 'EBoolean' then 'is' else 'get' endif) + attr.name.
    toUpperFirst() /]
```

Listing 5.1 – Queries de traduction des types Ecore → Java

Les templates fonctionnent **sans modification** sur `school.ecore` (Student, Course, Enrollment). Ajouter une cible (TypeScript, C#) revient à écrire **un template de plus**.

Tableau 5.3 – Correspondance des types Ecore, Java et SQL

Ecore	Java	SQL
`EString`	`String`	`VARCHAR(100)`
`EInt`	`int`	`INT`
`EBoolean`	`boolean` (getter `isXxx()`)	`BOOLEAN`
`EDouble`	`double`	`DECIMAL(10,2)`
`EDate`	`Date`	`DATE`
référence `0..1` vers Book	`Book`	clé étrangère `book_id`
référence `0..*` vers Book	`List<Book>`	–

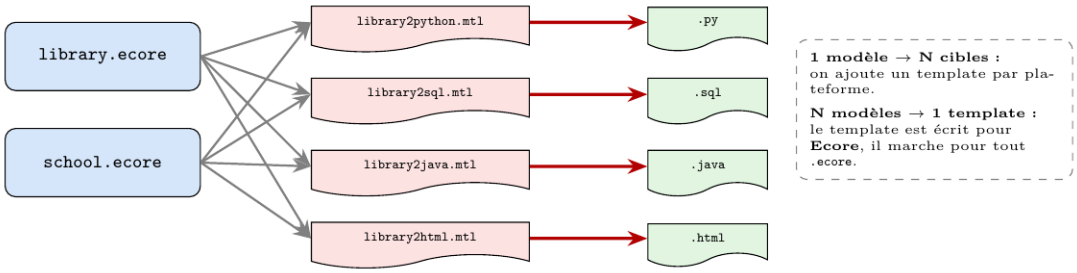


Figure 5.12 – Un modèle, plusieurs cibles… et plusieurs modèles (`school.ecore`)

5.5 Bonnes pratiques et erreurs fréquentes

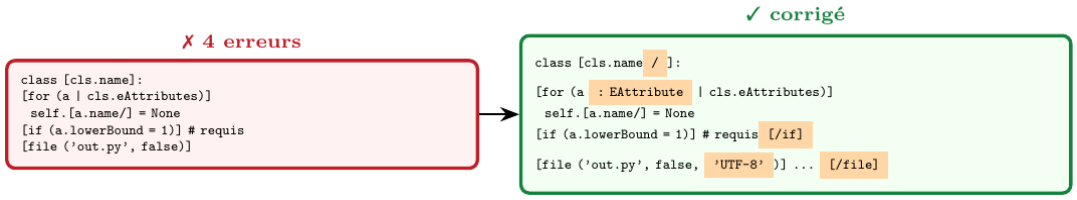


Figure 5.13 – Les erreurs les plus fréquentes en Acceleo

À retenir.

- **Petits templates** : un par morceau de fichier ; un module par cible.
- **Queries pour la plateforme** : types, conventions, clés.
- **Code généré = jetable** : on ne le modifie jamais, sauf dans les zones protégées.
- **Tester le résultat** : le code doit compiler, s'exécuter ou se charger.

Acceleo 4. Même idée, syntaxe AQL : types qualifiés (`ecore::EClass`), `[file (..., 'overwrite', ...)]` au lieu de `false`, corps de blocs indenté de 2 espaces (non générés), migration automatique Acceleo 3 → 4 proposée par Eclipse.

5.6 Exemples complémentaires

Exemple 5.13 – Générer une documentation HTML du modèle.

Tableau 5.4 – Diagnostic des erreurs Acceleo

Symptôme	Cause probable
« Invalid expression » sur une ligne	`/]` oublié, ou parenthèse non fermée
Rien n'est généré	pas de `[comment @main/]`, mauvais modèle dans la configuration, ou `[file]` absent
`Unknown type EClass`	URI du module incorrecte (doit être celle d'Ecore)
Virgule ou lignes vides en trop	`separator` oublié ; ligne vide ou indentation dans une boucle
Le code ajouté à la main disparaît	écrit hors d'une zone `[protected]`

```
[template public genDoc(root : EPackage)]
[comment @main/]
[file ('doc/' + root.name + '.html', false, 'UTF-8')]
<html><body>
<h1>Paquetage [root.name/]</h1>
[for (cls : EClass | root.eClassifiers->filter(EClass))]
<h2>[cls.name/][if (cls.abstract)] (abstraite)[/if]</h2>
<table border="1">
  <tr><th>Attribut</th><th>Type</th><th>Obligatoire</th></tr>
[for (a : EAttribute | cls.eAttributes)]
  <tr><td>[a.name/]</td><td>[a.eAttributeType.name/]</td><td>[if (a.lowerBound > 0)]oui[else]
    non[/if]</td></tr>
[/for]
</table>
[/for]
</body></html>
[/file]
[/template]
```

Pour `Member`, on obtient un tableau de trois lignes : `memberId | EString | oui`, `name | EString | oui`, `email | EString | non`. □

Exemple 5.14 – Un fichier Markdown par classe, avec compteur.

```
[for (cls : EClass | root.eClassifiers->filter(EClass))]
[file ('md/' + cls.name + '.md', false, 'UTF-8')]
# [cls.name/]

[let n : Integer = cls.eAttributes->size()]
La classe [cls.name/] a [n/] attribut[if (n > 1)]s[/if] et [cls.eReferences->size()/] référé
    rence(s).
[/let]

[for (r : EReference | cls.eReferences)]
- [r.name/] -> [r.eReferenceType.name/] ([if (r.many)]plusieurs[else]un[/if][if (r.containment
    )], composition[/if])
[/for]
[/file]
[/for]
```

Fichier `md/Loan.md` : « La classe Loan a 2 attributs et 2 référence(s). », puis `- book -> Book (un)` et `- member -> Member (un)`. □

Exemple 5.15 – Une query récursive : tous les attributs hérités.

```
[query public tousAttributs(c : EClass) : Sequence(EAttribute) =
```

```
    c.eSuperTypes->collect(s | s.tousAttributs())->flatten()->union(c.eAttributes->asSequence
    ()) /]
```

Avec `Book` héritant de `Item(id, title)`, `Book.tousAttributs()` renvoie `id, title, isbn`. Ecore fournit aussi directement `eAllAttributes`. □

Exemple 5.16 – Un template sur un modèle `.xmi` : la machine à états du feu en C. Le module déclare cette fois le méta-modèle des machines à états (`[module sm2c('http://ensias.ma/sm/1.0')/]`) et prend un `StateMachine` en paramètre :

```
[template public genC(sm : StateMachine)]
[comment @main/]
[file (sm.name.toLower() + '.c', false, 'UTF-8')]
typedef enum { [for (s : State | sm.states) separator(', ')][s.name.toUpper()/][/for] } Etat;

Etat suivant(Etat e) {
    switch (e) {
[for (t : Transition | sm.transitions)]
    case [t.source.name.toUpper()/]: return [t.target.name.toUpper()/];   /* [t.event/] */
[/for]
    }
    return e;
}
[/file]
[/template]
```

Sur le modèle du feu (exemple 2.14), on obtient `typedef enum { ROUGE, VERT, ORANGE } Etat;` et trois `case`, dont `case ROUGE: return VERT; /* after(25s) */`. C'est l'étape PSM → Code du feu dans la chaîne MDA. □

5.7 Exercices

Les corrigés figurent en annexe A.

Exercice 5.1 – Prédire le résultat. Que produit ce template pour la classe `Member` ?

```
[cls.name.toUpper()/]([for (a : EAttribute | cls.eAttributes) separator(';')][a.name/][/for])
```

Exercice 5.2 – En-têtes CSV. Écrire un template qui génère, pour chaque classe, un fichier `csv/<Classe>.csv` contenant une seule ligne : les noms des attributs séparés par des virgules.

Exercice 5.3 – Compléter la génération SQL. (a) Compléter `toSQLType` pour traduire `EDouble` en `DECIMAL(10,2)` et `ELong` en `BIGINT`. (b) Ajouter `NOT NULL` aux colonnes de clé étrangère des références obligatoires (`lowerBound = 1`).

Exercice 5.4 – Trouver les erreurs. Le template suivant contient quatre erreurs.

```
[template public gen(root : EPackage)]
[file (root.name + '.txt', false, 'UTF-8')]
[for (c : EClass | root.eClassifiers)]
Classe [c.name]
```

```
[if (c.abstract)] abstraite [/if
[/for]
[/template]
```

Exercice 5.5 – Interface TypeScript. Écrire un template qui génère, pour chaque classe, une interface TypeScript : `EString` → `string`, `EInt`/`EDouble` → `number`, `EBoolean` → `boolean`, `EDate` → `Date` ; un attribut facultatif est suivi de `?` ; une référence multiple devient un tableau `Book[]`.

Exercice 5.6 – Zones protégées. Un développeur a ajouté une méthode `estEnRetard()` dans `Loan.java` généré, juste après le constructeur. Que se passe-t-il à la prochaine génération ? Comment faire pour la conserver ?

5.8 Quiz

Répondez aux questions suivantes. Les corrigés commentés figurent en annexe A.

Q1. Acceleo implémente le standard de l'OMG :

a) QVT
b) MOFM2T
c) XMI
d) BPMN

Q2. Dans un template, le texte qui n'est pas entre crochets :

a) est ignoré
b) est recopié tel quel dans le fichier produit
c) est évalué comme une expression OCL
d) provoque une erreur

Q3. À quoi sert `[comment @main/]` ?

a) À commenter le module
b) À marquer le template point d'entrée
c) À fixer l'encodage
d) À créer une zone protégée

Q4. Que renvoie `root.eClassifiers->filter(EClass)` ?

a) toutes les classes et tous les types
b) seulement les EClass du paquetage
c) les attributs de la racine
d) le nom du paquetage

Q5. Dans `[file ('a.py', false, 'UTF-8')]`, la valeur `false` signifie :

a) ne pas créer le fichier
b) écraser le fichier s'il existe
c) ajouter à la fin du fichier
d) fichier en lecture seule

Q6. Quelle différence entre un template et une query ?

a) Aucune
b) Un template produit du texte, une query calcule une valeur
c) Une query produit un fichier
d) Un template ne peut pas avoir de paramètre

Q7. Le `separator` d'une boucle `[for]` est écrit :

a) après chaque élément
b) avant chaque élément
c) entre les éléments, jamais après le dernier
d) une seule fois à la fin

Q8. Du code écrit à la main dans un fichier généré, hors zone `[protected]` :

a) est conservé
b) est écrasé à la prochaine génération
c) provoque une erreur
d) est déplacé dans la zone protégée

Q9. Que produit l'expression suivante pour la classe `Book` ?

```
[for (r : EReference | cls.eReferences)][r.name/]:[r.eReferenceType.name/] [/for]
```

a) `book:Book`
b) une chaîne vide
c) une erreur
d) `id:EString`

Q10. Écrire l'expression Acceleo qui produit la liste des noms des attributs obligatoires d'une classe `cls`, séparés par des virgules.

Q11. On ajoute `nbPages : EInt` à `Book`. Que faut-il modifier pour que le Python, le SQL et le Java soient à jour ?

Chapitre 6

Transformation par programmation (Java et API EMF)

Résumé

*Transformer un modèle par programmation, c'est écrire **soi-même**, dans un langage généraliste (Java avec l'API **EMF**, ou Python avec **PyEcore**), le programme qui **charge** le modèle, le **parcourt** et **écrit** le résultat : du texte (M2T) ou un autre modèle (M2M). L'approche est **impérative** : on décrit pas à pas les actions. Ce chapitre montre comment créer un projet Java utilisant les bibliothèques EMF, charger un fichier `.ecore` avec un `ResourceSet`, naviguer avec les **getters** de l'API (propriété Ecore `xxx` → `getXxx()`/`isXxx()`), puis écrire, étape par étape, trois générateurs (`Library2Python`, `Library2SQL`, `Library2Java`) produisant les mêmes résultats que les templates Acceleo du chapitre précédent. Des générateurs plus courts (Markdown, Graphviz, validation, M2M) illustrent chaque technique. On compare enfin programmation et template.*

Objectifs d'apprentissage

À l'issue de ce chapitre, vous serez capable de :

- situer la transformation par programmation par rapport à ATL (M2M) et Acceleo (M2T) ;
- créer un projet Java utilisant l'API EMF (JAR `org.eclipse.emf.*`) ;
- charger un `.ecore` et le parcourir : `getEClassifiers()`, `getEAttributes()`, `getEReferences()` ;
- écrire un générateur : boucles, conditions, méthodes utilitaires, écriture de fichiers ;
- générer Python, SQL et Java à partir de `library.ecore` ; programmer des zones protégées ;
- choisir entre programmation et template.

Contenu du chapitre

6.1 Pourquoi programmer une transformation ? 89
6.2 Lire un modèle avec EMF 91
6.2.1 Des concepts Ecore aux interfaces Java 91
6.2.2 Créer le projet Java 91
6.2.3 Charger le modèle 92
6.3 Le générateur pas à pas 93

6.4 Cas complet : Python, SQL et Java 95
6.4.1 Library2SQL 96
6.4.2 Library2Java 97
6.4.3 Zones protégées programmées 98
6.4.4 La même transformation en Python : PyEcore 99
6.4.5 Vers le M2M par programmation 99
6.5 Bonnes pratiques et synthèse 100
6.6 Exemples complémentaires 100
6.7 Exercices 102
6.8 Quiz 103

6.1 Pourquoi programmer une transformation ?

Trois façons de transformer un modèle ont été présentées au chapitre 4 : M2M avec un langage dédié (ATL, QVT), M2T par template (Acceleo) et transformation **par programmation** : un **programme** lit le modèle grâce à une **API** et produit ce qu'il veut, texte **ou** modèle.

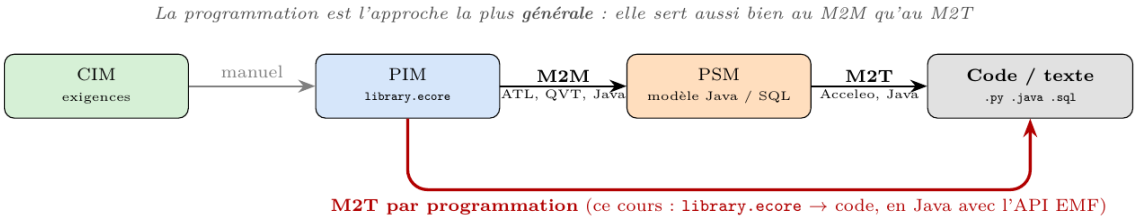


Figure 6.1 – La programmation : une approche générale, utilisable à chaque marche de MDA

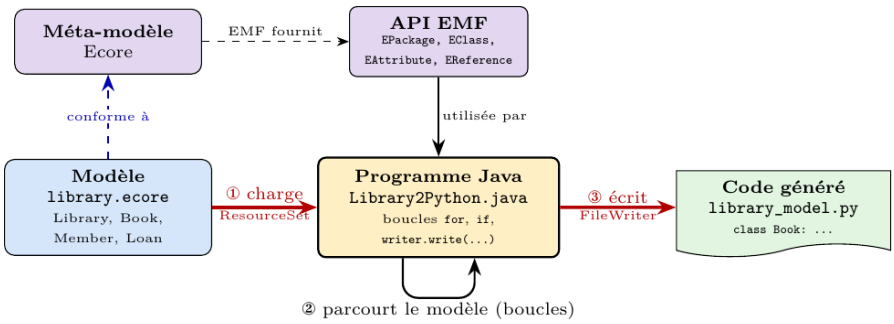


Figure 6.2 – Le principe : charger, parcourir, écrire

1. **Charger** le fichier `.ecore` en mémoire (objets Java) avec un `ResourceSet` ;
2. **Parcourir** le modèle avec des boucles `for` et les **getters** de l'API EMF ;
3. **Écrire** le texte produit dans un fichier (`FileWriter`, `Files.writeString`).

Analogie : le **secrétaire**. Par template, on lui donne une lettre type à trous ; par programmation, on lui **dicte** chaque mot (« pour chaque client, écris *Bonjour*, puis son nom, puis une virgule… »). Dans les deux cas, la liste des destinataires est le modèle.

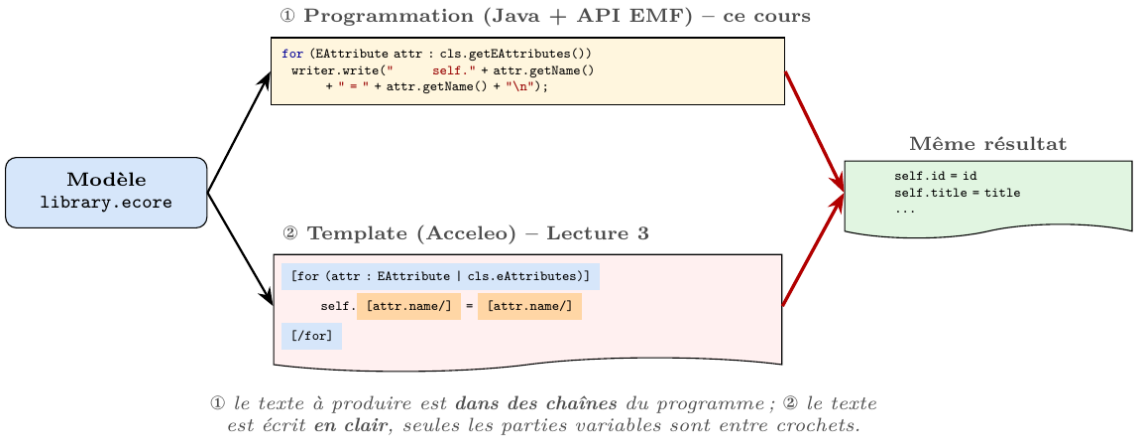


Figure 6.3 – Programmation ou template : deux écritures du même morceau de générateur

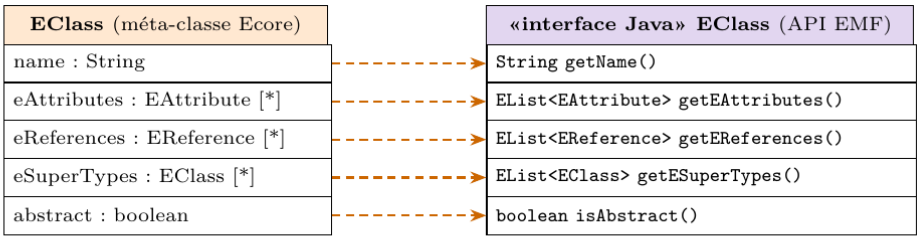
Tableau 6.1 – Template (Acceleo) et programmation (Java + EMF)

	Acceleo (template)	**Programmation (Java + API EMF)**
Style	déclaratif : on décrit le texte	impératif : on décrit les actions
Langage	`.mtl` + expressions OCL	Java (ou Python avec PyEcore)
Texte produit	en clair, avec des trous `[.../]`	dans des chaînes `"..."` + concaténations
Navigation	`cls.eAttributes`	`cls.getEAttributes()`
Zones protégées, multi-fichiers	intégrées (`[protected]`, `[file]`)	à programmer soi-même
Points forts	lisible, rapide à écrire	flexible, débogueur, toute bibliothèque Java

6.2 Lire un modèle avec EMF

6.2.1 Des concepts Ecore aux interfaces Java

EMF fournit une **interface Java pour chaque concept d'Ecore** (`EPackage`, `EClass`, `EAttribute`, `EReference`...). Si l'on sait lire le méta-modèle Ecore (figure 2.1), on connaît déjà l'API : il suffit d'ajouter `get` (ou `is` pour un booléen).



Règle : chaque propriété `xxx` d'un concept Ecore se lit avec `getXxx()` (`isXxx()` pour un booléen). Il en va de même pour `EAttribute` (`getLowerBound()`, `getEAttributeType()`...) et `EReference` (`isMany()`, `isContainment()`...).

Figure 6.4 – Des concepts Ecore aux interfaces et méthodes Java de l'API EMF

Tableau 6.2 – Méthodes de l'API EMF et leur valeur sur `library.ecore`

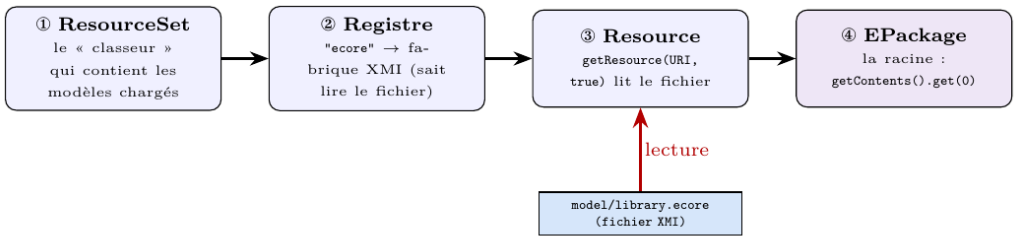
Objet	Méthode	Renvoie	Exemple
`EPackage`	`getName()`	`String`	`"library"`
`EPackage`	`getEClassifiers()`	`EList<EClassifier>`	Library, Book, Member, Loan
`EClass`	`getEAttributes()`	`EList<EAttribute>`	Book : id, title, isbn, author, available
`EClass`	`getEReferences()`	`EList<EReference>`	Loan : book, member
`EAttribute`	`getEAttributeType().getName()`	`String`	`"EString"`, `"EDate"`
`EAttribute`	`getLowerBound()`	`int`	email : 0 ; name : 1
`EReference`	`isMany()`	`boolean`	Library.book : true ; Loan.book : false
`EReference`	`isContainment()`	`boolean`	Library.book : true ; Loan.book : false
`EReference`	`getEReferenceType()`	`EClass`	Loan.book → Book

6.2.2 Créer le projet Java

1. *File* → *New* → *Java Project*, JRE JavaSE-17 ou plus ; décocher *Create module-info.java* ;
2. créer le paquetage `org.example.transform` et la classe `Library2Python` ;
3. créer un dossier `model/` et y copier `library.ecore` ;
4. *Build Path* → *Configure Build Path* → *Libraries* → *Classpath* → *Add External JARs* : depuis `<ECLIPSE_HOME>/plugins/`, ajouter `org.eclipse.emf.common_*.jar`, `org.eclipse.emf.ecore_*.jar` et `org.eclipse.emf.ecore.xmi_*.jar` (ou, avec Maven, les dépendances `org.eclipse.emf.common`, `org.eclipse.emf.ecore`, `org.eclipse.emf.ecore.xmi`).

Structure : `src/` (le programme), `model/` (le modèle d'entrée), `gen/` (le code généré, écrasé à chaque exécution ; faire *Refresh*/F5 pour le voir).

6.2.3 Charger le modèle



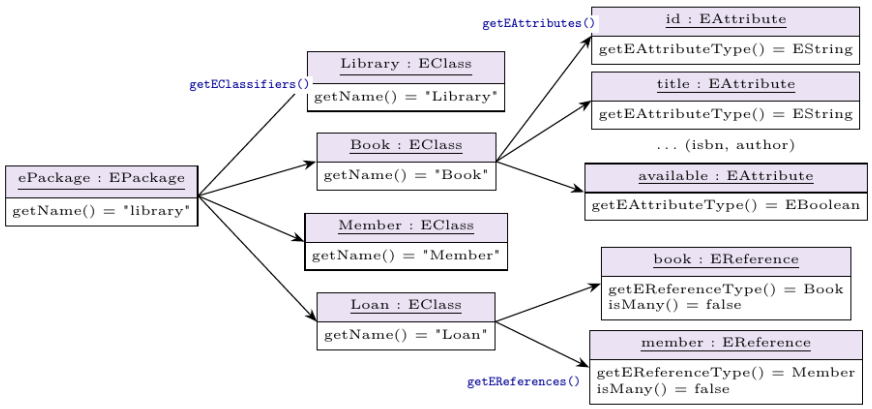
*En dehors d'Eclipse (programme `main`), il faut **enregistrer** la fabrique pour l'extension `.ecore` (étape ②) ; sinon : « Cannot create a resource… ».*

Figure 6.5 – Charger le modèle en 4 étapes : du fichier `.ecore` à l'objet `EPackage`

```
import java.io.IOException;
import org.eclipse.emf.common.util.URI;
import org.eclipse.emf.ecore.*;
import org.eclipse.emf.ecore.resource.*;
import org.eclipse.emf.ecore.resource.impl.ResourceSetImpl;
import org.eclipse.emf.ecore.xmi.impl.XMIResourceFactoryImpl;

public static void main(String[] args) throws IOException {
    // 1. ResourceSet + association de l'extension .ecore à une fabrique XMI
    ResourceSet resourceSet = new ResourceSetImpl();
    resourceSet.getResourceFactoryRegistry()
               .getExtensionToFactoryMap().put("ecore", new XMIResourceFactoryImpl());
    // 2. Charger le modèle
    Resource resource = resourceSet.getResource(URI.createFileURI("model/library.ecore"), true
    );
    // 3. Obtenir le package racine
    EPackage ePackage = (EPackage) resource.getContents().get(0);
    // 4. Lancer la génération
    new Library2Python().generatePython(ePackage, "gen");
}
```

Listing 6.1 – Chargement d'un modèle Ecore



*Après chargement, le programme manipule un **graphe d'objets Java** ; il le parcourt en appelant les **getters** de l'API EMF.*

Figure 6.6 – `library.ecore` chargé : un graphe d'objets Java

Exemple 6.1 – Explorer le modèle avant de générer.

```
for (EClassifier classifier : ePackage.getEClassifiers()) {
    if (classifier instanceof EClass cls) {
        System.out.println("Classe " + cls.getName());
        for (EAttribute attr : cls.getEAttributes())
            System.out.println("  - attribut " + attr.getName() + " : "
                + attr.getEAttributeType().getName()
                + " [" + attr.getLowerBound() + ".." + attr.getUpperBound() + "]");
        for (EReference ref : cls.getEReferences())
            System.out.println("  - référence " + ref.getName() + " -> "
                + ref.getEReferenceType().getName()
                + (ref.isMany() ? " (plusieurs)" : " (une seule)"));
    }
}
```

Console : « Classe Loan / - attribut loanDate : EDate [1..1] / - attribut returnDate : EDate [0..1] / - référence book -> Book (une seule) . . . ». (La borne « * » s'affiche `-1`.) Afficher d'abord le modèle permet de vérifier le chargement et de voir ce que l'on va parcourir. □

Exemple 6.2 – Trois requêtes simples sur le modèle chargé.

```
// (1) les classes qui ont au moins une composition
for (EClassifier c : p.getEClassifiers())
    if (c instanceof EClass cls && cls.getEReferences().stream().anyMatch(EReference::
    isContainment))
        System.out.println(cls.getName());                    // Library
// (2) la classe racine : celle qui n'est contenue par personne
// (3) le nombre total de références multiples
long n = p.getEClassifiers().stream().filter(EClass.class::isInstance).map(EClass.class::cast)
          .flatMap(c -> c.getEReferences().stream()).filter(EReference::isMany).count();   //
    3
```

Pour (2), on cherche la classe qui n'apparaît comme cible d'aucune composition : sur `library.ecore`, c'est `Library` (exercice 6.2). □

6.3 Le générateur pas à pas

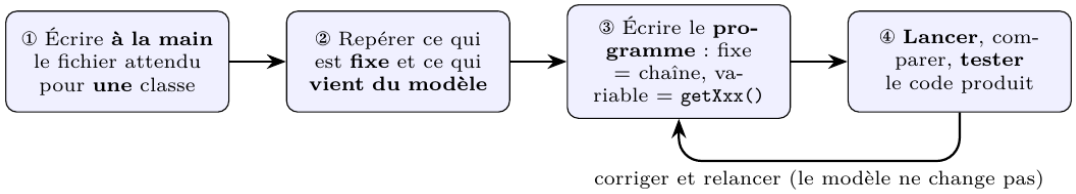


Figure 6.7 – Méthode « l'exemple d'abord » : écrire le fichier visé, repérer le fixe et le variable

Dans le fichier visé, le texte **fixe** (`class`, `def __init__(self, =None...`) devient une **chaîne Java** ; le texte **variable** (nom de la classe, noms des attributs) devient un **appel** `getName()`.

Exemple 6.3 – Étapes 1 et 2 : ouvrir le fichier, une classe Python par EClass.

```
public void generatePython(EPackage ePackage, String outDir) throws IOException {
    Files.createDirectories(Paths.get(outDir));
    String fileName = outDir + "/" + ePackage.getName() + "_model.py";
    try (Writer writer = new FileWriter(fileName, StandardCharsets.UTF_8)) {
        writer.write("# -*- coding: utf-8 -*-\n");
```

```
        for (EClassifier classifier : ePackage.getEClassifiers()) {
            if (classifier instanceof EClass cls) {   // ne garder que les classes
                writer.write("\n\n");
                genClass(cls, writer);
            }
        }
    }                                                 // fermeture automatique
}
```

`try ( ...)` ferme le fichier automatiquement, même en cas d'erreur ; `StandardCharsets.UTF_8` garantit les accents (constructeur disponible depuis Java 11) ; `instanceof EClass cls` filtre et nomme la classe (Java 16+). □

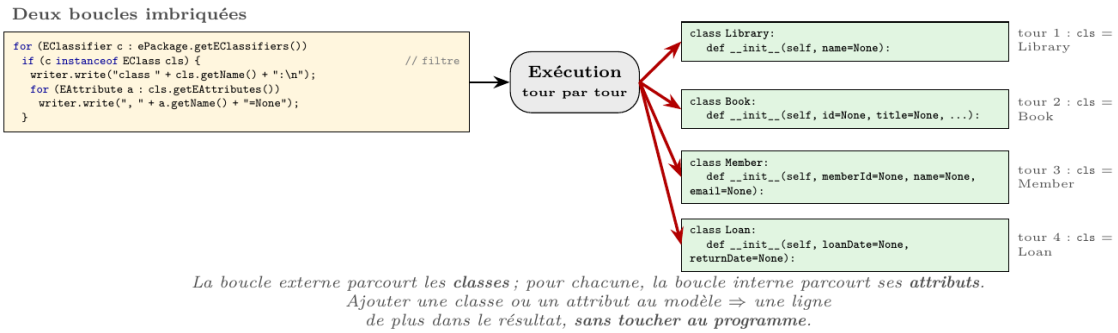


Figure 6.8 – Deux boucles imbriquées : ce qui se passe à l'exécution

Exemple 6.4 – Étapes 3 à 5 : constructeur, références, __repr__.

```
private void genClass(EClass cls, Writer writer) throws IOException {
    writer.write("class " + cls.getName() + ":\n");
    writer.write("    def __init__(self");
    for (EAttribute attr : cls.getEAttributes())
        writer.write(", " + attr.getName() + "=None");
    writer.write("):\n");
    for (EAttribute attr : cls.getEAttributes())
        writer.write("        self." + attr.getName() + " = " + attr.getName() + "\n");
    for (EReference ref : cls.getEReferences())         // [] si multiple, None sinon
        writer.write("        self." + ref.getName() + " = " + (ref.isMany() ? "[]" : "None")
    + "\n");
    if (cls.getEAttributes().isEmpty() && cls.getEReferences().isEmpty())
        writer.write("        pass\n");
    writer.write("\n    def __repr__(self):\n");
    writer.write("        return f\"" + cls.getName() + "(" + reprArgs(cls) + ")\"\n");
}

private String reprArgs(EClass cls) {
    List<String> parts = new ArrayList<>();
    for (EAttribute attr : cls.getEAttributes())
        parts.add(attr.getName() + "={self." + attr.getName() + "!r}");
    return String.join(", ", parts);                    // virgule ENTRE les éléments
}
```

Résultat pour Member :

```
return f"Member(memberId={self.memberId!r}, name={self.name!r}, email={self.email!r})"
```

□

Attention. Une première version écrivait `self.ref = []` pour **toutes** les références. Or `Loan.book` est `0..1` : c'est *un* livre, pas une liste. On teste donc `ref.isMany()`.

Tableau 6.3 – Trois façons d'écrire le texte en Java

Technique	Principe	Quand l'utiliser ?
`writer.write("...")`	écriture directe, morceau par morceau	petits générateurs, un seul fichier
`StringBuilder`	texte construit en mémoire, puis `Files.writeString`	tests, zones protégées
Bloc de texte `"""..."""`	texte en clair avec `%s` rempli par `.formatted(...)`	gros blocs fixes (Java 15+)

Exemple 6.5 – La même méthode d'accès écrite des trois façons.

```
String t = "String", n = "title", N = "Title";
// (1) écriture directe
w.write("    public " + t + " get" + N + "() {\n        return this." + n + ";\n    }\n");
// (2) StringBuilder
sb.append("    public ").append(t).append(" get").append(N).append("() {\n")
  .append("        return this.").append(n).append(";\n    }\n");
// (3) bloc de texte
String m = """
        public %s get%s() {
            return this.%s;
        }
    """.formatted(t, N, n);
```

Les trois produisent `public String getTitle() { return this.title; }` sur trois lignes. Le bloc de texte est le plus proche d'un template. □

Tableau 6.4 – Pièges de l'écriture de texte

Symptôme	Cause	Solution
Tout est sur une seule ligne	oubli de `\n`	terminer chaque ligne par `\n`
Fichier vide ou incomplet	fichier jamais fermé	`try (Writer w = ...) { ...}`
Accents illisibles	encodage par défaut	`StandardCharsets.UTF_8`
Virgule en trop	séparateur mal placé	`String.join(", ", liste)`
Erreur Python sur une classe vide	corps vide	écrire `pass`
`FileNotFoundException`	chemin ou casse du nom	vérifier `model/library.ecore`

6.4 Cas complet : Python, SQL et Java

Le chargement est identique pour les trois générateurs, ainsi que quelques utilitaires : on les factorise.

```
public final class EcoreLoader {
    private EcoreLoader() { }
    public static EPackage load(String path) {
        ResourceSet rs = new ResourceSetImpl();
        rs.getResourceFactoryRegistry().getExtensionToFactoryMap()
```

```
            .put("ecore", new XMIResourceFactoryImpl());
        return (EPackage) rs.getResource(URI.createFileURI(path), true).getContents().get(0);
    }
    public static String upperFirst(String s) {
        return s.isEmpty() ? s : Character.toUpperCase(s.charAt(0)) + s.substring(1);
    }
}
// Utilisation : le chemin peut être passé en argument (model/school.ecore...)
EPackage p = EcoreLoader.load(args.length > 0 ? args[0] : "model/library.ecore");
```

6.4.1 Library2SQL

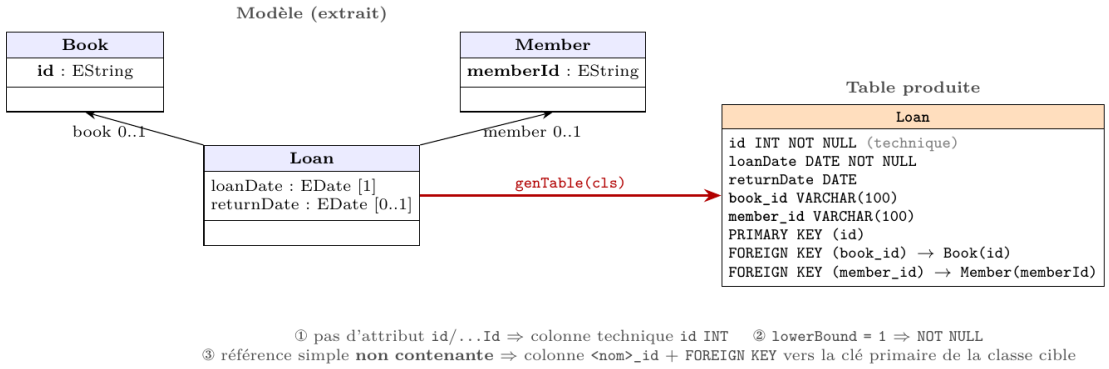


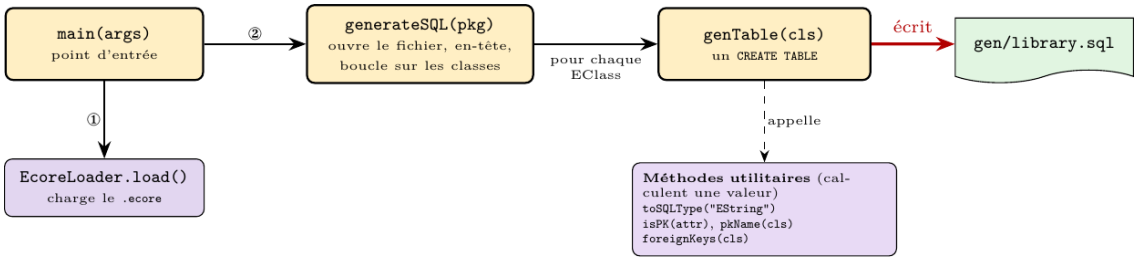
Figure 6.9 – Règles de traduction d'une EClass en table SQL

Exemple 6.6 – La méthode genTable et ses utilitaires.

```
private void genTable(EClass cls, Writer w) throws IOException {
    w.write("CREATE TABLE " + cls.getName() + " (\n");
    if (pkAttribute(cls) == null) w.write("    id INT NOT NULL,\n");
    for (EAttribute attr : cls.getEAttributes()) {
        String notNull = (attr.getLowerBound() > 0) ? " NOT NULL" : "";
        w.write("    " + attr.getName() + " "
              + toSQLType(attr.getEAttributeType().getName()) + notNull + ",\n");
    }
    for (EReference ref : foreignKeys(cls))
        w.write("    " + ref.getName() + "_id " + pkType(ref.getEReferenceType()) + ",\n");
    w.write("    PRIMARY KEY (" + pkName(cls) + ")");
    for (EReference ref : foreignKeys(cls))
        w.write(",\n    FOREIGN KEY (" + ref.getName() + "_id) REFERENCES "
              + ref.getEReferenceType().getName() + "(" + pkName(ref.getEReferenceType()) + ")
    ");
    w.write("\n);\n");
}
static String toSQLType(String t) {
    switch (t) {
        case "EString":  return "VARCHAR(100)";  case "EInt":    return "INT";
        case "EBoolean": return "BOOLEAN";       case "EDate":   return "DATE";
        case "EDouble":  return "DECIMAL(10,2)"; default:        return "VARCHAR(255)";
    }
}
static boolean isPK(EAttribute a) { return a.getName().equals("id") || a.getName().endsWith("
    Id"); }
/** Le premier attribut clé, ou null s'il n'y en a pas (une colonne id INT sera ajoutée). */
static EAttribute pkAttribute(EClass c) {
    for (EAttribute a : c.getEAttributes()) if (isPK(a)) return a;
```

```
    return null;
}
static String pkName(EClass c) { EAttribute a = pkAttribute(c); return a == null ? "id" : a.
    getName(); }
static String pkType(EClass c) {
    EAttribute a = pkAttribute(c);
    return a == null ? "INT" : toSQLType(a.getEAttributeType().getName());
}
/** Références simples et non contenantes = clés étrangères. */
static List<EReference> foreignKeys(EClass cls) {
    List<EReference> res = new ArrayList<>();
    for (EReference r : cls.getEReferences())
        if (!r.isContainment() && !r.isMany()) res.add(r);
    return res;
}
```

`Library.book` (composition multiple) ne produit aucune colonne ; `Loan.book` (simple, non contenante) devient la clé étrangère `book_id` de type `VARCHAR(100)`, comme la clé `id` de `Book`. Le résultat est identique à celui d'Acceleo (exemple 5.11). □



*Comme en Acceleo (templates + queries) : une méthode **par concept** à produire, et des méthodes **utilitaires** qui ne produisent pas de texte mais calculent une valeur.*

Figure 6.10 – Organisation d'un générateur : une méthode par concept + des méthodes utilitaires

6.4.2 Library2Java

Exemple 6.7 – Un fichier par classe, texte construit dans un `StringBuilder`.

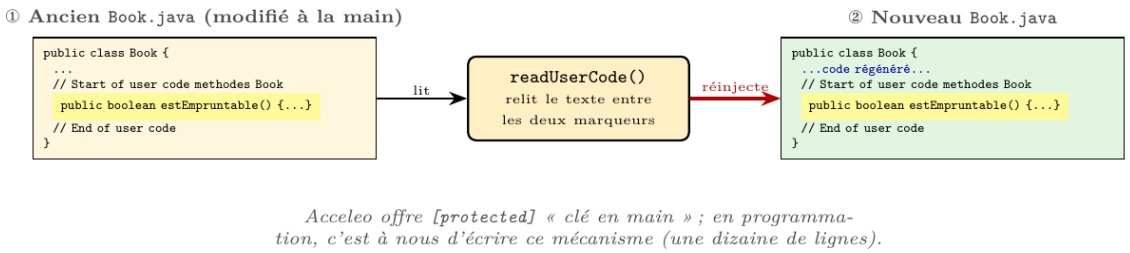
```
public void generateJava(EPackage ePackage, String outDir) throws IOException {
    Path dir = Paths.get(outDir, ePackage.getName());          // gen/library/
    Files.createDirectories(dir);
    for (EClassifier c : ePackage.getEClassifiers())
        if (c instanceof EClass cls) {
            Path file = dir.resolve(cls.getName() + ".java");  // un fichier par classe
            String userCode = readUserCode(file, cls.getName());
            Files.writeString(file, genClass(cls, ePackage, userCode), StandardCharsets.UTF_8)
    ;
        }
}

String genClass(EClass cls, EPackage p, String userCode) {
    StringBuilder sb = new StringBuilder();
    sb.append("package ").append(p.getName()).append(";\n\n");
    sb.append("import java.util.ArrayList;\nimport java.util.Date;\nimport java.util.List;\n\n
    ");
    sb.append("public class ").append(cls.getName()).append(" {\n\n");
    for (EAttribute a : cls.getEAttributes())
```

```
        sb.append("    private ").append(attrType(a)).append(" ").append(a.getName()).append("
    ;\n");
    for (EReference r : cls.getEReferences()) {
        sb.append("    private ").append(refType(r)).append(" ").append(r.getName());
        if (r.isMany()) sb.append(" = new ArrayList<>()");
        sb.append(";\n");
    }
    for (EAttribute a : cls.getEAttributes()) {            // accesseurs
        String n = a.getName();
        sb.append("\n    public ").append(attrType(a)).append(" ").append(getterName(a))
          .append("() {\n        return this.").append(n).append(";\n    }\n");
        sb.append("\n    public void set").append(upperFirst(n)).append("(").append(attrType(a
    ))
          .append(" ").append(n).append(") {\n        this.").append(n).append(" = ").append(n
    )
          .append(";\n    }\n");
    }
    sb.append("\n    // Start of user code methodes ").append(cls.getName()).append("\n")
      .append(userCode).append("    // End of user code\n}\n");
    return sb.toString();
}
static String attrType(EAttribute a) {
    switch (a.getEAttributeType().getName()) {
        case "EString": return "String";   case "EInt":    return "int";
        case "EBoolean": return "boolean"; case "EDouble": return "double";
        case "EDate":   return "Date";     default:        return "Object";
    }
}
static String refType(EReference r) {
    String t = r.getEReferenceType().getName();
    return r.isMany() ? "List<" + t + ">" : t;
}
static String getterName(EAttribute a) {
    return (a.getEAttributeType().getName().equals("EBoolean") ? "is" : "get") + upperFirst(a.
    getName());
}
```

(`upperFirst` est la méthode utilitaire d'`EcoreLoader`, importée en `static`.) Pour `Book`, le getter booléen s'appelle `isAvailable()`, comme dans l'API générée par EMF. □

6.4.3 Zones protégées programmées



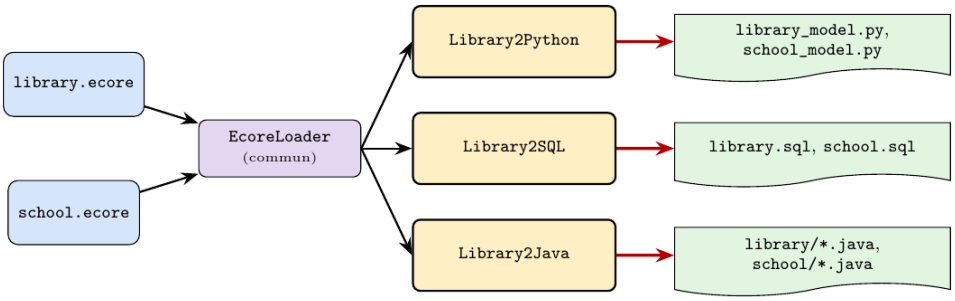
Acceleo offre `[protected]` « clé en main » ; en programmation, c'est à nous d'écrire ce mécanisme (une dizaine de lignes).

Figure 6.11 – Zones protégées : relire l'ancien fichier pour conserver le code écrit à la main

```
static String readUserCode(Path file, String className) throws IOException {
    String defaultCode = "    // Ajoutez ici votre code : il sera conservé\n";
    if (!Files.exists(file)) return defaultCode;               // 1re génération
    String old = Files.readString(file, StandardCharsets.UTF_8);
    String begin = "// Start of user code methodes " + className + "\n";
```

```
    String end   = "    // End of user code";
    int i = old.indexOf(begin);
    int j = (i < 0) ? -1 : old.indexOf(end, i + begin.length()); // fin cherchée APRÈS le dé
    but
    return (i >= 0 && j > i) ? old.substring(i + begin.length(), j) : defaultCode;
}
```

Listing 6.2 – Relire le code utilisateur d'une zone protégée



*Les générateurs utilisent les concepts d'**Ecore**, pas ceux de la bibliothèque : 3 générateurs × 2 modèles = 6 résultats, **sans modifier une ligne**.*

Figure 6.12 – Un modèle, trois cibles… et plusieurs modèles

6.4.4 La même transformation en Python : PyEcore

```
from pyecore.resources import ResourceSet, URI
from pyecore.ecore import EClass

rset = ResourceSet()
pkg = rset.get_resource(URI("model/library.ecore")).contents[0]    # EPackage
lines = ["# -*- coding: utf-8 -*-"]
for cls in pkg.eClassifiers:
    if not isinstance(cls, EClass):
        continue
    params = "".join(f", {a.name}=None" for a in cls.eAttributes)
    lines += ["", "", f"class {cls.name}:", f"    def __init__(self{params}):"]
    lines += [f"        self.{a.name} = {a.name}" for a in cls.eAttributes]
    lines += [f"        self.{r.name} = {'[]' if r.many else 'None'}" for r in cls.eReferences
    ]
with open(f"gen/{pkg.name}_model.py", "w", encoding="utf-8") as f:
    f.write("\n".join(lines) + "\n")
```

Mêmes concepts (`eClassifiers`, `eAttributes`, `many`), sans les `get` : on retrouve les noms du méta-modèle.

6.4.5 Vers le M2M par programmation

Le même programme peut **créer un modèle** au lieu d'un texte. On crée un **nouveau** paquetage cible plutôt que de modifier la source :

```
EcoreFactory ef = EcoreFactory.eINSTANCE;
EPackage cible = ef.createEPackage();                    // créer la racine du modèle cible
cible.setName("library_rel"); cible.setNsURI("http://ensias.ma/library_rel"); cible.
    setNsPrefix("rel");
for (EClassifier c : ePackage.getEClassifiers())
    if (c instanceof EClass cls) {
```

```
        EClass t = ef.createEClass();                    // créer un élément
        t.setName("Table_" + cls.getName());             // le remplir
        cible.getEClassifiers().add(t);                  // le ranger dans son conteneur
    }
Resource r = resourceSet.createResource(URI.createFileURI("gen/library_rel.ecore"));
r.getContents().add(cible);
r.save(null);                                           // enregistrer
```

C'est ce que fait ATL… mais écrit à la main, sans résolution automatique des liens (il faut gérer soi-même une `Map` de trace).

Attention. Ajouter à une nouvelle ressource un objet déjà contenu ailleurs (`r.getContents().add(ePackage)`) le **déplace** : il disparaît de la ressource source. Pour réutiliser un élément de la source dans la cible, on le copie avec `EcoreUtil.copy(...)`.

6.5 Bonnes pratiques et synthèse

Tableau 6.5 – Erreurs fréquentes avec l'API EMF

Message	Cause	Solution
`The import org.eclipse cannot be resolved`	JAR EMF absents du Classpath	ajouter common, ecore, ecore.xmi
`Cannot create a resource for 'file:...'`	fabrique non enregistrée pour `ecore`	enregistrer `XMIResourceFactoryImpl` pour l'extension `ecore`
`ClassCastException ... EPackage`	le fichier chargé n'est pas un `.ecore`	charger le bon fichier
`must be declared in a module`	`module-info.java` présent	le supprimer
Rien dans `gen/`	vue non rafraîchie	*Refresh* (F5)

À retenir.

- Une méthode par concept à produire (`genClass`, `genTable`) et des méthodes utilitaires (`toSQLType`, `isPK`) ; un seul code de chargement (`EcoreLoader`).
- Jamais de nom du modèle « en dur » (`"Book"`) : toujours `cls.getName()`.
- Toujours `try (...)`, UTF-8 et un dossier de sortie dédié ; tester le **code produit** sur au moins deux modèles.
- **Template** pour un texte long et régulier ; **programmation** pour une logique complexe (calculs, tris, graphes) ou une intégration dans une application. On combine souvent les deux.

6.6 Exemples complémentaires

Exemple 6.8 – Un générateur de documentation Markdown.

```
public String genMarkdown(EPackage p) {
    StringBuilder sb = new StringBuilder("# Modèle " + p.getName() + "\n\n");
```

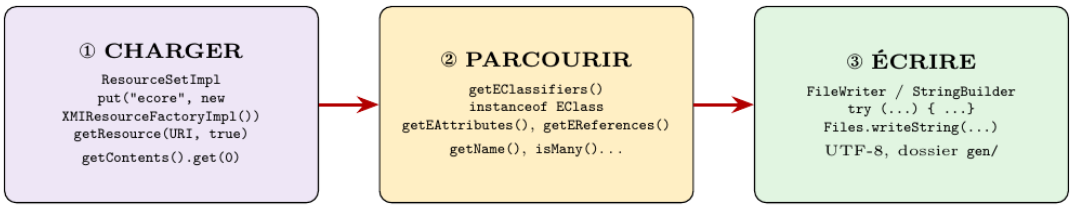


Figure 6.13 – Synthèse : la transformation par programmation

```
    for (EClassifier c : p.getEClassifiers()) {
        if (!(c instanceof EClass cls)) continue;
        sb.append("## ").append(cls.getName()).append("\n\n");
        sb.append("| Propriété | Type | Multiplicité |\n|---|---|---|\n");
        for (EStructuralFeature f : cls.getEStructuralFeatures())
            sb.append("| ").append(f.getName()).append(" | ").append(f.getEType().getName())
              .append(" | ").append(f.getLowerBound()).append("..")
              .append(f.getUpperBound() == -1 ? "*" : String.valueOf(f.getUpperBound())).
    append(" |\n");
        sb.append("\n");
    }
    return sb.toString();
}
```

`getEStructuralFeatures()` renvoie à la fois les attributs et les références ; `getEType()` est le type commun (`EDataType` ou `EClass`). Pour `Loan`, le tableau contient `loanDate | EDate | 1..1`, `returnDate | EDate | 0..1`, `book | Book | 0..1`, `member | Member | 0..1`. □

Exemple 6.9 – Un générateur Graphviz (DOT).

```
StringBuilder sb = new StringBuilder("digraph " + p.getName() + " {\n  node [shape=box];\n");
for (EClassifier c : p.getEClassifiers())
    if (c instanceof EClass cls)
        for (EReference r : cls.getEReferences())
            sb.append("  ").append(cls.getName()).append(" -> ").append(r.getEReferenceType().
    getName())
              .append(" [label=\"").append(r.getName()).append("\"")
              .append(r.isContainment() ? ", arrowtail=diamond, dir=both" : "").append("];\n")
    ;
sb.append("}\n");
```

Résultat pour `library.ecore` : `Library -> Book [label="book", arrowtail=diamond, dir=both]; ... Loan -> Member [label="member"];`. La commande `dot -Tpng` dessine le diagramme. □

Exemple 6.10 – Un validateur de conventions de nommage.

```
List<String> erreurs = new ArrayList<>();
for (EClassifier c : p.getEClassifiers()) {
    if (!Character.isUpperCase(c.getName().charAt(0)))
        erreurs.add("La classe " + c.getName() + " doit commencer par une majuscule");
    if (c instanceof EClass cls)
        for (EAttribute a : cls.getEAttributes())
            if (!Character.isLowerCase(a.getName().charAt(0)))
                erreurs.add(cls.getName() + "." + a.getName() + " doit commencer par une
    minuscule");
}
erreurs.forEach(System.out::println);
```

La programmation se prête bien à ce type de **vérification** qui n'écrit pas de code mais un rapport. Sur `library.ecore`, la liste est vide. □

Exemple 6.11 – Lire un modèle d'instances (`.xmi`) : rapport des emprunts. Le programme peut aussi parcourir un modèle M1 conforme à `library.ecore`, avec l'API générée :

```
LibraryPackage.eINSTANCE.eClass();                          // enregistre le méta-modèle
rs.getResourceFactoryRegistry().getExtensionToFactoryMap().put("xmi", new
    XMIResourceFactoryImpl());
Library lib = (Library) rs.getResource(URI.createFileURI("model/ensias.xmi"), true).
    getContents().get(0);
for (Loan l : lib.getLoan())
    System.out.println(l.getMember().getName() + " a emprunté « " + l.getBook().getTitle() + "
     »"
        + (l.getReturnDate() == null ? " (en cours)" : ""));
```

Sur le modèle de l'exemple 2.2 : « Sara a emprunté « Le Petit Prince » (en cours) ». On transforme ici des **données** (M1 pour EMF), et non plus le méta-modèle. □

6.7 Exercices

Les corrigés figurent en annexe A.

Exercice 6.1 – Prédire la sortie. Qu'affiche ce programme sur `library.ecore` ?

```
for (EClassifier c : p.getEClassifiers())
    if (c instanceof EClass cls && !cls.getEReferences().isEmpty())
        System.out.println(cls.getName() + " : " + cls.getEReferences().size());
```

Exercice 6.2 – Compter et chercher. (a) Écrire une méthode `int nbAttributsObligatoires(EPackage p)` qui renvoie le nombre total d'attributs obligatoires (`lowerBound > 0`) du paquetage. Quelle valeur obtient-on sur `library.ecore` (tous les attributs sont obligatoires sauf `email`, `returnDate` et `available`) ? (b) Écrire une méthode qui renvoie la classe racine (celle qui n'est la cible d'aucune composition).

Exercice 6.3 – Corriger un générateur. Trouver les trois défauts de ce code :

```
Writer w = new FileWriter("gen/model.py");
for (EClassifier c : p.getEClassifiers()) {
    EClass cls = (EClass) c;
    w.write("class " + cls.getName() + ":");
    for (EAttribute a : cls.getEAttributes()) w.write(a.getName() + ", ");
}
```

Exercice 6.4 – Générer du JSON Schema. Écrire une méthode qui produit, pour une `EClass`, un objet JSON Schema : `"type": "object"`, une entrée `"properties"` par attribut (`EString` → `"string"`, `EInt` → `"integer"`, `EBoolean` → `"boolean"`) et la liste `"required"` des attributs obligatoires.

Exercice 6.5 – M2M par programmation. Écrire un programme qui lit `library.ecore` et crée un **nouveau** `EPackage` `library_dto` contenant, pour chaque classe `C`, une classe `CDto` avec les mêmes attributs (sans références), puis l'enregistre dans `gen/library_dto.ecore`.

Exercice 6.6 – Choisir. Pour chacun des besoins suivants, choisir entre template et programmation et justifier : (a) un fichier `.java` par classe ; (b) un rapport des cycles dans les références ; (c) une page HTML par classe ; (d) un outil en ligne de commande qui génère du SQL pour n'importe quel `.ecore` passé en argument.

6.8 Quiz

Répondez aux questions suivantes. Les corrigés commentés figurent en annexe A.

Q1. L'approche par programmation est :

a) déclarative

b) impérative

c) bidirectionnelle

d) réservée au M2M

Q2. Quels JAR faut-il ajouter au Classpath pour charger un `.ecore` ?

a) emf.common, emf.ecore, emf.ecore.xmi

b) acceleo.engine

c) atl.core

d) ocl.ecore uniquement

Q3. Quel objet sert à charger un modèle EMF ?

a) `FileReader`

b) `ResourceSet`

c) `EcoreFactory`

d) `StringBuilder`

Q4. La propriété Ecore `containment` d'une `EReference` se lit en Java par :

a) `getContainment()`

b) `isContainment()`

c) `containment()`

d) `hasContainment()`

Q5. Comment ne garder que les classes dans `getEClassifiers()` ?

a) `filter(EClass)`

b) `if (c instanceof EClass cls)`

c) `select(EClass)`

d) c'est inutile

Q6. Pour éviter une virgule finale dans une liste de paramètres, on utilise :

a) `String.join(", ", liste)`
b) `writer.flush()`
c) `\n`
d) `try-with-resources`

Q7. Pourquoi utiliser `try (Writer w = ...)` ?

a) Pour accélérer l'écriture
b) Pour fermer automatiquement le fichier, même en cas d'erreur
c) Pour écrire en UTF-8
d) Pour créer le dossier

Q8. Dans `Library2SQL`, quelles références deviennent des clés étrangères ?

a) toutes
b) les compositions
c) les références simples et non contenantes
d) les références multiples

Q9. Quel est l'équivalent Python (PyEcore) de `ref.isMany()` ?

a) `ref.is_many()`
b) `ref.many`
c) `ref.getMany()`
d) `len(ref) > 1`

Q10. On ajoute une classe `Author` au modèle. Que faut-il modifier dans `Library2Python` ?

Q11. Citer un cas où la programmation est préférable à un template, et un cas inverse.

Transformation M2M avec ATL

Résumé

*Une transformation **modèle vers modèle** (M2M) produit, à partir d'un modèle source conforme à un méta-modèle source, un modèle cible conforme à un méta-modèle cible, en appliquant des **règles de correspondance**. C'est typiquement le passage du PIM au PSM. **ATL** (*ATLAS Transformation Language*) est un langage **hybride**, surtout **déclaratif**, dont les expressions sont écrites en **OCL**. Un module ATL comporte un en-tête (`module`, `create OUT : ...` `from IN : ...`), des **helpers** (fonctions OCL attachées à un type) et des **règles** (`from` + filtre, `to` + liaisons `<-`). Le moteur parcourt le modèle, crée les éléments cibles et les **relie automatiquement** grâce à la **trace**. Le chapitre déroule **Families2Persons**, de la création des méta-modèles à l'exécution, puis **Class2Relational**, des règles à plusieurs éléments cibles, une transformation endogène et les autres sortes de règles (*lazy, called*).*

Objectifs d'apprentissage

À l'issue de ce chapitre, vous serez capable de :

- définir une transformation M2M et la situer dans MDA ;
- identifier les éléments d'une transformation : méta-modèles et modèles source/cible, module ATL ;
- lire et écrire un module ATL : en-tête, helpers, règles (`from`, filtre, `to`, `<-`) ;
- expliquer l'exécution d'une transformation et la **résolution automatique** par la trace ;
- réaliser dans Eclipse la transformation **Families2Persons** ;
- choisir entre ATL, Acceleo et la programmation.

Contenu du chapitre

7.1 Pourquoi une transformation M2M ? 106
7.2 Le langage ATL 107
7.2.1 ATL en bref 107
7.2.2 Le problème Families2Persons 107
7.2.3 Structure d'un module 108
7.2.4 Les expressions OCL dans ATL 108
7.3 Règles et helpers 108
7.3.1 Anatomie d'une règle (*matched rule*) 109

7.3.2 Les helpers . . . 110
7.3.3 L'exécution d'une transformation . . . 111
7.4 Cas complet : Families2Persons dans Eclipse . . . 112
7.5 Aller plus loin . . . 113
7.5.1 Class2Relational et la résolution automatique . . . 113
7.5.2 Relier les personnes : le foyer . . . 115
7.5.3 Une règle, plusieurs éléments cibles . . . 115
7.5.4 Les autres sortes de règles . . . 116
7.6 Exemples complémentaires . . . 118
7.7 Exercices . . . 119
7.8 Quiz . . . 119

7.1 Pourquoi une transformation M2M ?

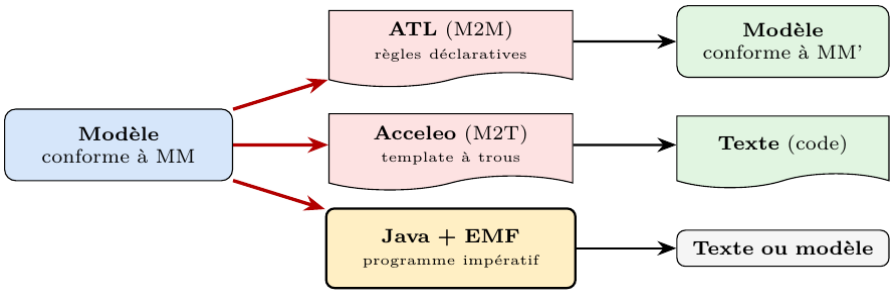


Figure 7.1 – Trois façons de transformer un modèle : ATL produit un modèle, Acceleo et Java surtout du texte

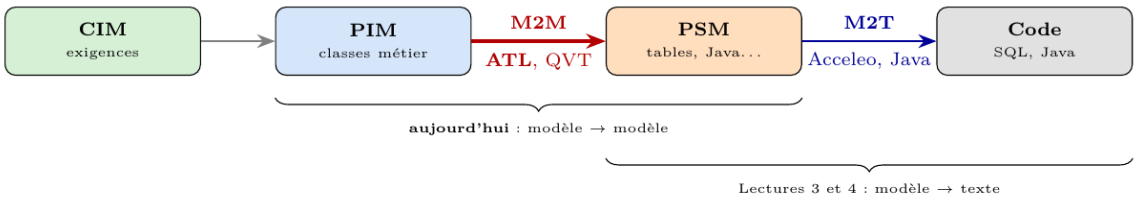


Figure 7.2 – Place de la transformation M2M dans MDA : du PIM au PSM

Définition 7.1 (Transformation de modèle (M2M)). *Programme qui, à partir d'un* ***modèle source*** *conforme à un* ***méta-modèle source****, produit automatiquement un* ***modèle cible*** *conforme à un* ***méta-modèle cible****, en appliquant des* ***règles de correspondance****. Les règles sont écrites* ***au niveau des méta-modèles*** *(M2) : elles valent pour* ***tous*** *les modèles (M1).*

Exemple 7.1 – Quatre transformations M2M.

- **Familles → Personnes** : chaque membre d'une famille devient un homme ou une femme.
- **Classes → tables relationnelles** : chaque classe devient une table, chaque attribut une colonne.

- **Machine à états → table de transitions** : chaque transition devient une ligne (source, événement, cible) – utile pour générer un contrôleur.
- **PIM `Library` → PSM Java EE** : chaque classe devient une entité JPA.

□

Pourquoi un langage dédié ? Une transformation ATL est **plus courte** (on décrit ce qui correspond à quoi, pas comment parcourir le modèle), **plus sûre** (le moteur vérifie les types et relie automatiquement les éléments créés) et elle est **elle-même un modèle** (conforme au méta-modèle d'ATL) : on peut l'analyser, la transformer, la réutiliser. ATL écrit des transformations **unidirectionnelles** : le modèle source est **lu seulement**, le modèle cible est **écrit seulement** (sauf en mode `refining`).

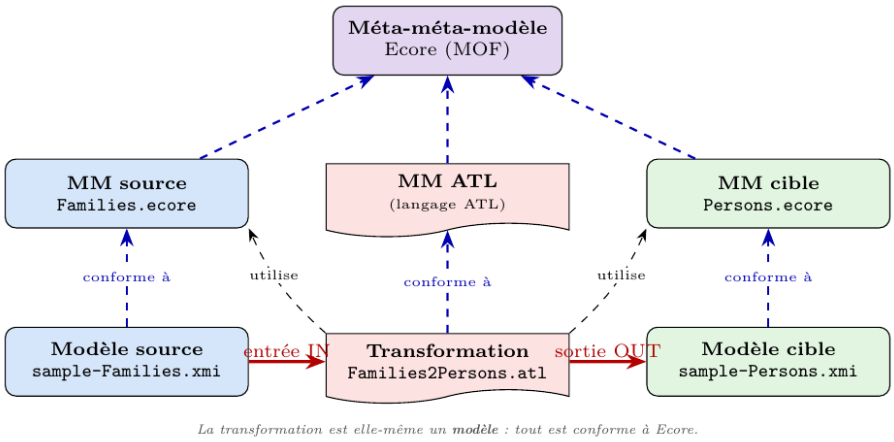


Figure 7.3 – Schéma général d'une transformation ATL : six fichiers, trois niveaux

7.2 Le langage ATL

7.2.1 ATL en bref

ATL a été créé par l'équipe **ATLAS** (INRIA / Université de Nantes) ; c'est aujourd'hui un projet **Eclipse**. Langage hybride (règles déclaratives + partie impérative), expressions en **OCL**, éditeur Eclipse avec coloration et complétion. Le fichier source `.atl` est compilé en `.asm`, exécuté par la machine virtuelle ATL.

7.2.2 Le problème Families2Persons

- **Source** : une liste de familles. Chaque **famille** a un nom (`lastName`) ; elle comprend un **père**, une **mère**, des **fils** et des **filles**. Chaque **membre** a un prénom (`firstName`).
- **Cible** : une liste de personnes. Chaque membre devient **une personne**, avec un nom complet (`fullName` = prénom + nom) ; elle est soit `Male`, soit `Female`.

Exemple : dans la famille March, le père Jim devient l'homme « Jim March » et la fille Brenda la femme « Brenda March ».

`father`, `mother`, `sons`, `daughters` sont des **compositions** ; chacune a une référence **opposée** dans `Member` (`familyFather`, `familyMother`, `familySon`, `familyDaughter`) qui indique le **rôle**

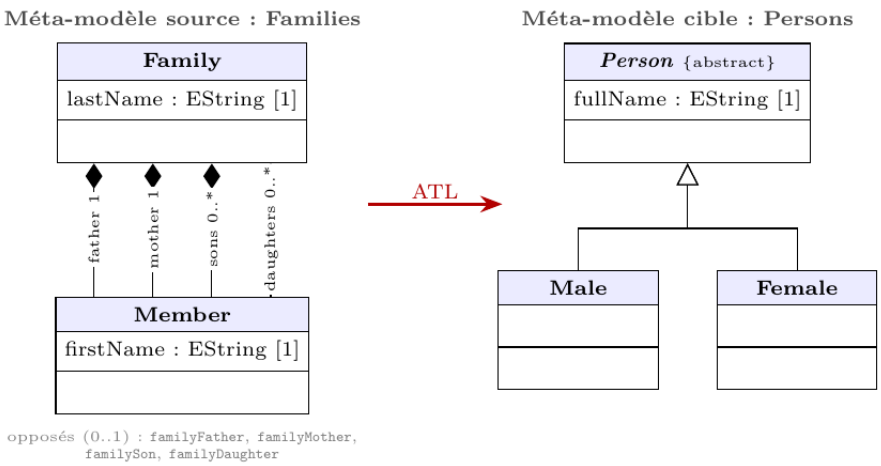


Figure 7.4 – Les méta-modèles Families (source) et Persons (cible)

du membre : pour un membre donné, exactement une de ces quatre références est renseignée. `Person` est **abstraite** : on ne crée que des `Male` ou des `Female`.

7.2.3 Structure d'un module

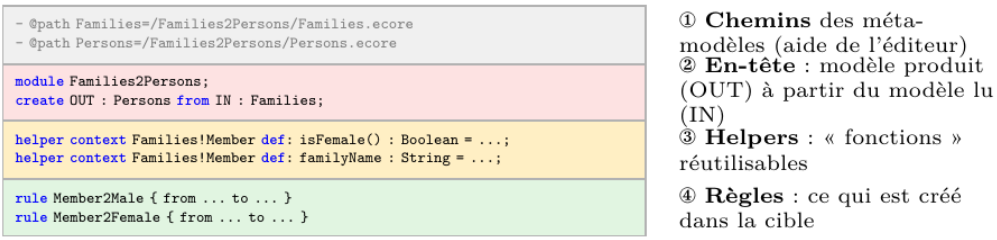


Figure 7.5 – Structure d'un module ATL : quatre parties, dans cet ordre

```
-- @path Families=/Families2Persons/Families.ecore
-- @path Persons=/Families2Persons/Persons.ecore

module Families2Persons;                    -- nom du module = nom du fichier
create OUT : Persons from IN : Families;    -- OUT conforme à Persons, IN à Families
```

Listing 7.1 – En-tête du module Families2Persons

- `IN` et `OUT` sont des **noms de modèles** ; `Families` et `Persons` des **noms de méta-modèles**.
- Une classe d'un méta-modèle se désigne par `Méta-modèle!Classe` : `Families!Member`, `Persons!Female`.
- Les commentaires spéciaux `-- @path` permettent à l'éditeur de trouver les méta-modèles (complétion, vérification).

7.2.4 Les expressions OCL dans ATL

7.3 Règles et helpers

Tableau 7.1 – Expressions ATL / OCL courantes

Besoin	Écriture	Remarque
lire une propriété	`s.firstName`, `s.familyFather`	navigation pointée
« pas de valeur »	`s.familyMother.oclIsUndefined()`	« est null »
concaténer	`s.firstName + ' ' + s.familyName`	apostrophes
condition	`if c then a else b endif`	`else` obligatoire
taille, filtre	`f.sons->size()`, `->select(a | a.isKey)`	flèche sur collection
transformer	`f.sons->collect(m | m.firstName)`	résultat : collection
littéral d'énumération	`#public`, `#STRING`	notation ATL
toutes les instances	`Families!Member.allInstances()`	modèle source

Tableau 7.2 – Navigation sur la famille March (m1 = Jim, père ; m4 = Brenda, fille)

Expression	Résultat
`m1.firstName`	`'Jim'`
`m1.familyFather`	la famille `f1`
`m1.familyMother.oclIsUndefined()`	`true`
`m4.familyDaughter.lastName`	`'March'`
`f1.sons->size() + f1.daughters->size()`	2
`f1.sons->collect(m | m.firstName)`	`Sequence{'Brandon'}`
`Families!Family.allInstances()->size()`	2 (March et Sailor)

7.3.1 Anatomie d'une règle (*matched rule*)

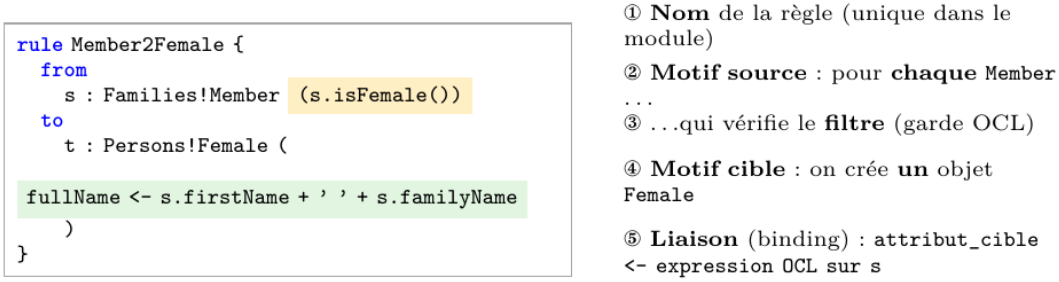


Figure 7.6 – Anatomie d'une règle ATL : motif source, filtre, motif cible, liaisons

Lecture à voix haute : « **Pour chaque** `Member` **qui est une femme**, **créer** une `Female` **dont** le `fullName` vaut prénom + espace + nom de famille. »

Le motif source (`from`). `s : Families!Member (not s.isFemale())` : la variable `s` désigne tour à tour chaque élément de type `Member` ; entre parenthèses, le **filtre** (garde), expression OCL booléenne. Sans filtre, la règle s'applique à tous les `Member`.

Le motif cible (`to`). `t : Persons!Female (fullName <- ...)` : `t` est l'élément **créé** (type **concret**) ; chaque **liaison** `propriété <- expression` donne une valeur à une propriété de `t` ; les liaisons sont séparées par des virgules. Une règle peut créer plusieurs éléments : `to t1 : ...(...), t2 : ...(...)`.

Attention. Un élément source ne doit être pris que par **une seule** règle : les filtres de `Member2Male` et `Member2Female` sont **complémentaires**. Un élément qui ne correspond à aucune règle n'est **pas transformé** (c'est le cas de `Family`).

```
rule Member2Male {
    from s : Families!Member (not s.isFemale())
    to   t : Persons!Male ( fullName <- s.firstName + ' ' + s.familyName )
}
rule Member2Female {
    from s : Families!Member (s.isFemale())
    to   t : Persons!Female ( fullName <- s.firstName + ' ' + s.familyName )
}
```

Listing 7.2 – Les deux règles de Families2Persons

Exemple 7.2 – Trois autres règles avec filtre.

```
-- Boutique : seuls les clients ayant commandé deviennent des contacts commerciaux
rule Client2Contact {
    from c : Shop!Client (c.commandes->notEmpty())
    to   k : CRM!Contact ( nom <- c.nom, email <- c.email )
}
-- Machine à états : chaque transition devient une ligne de table
rule Transition2Row {
    from tr : SM!Transition
    to   r  : Table!Row ( source <- tr.source.name, event <- tr.event, target <- tr.target.
    name )
}
-- Bibliothèque : seuls les livres disponibles figurent dans le catalogue en ligne
rule Book2Item {
    from b : Library!Book (b.available)
    to   i : Catalog!Item ( label <- b.title + ' (' + b.author + ')' )
}
```

Dans chaque cas, les éléments qui ne passent pas le filtre (clients sans commande, livres empruntés) sont simplement ignorés. □

7.3.2 Les helpers

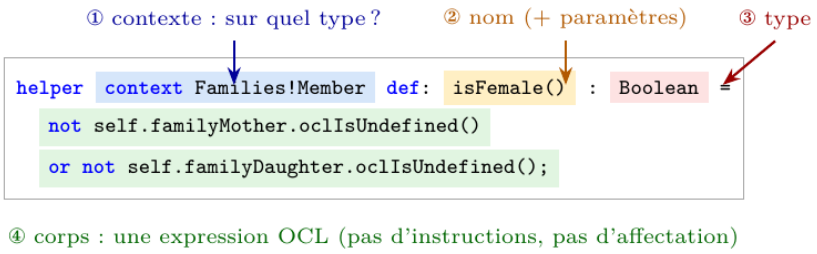


Figure 7.7 – Un helper : une fonction OCL attachée à un type du méta-modèle

Le **contexte** indique sur quel type on appelle le helper ; dans le corps, `self` désigne cet objet. Un helper ne modifie rien : il **calcule** une valeur.

```
helper context Families!Member def: isFemale() : Boolean =
    not self.familyMother.oclIsUndefined()
    or not self.familyDaughter.oclIsUndefined();

helper context Families!Member def: familyName : String =
    if not self.familyFather.oclIsUndefined() then self.familyFather.lastName
    else if not self.familyMother.oclIsUndefined() then self.familyMother.lastName
```

```
    else if not self.familySon.oclIsUndefined() then self.familySon.lastName
    else self.familyDaughter.lastName
    endif endif endif;
```

Listing 7.3 – Les helpers de Families2Persons

Tableau 7.3 – Helper opération et helper attribut

	Helper opération	**Helper attribut**
Déclaration	`def: isFemale() : Boolean = ...`	`def: familyName : String = ...`
Paramètres	possibles	aucun
Appel	`s.isFemale()`	`s.familyName`
Calcul	à chaque appel	une seule fois par objet (cache)
Sans contexte	`thisModule.nom()`	`thisModule.nom`

Exemple 7.3 – Quatre helpers courants.

```
-- (1) booléen : le membre est-il un parent ?
helper context Families!Member def: isParent : Boolean =
    not self.familyFather.oclIsUndefined() or not self.familyMother.oclIsUndefined();
-- (2) calcul sur une collection : nombre d'enfants
helper context Families!Family def: nbChildren : Integer =
    self.sons->size() + self.daughters->size();
-- (3) helper sur un type de base : traduction d'un type en SQL (le PDM)
helper context String def: toSQL() : String =
    if self = 'String' then 'VARCHAR(100)' else if self = 'Integer' then 'INT'
    else 'VARCHAR(255)' endif endif;
-- (4) helper global, sans contexte : appel thisModule.nbFamilles
helper def: nbFamilles : Integer = Families!Family.allInstances()->size();
```

Valeurs : Brandon `isParent = false` ; Sailor `nbChildren = 3` ; `'Integer'.toSQL() = 'INT'` ; `thisModule.nbFamilles = 2`. □

7.3.3 L'exécution d'une transformation

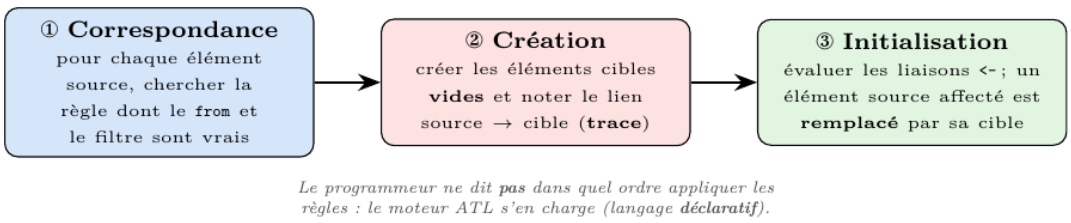


Figure 7.8 – Les trois phases d'exécution du moteur ATL

① **Correspondance** : chaque élément source est testé contre le `from` de chaque règle ;

② **Création** : les éléments cibles sont créés et le lien source → cible est gardé (la **trace**) ;

③ **Initialisation** : les liaisons `<-` sont calculées ; si une liaison reçoit un élément **source**, ATL met à la place l'élément **cible** correspondant.

L'ordre d'écriture des règles n'a donc pas d'importance.

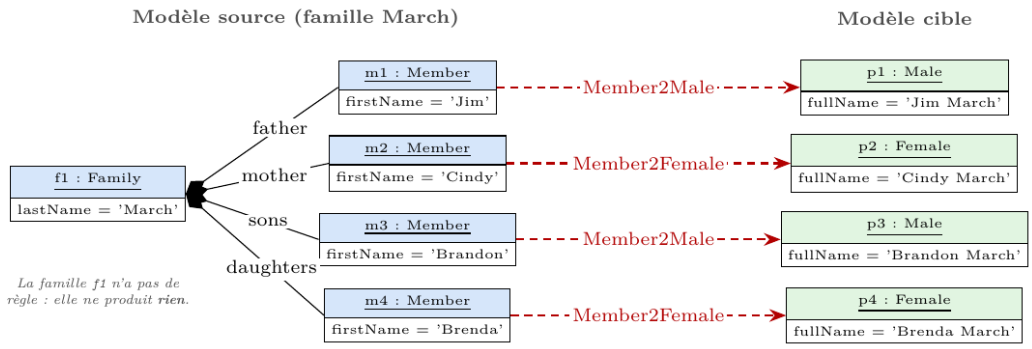


Figure 7.9 – Exécution pas à pas sur la famille March

Tableau 7.4 – Quelle règle pour quel membre ?

Membre	`isFemale()`	**Règle**	`fullName`
Jim (father)	false	Member2Male	Jim March
Cindy (mother)	true	Member2Female	Cindy March
Brandon (sons)	false	Member2Male	Brandon March
Brenda (daughters)	true	Member2Female	Brenda March

7.4 Cas complet : Families2Persons dans Eclipse

Tableau 7.5 – Étapes du tutoriel Families2Persons

Étape	**Action**	**Fichier produit**
0	installer ATL (*Help* → *Install New Software*, ATL SDK)	–
1	créer un projet (*New* → *ATL Project*)	`Families2Persons/`
2	créer les deux méta-modèles (*Ecore Model*)	`Families.ecore`, `Persons.ecore`
3	écrire la transformation (*New* → *ATL File*)	`Families2Persons.atl`
4	écrire un modèle d'exemple	`sample-Families.xmi`
5–6	configurer (*Run Configurations* → *ATL Transformation*) et exécuter	`sample-Persons.xmi`

Méta-modèle Families. `Family` (`lastName : EString [1..1]`) et `Member` (`firstName : EString [1..1]`). Références de `Family` vers `Member` : `father`, `mother` (`1..1`, containment) ; `sons`, `daughters` (`0..*`, containment). Références de `Member` vers `Family` : `familyFather`... `familyDaughter` (`0..1`), avec **EOpposite** renseignée. Le `nsURI` du paquetage vaut `Families` (repris dans le `xmlns` du modèle d'exemple).

Méta-modèle Persons. `Person` (**abstract**, `fullName : EString [1..1]`), `Male` et `Female` avec **ESuper Types** = `Person` ; `nsURI` = `Persons`.

Attention. Oublier **ESuper Types** fait disparaître `fullName` de `Male` et `Female` : les liaisons échouent (« feature does not exist »).

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xmi:XMI xmi:version="2.0" xmlns:xmi="http://www.omg.org/XMI" xmlns="Families">
  <Family lastName="March">
    <father firstName="Jim"/>      <mother firstName="Cindy"/>
```

```
    <sons firstName="Brandon"/>    <daughters firstName="Brenda"/>
  </Family>
  <Family lastName="Sailor">
    <father firstName="Peter"/>    <mother firstName="Jackie"/>
    <sons firstName="David"/>      <sons firstName="Dylan"/>
    <daughters firstName="Kelly"/>
  </Family>
</xmi:XMI>
```

Listing 7.4 – Modèle source sample-Families.xmi

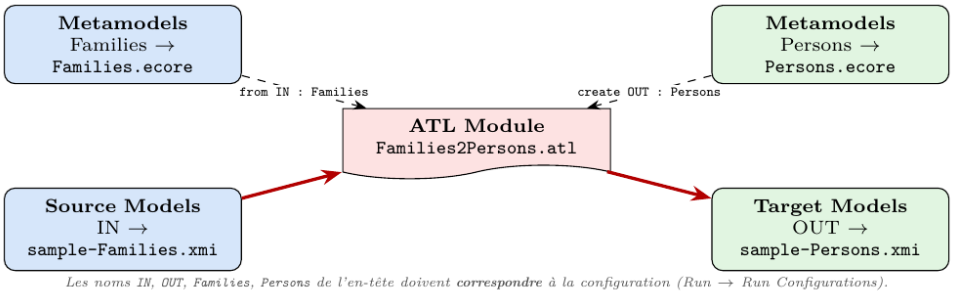


Figure 7.10 – Configuration de l'exécution : modèles IN et OUT et leurs méta-modèles

```
<xmi:XMI xmi:version="2.0" xmlns:xmi="http://www.omg.org/XMI" xmlns="Persons">
  <Male fullName="Jim March"/>      <Male fullName="Brandon March"/>
  <Male fullName="Peter Sailor"/>   <Male fullName="David Sailor"/>
  <Male fullName="Dylan Sailor"/>
  <Female fullName="Cindy March"/> <Female fullName="Brenda March"/>
  <Female fullName="Jackie Sailor"/> <Female fullName="Kelly Sailor"/>
</xmi:XMI>
```

Listing 7.5 – Modèle cible produit sample-Persons.xmi

9 membres en entrée donnent 9 personnes en sortie (5 `Male`, 4 `Female`) ; les familles ont disparu puisqu'aucune règle ne les transforme. L'ordre des éléments dans le fichier produit peut varier d'une exécution à l'autre.

7.5 Aller plus loin

7.5.1 Class2Relational et la résolution automatique

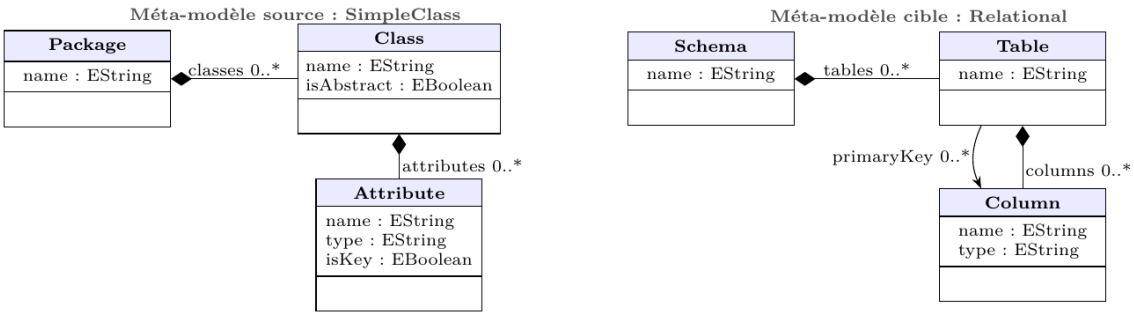


Figure 7.11 – Méta-modèles SimpleClass et Relational

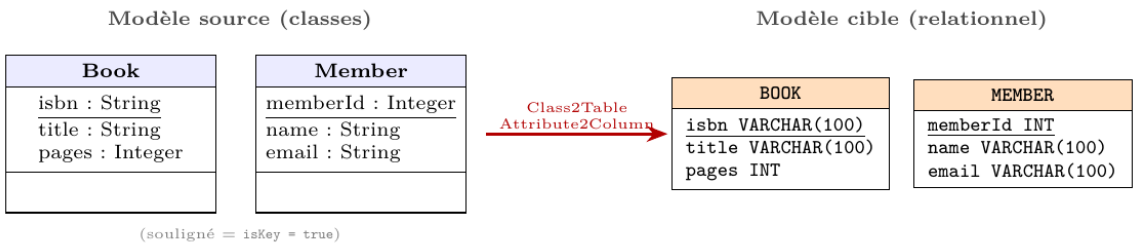


Figure 7.12 – Class2Relational : Package → Schema, Class → Table, Attribute → Column

```
module Class2Relational;
create OUT : Relational from IN : SimpleClass;

helper context String def: toSQL() : String =
    if self = 'String' then 'VARCHAR(100)'
    else if self = 'Integer' then 'INT'
    else if self = 'Boolean' then 'BOOLEAN'
    else if self = 'Date' then 'DATE' else 'VARCHAR(255)'
    endif endif endif endif;

-- la classe qui possède un attribut (le méta-modèle n'a pas de référence inverse)
helper context SimpleClass!Attribute def: owner : SimpleClass!Class =
    SimpleClass!Class.allInstances()->any(c | c.attributes->includes(self));

rule Package2Schema {
    from p : SimpleClass!Package
    to   s : Relational!Schema ( name <- p.name,
                                 tables <- p.classes->select(c | not c.isAbstract) )
}
rule Class2Table {
    from c : SimpleClass!Class (not c.isAbstract)
    to   t : Relational!Table (
           name       <- c.name.toUpper(),
           columns    <- c.attributes,
           primaryKey <- c.attributes->select(a | a.isKey) )
}
rule Attribute2Column {
    from a : SimpleClass!Attribute (not a.owner.isAbstract)
    to   col : Relational!Column ( name <- a.name, type <- a.type.toSQL() )
}
```

Listing 7.6 – Class2Relational (simplifié)

Attention. Les filtres doivent être **cohérents entre eux**. Si `Class2Table` ignore les classes abstraites mais que `Attribute2Column` transforme *tous* les attributs, les colonnes des classes abstraites sont créées sans table pour les contenir : ce sont des objets orphelins dans le modèle cible. D'où le filtre `not a.owner.isAbstract` et le `select` dans `Package2Schema`.

À retenir. `p.classes` contient des `Class` (source), mais `tables` attend des `Table` (cible). ATL **remplace** automatiquement chaque `Class` par la `Table` créée pour elle par `Class2Table` : on n'écrit **aucun** code pour relier les objets. De même, `primaryKey <- ...` reçoit des `Attribute` et obtient les `Column` correspondantes, déjà contenues par `columns` (`primaryKey` est une simple référence). C'est le point le plus important d'ATL.

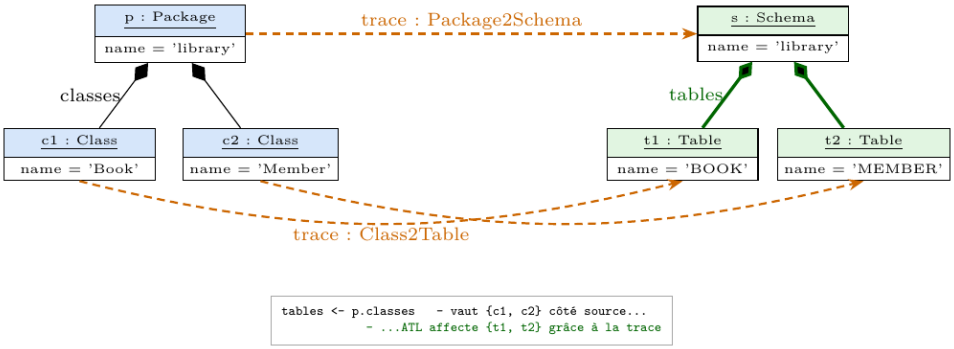


Figure 7.13 – La résolution automatique par la trace

7.5.2 Relier les personnes : le foyer

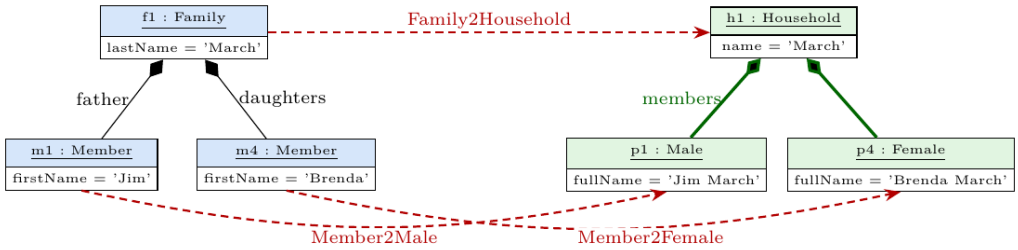


Figure 7.14 – Extension : une règle Family2Household qui utilise la trace

On ajoute au méta-modèle `Persons` une classe `Household` (`name`) qui contient (`members : Person [0..*]`, composition) les personnes du foyer.

```
rule Family2Household {
    from f : Families!Family
    to   h : Persons!Household (
            name    <- f.lastName,
            members <- Sequence{f.father, f.mother, f.sons, f.daughters}->flatten() )
}
```

`members` reçoit des `Member` (source) : ATL y met les `Male`/`Female` créés par les deux autres règles ; `->flatten()` aplatit la séquence de collections. Les personnes sont alors contenues par leur foyer, et non plus à la racine du modèle.

7.5.3 Une règle, plusieurs éléments cibles

Exemple 7.4 – Créer deux éléments cibles avec une seule règle. On veut, pour chaque homme, une personne **et** une adresse e-mail par défaut (le méta-modèle `Persons` reçoit une classe `Contact(email)` et une composition `Person.contact`) :

```
rule Member2MaleAvecContact {
    from s : Families!Member (not s.isFemale())
    to   t : Persons!Male ( fullName <- s.firstName + ' ' + s.familyName, contact <- c ),
         c : Persons!Contact ( email <- s.firstName.toLower() + '.' + s.familyName.toLower() +
     '@mail.ma' )
}
```

La liaison `contact <- c` relie directement les deux éléments créés par la même règle. Jim March devient `Male "Jim March"` avec le contact `jim.march@mail.ma`. Cette règle **remplace** `Member2Male` (sinon deux règles prendraient les mêmes membres). □

Exemple 7.5 – Règle R3 de la chaîne MDA : ajouter une colonne technique.

```
rule ClasseSansId2Table {
    from c : Pim!Classe (c.attributs->forAll(a | not a.estIdentifiant))
    to   t  : Rel!Table   ( nom <- c.nom.toUpper(), colonnes <- Sequence{id}->union(c.
    attributs) ),
         id : Rel!Colonne ( nom <- 'id', type <- 'BIGINT', estCle <- true )
}
```

La colonne `id` n'existe pas dans le PIM : elle est créée par la règle, en plus de la table. La règle `Classe2Table` du chapitre 4 reçoit alors le filtre complémentaire `(c.attributs->exists(a | a.estIdentifiant))`. □

7.5.4 Les autres sortes de règles

Tableau 7.6 – Les sortes de règles ATL

Sorte	Déclenchement	Usage
Matched rule (`rule`)	automatique, pour chaque élément qui satisfait le `from`	cas général
Lazy rule	appelée : `thisModule.R(e)` ; un nouvel élément à chaque appel	copies à la demande
Unique lazy rule	appelée, un seul élément par élément source	partager un élément
Called rule (`rule R(p : T)`)	appelée comme une procédure	partie impérative (`do { }`)

Exemple 7.6 – Une *lazy rule* et une *called rule*.

```
-- Lazy rule : un Contact créé uniquement quand on l'appelle
lazy rule Member2Contact {
    from s : Families!Member
    to   c : Persons!Contact ( email <- s.firstName.toLower() + '@mail.ma' )
}
rule Member2Female {
    from s : Families!Member (s.isFemale())
    to   t : Persons!Female ( fullName <- s.firstName + ' ' + s.familyName,
                              contact  <- thisModule.Member2Contact(s) )
}

-- Called rule : procédure avec partie impérative
rule CreerRapport(n : Integer) {
    to   r : Persons!Report ( title <- 'Transformation Families2Persons' )
    do   { r.count <- n; }
}
-- appelée par exemple depuis un point d'entrée :
entrypoint rule Debut() { do { thisModule.CreerRapport(Families!Member.allInstances()->size())
    ; } }
```

La lazy rule n'est exécutée que pour les femmes, parce que seule `Member2Female` l'appelle. La called rule crée un rapport unique indiquant 9 membres pour le modèle d'exemple. □

Tableau 7.7 – Erreurs fréquentes en ATL

Symptôme	Cause	Solution
erreur sur `module`	nom ≠ nom du fichier, ou `create` absent	corriger l'en-tête
« several rules match »	deux filtres vrais pour un même élément	filtres complémentaires
modèle cible vide	mauvais IN, ou `xmlns` ≠ `nsURI`	vérifier la configuration
« feature does not exist »	propriété mal écrite ou super-type oublié	comparer au méta-modèle
erreur dans un `if`	`else`/`endif` oublié	`if ...then ...else ...endif`
chaîne refusée	guillemets au lieu d'apostrophes	`' '`
création impossible	type cible abstrait	créer `Male`/`Female`
erreur de syntaxe sur une propriété	nom réservé (`from`, `to`, `rule`...)	renommer la propriété

Tableau 7.8 – ATL, Acceleo ou programmation ?

	ATL (M2M)	Acceleo (M2T)	Java + EMF
Produit	un modèle	du texte	texte ou modèle
Style	déclaratif (règles)	template à trous	impératif (boucles)
Parcours du modèle	fait par le moteur	boucles `[for]`	écrit à la main
Relier les objets créés	automatique (trace)	–	à programmer (`Map`)
Typique en MDA	PIM → PSM	PSM → Code	cas particuliers

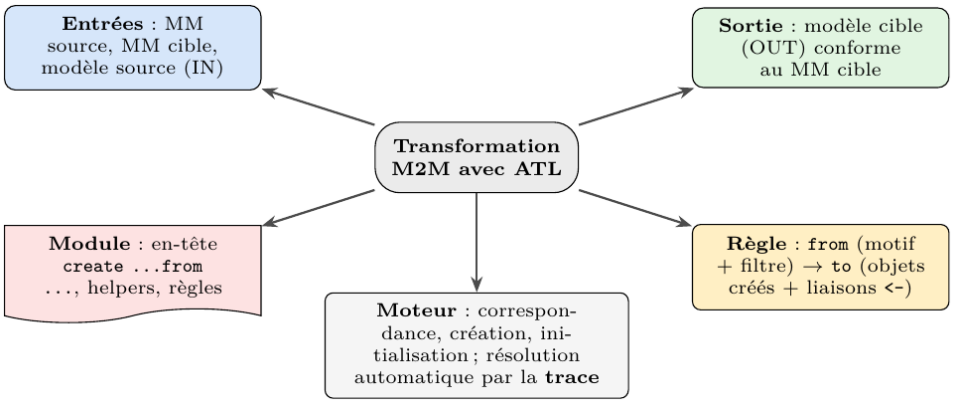


Figure 7.15 – Synthèse : la transformation M2M avec ATL

7.6 Exemples complémentaires

Exemple 7.7 – Book2Publication : agréger des valeurs. Méta-modèle source : un `Book` (`title`) contient des `Chapter` (`title`, `nbPages`, `author`). Méta-modèle cible : une `Publication` (`title`, `nbPages`, `authors`). On ne garde que les livres de plus de 2 pages.

```
module Book2Publication;
create OUT : Publication from IN : Book;

helper context Book!Book def: nbPages : Integer =
    self.chapters->collect(c | c.nbPages)->sum();
helper context Book!Book def: authors : String =
    self.chapters->collect(c | c.author)->asSet()
        ->iterate(a ; acc : String = '' | if acc = '' then a else acc + ' and ' + a endif);

rule Book2Publication {
    from b : Book!Book (b.nbPages > 2)
    to   p : Publication!Publication (
           title   <- b.title,
           nbPages <- b.nbPages,
           authors <- b.authors )
}
```

Un livre de deux chapitres (12 pages par Alice, 8 pages par Bob) donne une publication de 20 pages dont les auteurs sont « Alice and Bob » (l'ordre des noms n'est pas garanti, `asSet` n'étant pas ordonné). Les helpers *attribut* évitent de recalculer les sommes. □

Exemple 7.8 – Une transformation endogène : copier en filtrant. On veut une copie d'un modèle de familles sans les familles qui n'ont pas d'enfant.

```
module FiltrerFamilles;
create OUT : Families from IN : Families;

helper context Families!Family def: hasChildren : Boolean =
    self.sons->notEmpty() or self.daughters->notEmpty();
helper context Families!Member def: family : Families!Family =
    if not self.familyFather.oclIsUndefined() then self.familyFather
    else if not self.familyMother.oclIsUndefined() then self.familyMother
    else if not self.familySon.oclIsUndefined() then self.familySon
    else self.familyDaughter endif endif endif;

rule CopyFamily {
    from s : Families!Family (s.hasChildren)
    to   t : Families!Family ( lastName <- s.lastName, father <- s.father, mother <- s.mother,
                               sons <- s.sons, daughters <- s.daughters )
}
rule CopyMember {
    from s : Families!Member (s.family.hasChildren)
    to   t : Families!Member ( firstName <- s.firstName )
}
```

Source et cible ont le même méta-modèle : la transformation est **endogène**. Les deux filtres s'appuient sur le même helper, si bien qu'aucun membre n'est copié sans sa famille. ATL propose aussi un mode `refining` qui copie automatiquement tout ce qui n'est pas modifié. □

7.7 Exercices

Les corrigés figurent en annexe A.

Exercice 7.1 – Lire une règle. Expliquer en français ce que fait la règle suivante et donner le résultat pour les familles March et Sailor.

```
rule Family2Household {
    from f : Families!Family (f.sons->size() + f.daughters->size() >= 2)
    to   h : Persons!Household ( name <- 'Foyer ' + f.lastName )
}
```

Exercice 7.2 – Helper. Écrire un helper `role : String` sur `Families!Member` qui renvoie `'père'`, `'mère'`, `'fils'` ou `'fille'`.

Exercice 7.3 – Nouvelle cible. Le méta-modèle `Persons` reçoit un attribut `role : String` dans `Person`. Modifier les deux règles pour le renseigner, en utilisant le helper de l'exercice 7.2.

Exercice 7.4 – Class2Relational. Le méta-modèle `SimpleClass` reçoit un attribut `multiValued : EBoolean` dans `Attribute`. Ajouter à `Class2Relational` une règle qui transforme chaque attribut **multivalué** en une **table** séparée nommée `<CLASSE>_<attribut>` avec deux colonnes : `id` et la valeur. Quelle modification faut-il faire au filtre de `Attribute2Column` ?

Exercice 7.5 – Trouver les erreurs. Ce module (fichier `F2P.atl`) contient quatre erreurs.

```
module F2P;
create OUT : Persons from IN : Families
rule Member2Person {
    from s : Families!Member
    to   t : Persons!Person ( fullName <- s.firstName + " " + s.familyName )
}
```

Exercice 7.6 – Concevoir une transformation. Écrire une transformation ATL `StateMachine2Table` qui, à partir du méta-modèle de machines à états du chapitre 2 (exemple 2.14), produit un modèle `Table` composé d'une `Table(name)` contenant des `Row(source, event, target)` : une ligne par transition.

7.8 Quiz

Répondez aux questions suivantes. Les corrigés commentés figurent en annexe A.

Q1. Dans `create OUT : Persons from IN : Families;`, `Families` désigne :

a) le modèle source

b) le méta-modèle source

c) le modèle cible

d) le nom du module

Q2. Le nom d'un module ATL doit être :

a) celui du méta-modèle source
b) celui du fichier `.atl`
c) libre
d) celui du modèle cible

Q3. Dans une règle, la partie entre parenthèses après `from s : Families!Member` est :

a) une liaison
b) le filtre (garde)
c) un helper
d) le motif cible

Q4. Que signifie l'opérateur `<-` ?

a) une affectation dans le modèle source
b) une liaison : donner une valeur à une propriété de l'élément cible
c) une comparaison
d) un appel de règle

Q5. Peut-on créer un élément de type `Persons!Person` ?

a) Oui
b) Non, `Person` est abstraite
c) Oui, avec un helper
d) Seulement dans une lazy rule

Q6. Un helper attribut se distingue d'un helper opération car :

a) il modifie le modèle
b) il n'a pas de paramètres et sa valeur est mise en cache
c) il est appelé avec `thisModule`
d) il produit un élément cible

Q7. Si deux règles ont des filtres vrais pour un même élément :

a) les deux s'appliquent
b) seule la première s'applique
c) ATL signale une erreur
d) aucune ne s'applique

Q8. Dans `tables <- p.classes`, ATL :

a) copie les classes dans le modèle cible
b) remplace chaque Class par la Table créée pour elle (trace)

c) provoque une erreur de type

d) crée de nouvelles tables vides

Q9. Families → Persons est une transformation :

a) endogène

b) exogène et horizontale

c) exogène et verticale

d) bidirectionnelle

Q10. Une *lazy rule* est déclenchée :

a) automatiquement pour chaque élément

b) uniquement lorsqu'elle est appelée explicitement

c) à la fin de la transformation

d) jamais

Q11. On ajoute la famille Benali (père Ahmed, mère Fatima, fils Yassine, filles Salma et Nour). Combien d'éléments contient le modèle cible ? Faut-il modifier le module ?

Q12. Écrire un helper `nbMembers : Integer` sur `Families!Family`.

Corrigés des exercices et des quiz

A.1 Chapitre 1 – Introduction à l'IDM

Exercices

Exercice 1.1. (a) M1 ; (b) M2 ; (c) M3 ; (d) M0 ; (e) M2 ; (f) M1 ; (g) M3 ; (h) M1 ; (i) M0.

Exercice 1.2. Le livre physique (M0) est *instance de* la classe `Livre` du diagramme de la bibliothèque (M1) ; `Livre` est *instance de* la méta-classe `Class` d'UML (M2) ; `Class` est *instance de* `MOF::Class` (M3), elle-même instance d'elle-même. Les modèles sont *conformes* au niveau supérieur : le diagramme est conforme à UML, UML au MOF.

Exercice 1.3. (a) prescriptif, contemplatif ; (b) prescriptif, productif ; (c) descriptif, productif ; (d) prescriptif ou descriptif selon l'usage, contemplatif.

Exercice 1.4. Méta-classes : `System(name)` contient (composition) des `UseCase(name)` ; `Actor(name)` ; `Association` reliant un `Actor [1]` et un `UseCase [1]` ; `Include` et `Extend` reliant deux `UseCase` (`base [1]`, `addition [1]`). Une classe racine `Model` contient les systèmes, les acteurs et les relations.

Exercice 1.5. Non conforme : la relation *Include* relie deux `UseCase` ; « Banque inclut Retirer » relie un `Actor` à un `UseCase` avec une `Include`, ce que le méta-modèle n'autorise pas. Il faut une `Association` Banque–Retirer.

Exercice 1.6. (a) Vrai (il est conforme au MOF). (b) Vrai en principe : un modèle peut respecter plusieurs méta-modèles compatibles (par exemple deux versions d'un même méta-modèle, ou un méta-modèle et une extension) ; mais un fichier est toujours rattaché à *un* méta-modèle de référence par son `nsURI`. (c) Faux : Ecore est un méta-méta-modèle (M3). (d) Faux : M0 contient des objets réels, les classes sont en M1. (e) Faux : le comportement spécifique est souvent complété à la main (zones protégées).

Quiz

Q1. c) Environ 100 fois plus (ordre de grandeur) : d'où l'intérêt de vérifier les modèles dès le début du projet.

Q2. b) Contemplatif : il a servi à communiquer ; aucun outil ne peut l'exploiter.

Q3. b) Prescriptif (un modèle descriptif décrit un système existant).

Q4. b) Oui : c'est un modèle conforme au MOF (M3). Il est « méta » seulement par rapport à vos diagrammes.

Q5. b) « instance de ». La relation « conforme à » relie un modèle M1 à son méta-modèle M2, et M2 à M3.

Q6. b) Le MOF est réflexif : il est décrit avec lui-même.

Q7. c) M2 : `Transition` est un concept du langage des machines à états.

Q8. d) M3 : `EReference` est un concept d'Ecore.

Q9. b) Un schéma XSD définit les éléments et types autorisés : il joue le rôle de méta-modèle.

Q10. b) Non conforme : la contrainte « exactement un état initial » est violée.

Q11. (a) M2 ; (b) M1 ; (c) M0 ; (d) M1 ; (e) M0.

Q12. Apports : détection précoce des erreurs, langage commun, productivité, traçabilité, pérennité. Limites : coût d'apprentissage, outillage lourd, un mauvais modèle génère un mauvais code.

A.2 Chapitre 2 – Méta-modélisation avec EMF et Ecore

Exercices

Exercice 2.1.

- `Ecole` (`nom`) : ♦ `cours : Cours [0..*]`, ♦ `enseignants : Enseignant [0..*]`, ♦ `etudiants : Etudiant [0..*]` ;
- `Cours : code : EString [1]`, `intitule : EString [1]`, `credits : EInt [1]`, `responsable : Enseignant [1]` (simple référence) ;
- `Enseignant : nom : EString [1]`, `grade : Grade [1]` ; EEnum `Grade` = {ASSISTANT, PROFESSEUR} ;
- `Etudiant : cne : EString [1]`, `nom : EString [1]`, ♦ `notes : Note [0..*]` ;
- `Note : valeur : EDouble [1]`, `cours : Cours [1]` (simple référence).

« Un étudiant suit des cours » se déduit de ses notes ; on peut aussi ajouter `Etudiant.suit : Cours [0..*]` (simple référence) si un étudiant peut suivre un cours sans encore avoir de note.

Exercice 2.2. (a) 1, -1 ; (b) 0, 3 ; (c) 0, 1 ; (d) 1, -1.

Exercice 2.3.

```
<library:Library xmi:version="2.0" xmlns:xmi="http://www.omg.org/XMI"
    xmlns:library="http://fr.obeo.library/1.0" name="ENSIAS">
  <book id="B1" title="Le Petit Prince" isbn="978-2070612758" author="Saint-Exupéry" available
    ="true"/>
  <book id="B2" title="L'Alchimiste" isbn="978-2290004449" author="P. Coelho"/>
  <member memberId="M1" name="Sara" email="sara@um5.ac.ma"/>
  <loan loanDate="2026-11-02" book="//@book.1" member="//@member.0"/>
</library:Library>
```

Le second livre est emprunté : `available` garde sa valeur par défaut `false` et n'est pas écrit.

Exercice 2.4. (1) le `nsURI .../2.0` ne correspond pas à celui du méta-modèle (`.../1.0`) ; (2) `available="oui"` n'est pas un `EBoolean` (`true`/`false`) ; (3) `//@book.3` désigne un 4e livre qui n'existe pas. On peut aussi noter l'absence de la déclaration `xmlns:xmi`, des attributs obligatoires (`isbn`, `author`) et du membre de l'emprunt (facultatif dans le méta-modèle, mais attendu par le métier).

Exercice 2.5.

```
LibraryFactory f = LibraryFactory.eINSTANCE;
Member m = f.createMember();  m.setName("Sara");
Book b = f.createBook();      b.setTitle("Le Petit Prince");
Loan l = f.createLoan();      l.setMember(m); l.setBook(b); l.setLoanDate(new Date());
lib.getMember().add(m); lib.getBook().add(b); lib.getLoan().add(l);
System.out.println(lib.getBook().size());
```

Exercice 2.6. `Etudiant` devient **abstraite** ; deux sous-classes `Licence` et `Master` (`eSuperTypes = Etudiant`) ; `Master` ajoute `sujetMemoire : EString [1]`. La composition `Ecole.etudiants : Etudiant [0..*]` accepte les deux sous-classes.

Quiz

Q1. b) une `EReference`.

Q2. b) composition : l'objet cible est contenu par l'objet source.

Q3. c) le `nsURI`, repris dans l'attribut `xmlns` du fichier XMI.

Q4. b) générer l'API Java.

Q5. c) -1.

Q6. b) il n'est pas enregistré dans le fichier XMI (et la sauvegarde échoue s'il est référencé).

Q7. b) `LibraryFactory.eINSTANCE.createBook()`.

Q8. b) maintenir automatiquement deux références qui se répondent.

Q9. b) `xsi:type="media:Dvd"`.

Q10. Une `EEnum` (`BookState` : AVAILABLE, BORROWED, LATE) et un attribut `state : BookState [1]`.

Q11. Un attribut ne porte qu'une valeur (le nom) ; une classe `Author` a une identité, plusieurs propriétés (nationalité…) et peut être partagée par plusieurs livres.

A.3 Chapitre 3 – Le langage OCL

Exercices

Exercice 3.1. (a) `'Amrani'` ; (b) `65` (42 + 23) ; (c) `1` (p4) ; (d) `Bag{'Youssef', 'Omar'}` ; (e) `Bag{'R&D'}` ; (f) `'Sara'` (conjoint de p4 = p5, dont le manager est p1) ; (g) `false` (le plus jeune a 23 ans).

Exercice 3.2.

```
context Departement inv budgetPositif: self.budget > 0
context Entreprise  inv nomLong:       self.nom.size() >= 2
context Personne    inv deptDeMonEmployeur:
  not self.departement.oclIsUndefined() implies
      self.employeur.departements->includes(self.departement)
context Personne    inv memeDeptQueManager:
  not self.manager.oclIsUndefined() implies self.manager.departement = self.departement
```

Aucun n'est violé par le diagramme d'étude (p2 et p5 sont en R&D comme p1 ; p4 est aux Ventes comme p3).

Exercice 3.3.

```
e1.employes.salaire->sum() / e1.employes->size()                          -- 9800.0
e1.departements->select(d | d.chef.oclIsUndefined()).nom                   -- Bag{} (vide)
e1.employes->select(p | p.manager.oclIsUndefined() and p.subordonnes->isEmpty())   -- Set{}
e1.employes->sortedBy(p | p.age)->first()                                 -- p4
e1.employes->select(p | not p.conjoint.oclIsUndefined())->size() div 2    -- 1
```

Exercice 3.4.

```
context Cadre inv primeMax: self.prime <= 0.3 * self.salaire
e1.employes->select(p | p.oclIsKindOf(Cadre))->collect(p | p.oclAsType(Cadre).prime)->sum()
    -- 5000
```

(c) `p2.oclIsTypeOf(Cadre)` vaut `false`. L'invariant (a) est vrai pour p1 ($3000 \leq 4500$) et p3 ($2000 \leq 3600$).

Exercice 3.5.

```
context Departement::ajouterMembre(p : Personne)
  pre:  self.membres->excludes(p) and p.employeur.departements->includes(self)
  post: self.membres = self.membres@pre->including(p) and p.departement = self

context Personne::changerManager(m : Personne)
  pre:  m <> self and m.employeur = self.employeur
  post: self.manager = m and m.subordonnes->includes(self)

context Entreprise::licencier(p : Personne)
  pre:  self.employes->includes(p)
  post: self.employes = self.employes@pre->excluding(p)
```

Exercice 3.6. Ici `emprunts` désigne les livres actuellement empruntés :

```
context Adherent inv max3:        self.emprunts->size() <= 3
context Adherent inv indispo:     self.emprunts->forAll(l | not l.disponible)
context Adherent inv mineur:      self.age < 18 implies self.emprunts->size() <= 1
context Livre    inv isbnUnique:  Livre.allInstances()->isUnique(l | l.isbn)
```

Quiz

Q1. c) Non, OCL est sans effet de bord.

Q2. b) `'Sara'` : le manager de p2 est p1.

Q3. b) `Set(Personne)`.

Q4. c) `Bag(Real)` : navigation à partir d'une collection.

Q5. c) `Sexe::FEMME`.

Q6. b) false : `oclIsTypeOf` teste le type exact (`Cadre`).

Q7. b) `forAll`.

Q8. b) 2 (p1 et p3).

Q9. b) la valeur avant l'exécution de l'opération.

Q10. b) `oclIsUndefined()`.

Q11. `false` : Omar Tazi et Salma Tazi ont le même nom.

Q12. `context Departement inv: self.membres->size() <= 50`.

A.4 Chapitre 4 – MDA

Exercices

Exercice 4.1. (a) CIM ; (b) PIM ; (c) PSM relationnel ; (d) code (ou PSM Spring Data) ; (e) CIM ; (f) PSM Java/JPA.

Exercice 4.2.

```
CREATE TABLE CLIENT (email VARCHAR(255) PRIMARY KEY, nom VARCHAR(255));      -- R1, R2
CREATE TABLE COMMANDE (id BIGINT PRIMARY KEY,                                -- R3
  "date" DATE, montant DECIMAL(10,2),                                        -- R2
  client_id VARCHAR(255) REFERENCES CLIENT(email));                          -- R4
```

La clé étrangère reprend le **type de la clé primaire** de `CLIENT` (`VARCHAR(255)`), et non `BIGINT`. `DATE` étant un mot réservé dans plusieurs SGBD, on protège le nom de colonne par des guillemets ou on la renomme (`date_commande`) : une règle de nommage de plus pour le PDM.

Exercice 4.3. (a) PIM → texte, exogène, verticale, M2T ; (b) PIM → PIM, endogène, horizontale, M2M ; (c) texte → modèle, T2M ; (d) exogène (BPMN ≠ UML), horizontale, M2M.

Exercice 4.4.

```
<xsl:template match="/">
  <html><body><ul>
    <xsl:for-each select="/films/film">
      <xsl:sort select="@annee" data-type="number"/>
      <li><xsl:value-of select="titre"/> (<xsl:value-of select="@annee"/>)</li>
    </xsl:for-each>
  </ul></body></html>
</xsl:template>
```

Résultat : « Matrix (1999) » puis « Inception (2010) ». `data-type="number"` assure un tri numérique.

Exercice 4.5. En français : (1) chaque classe du PIM devient une classe Java publique de même nom ; (2) chaque attribut devient un champ privé et une méthode `getXxx` ; (3) chaque extrémité d'association de borne supérieure * devient un champ `List<Cible>`. En ATL :

```
helper context String def: firstToUpper() : String =
    self.substring(1, 1).toUpper() + self.substring(2, self.size());
helper context Pim!TypePrimitif def: javaType() : String =
    if self = #STRING then 'String' else if self = #INTEGER then 'int'
    else if self = #REAL then 'double' else if self = #BOOLEAN then 'boolean'
    else 'java.util.Date' endif endif endif endif;
```

```
rule Classe2JavaClass {
  from c : Pim!Classe
  to   j : Java!JavaClass ( name <- c.nom, visibility <- #public,
                            fields <- c.attributs->union(c.extremites->select(e | e.upper =
    -1)) )
}
rule Attribut2Field {
  from a : Pim!Attribut
  to   f : Java!Field  ( name <- a.nom, type <- a.type.javaType(), visibility <- #private ),
       g : Java!Method ( name <- 'get' + a.nom.firstToUpper(), returnType <- a.type.javaType()
     )
}
rule AssocN2List {
  from e : Pim!Extremite (e.upper = -1)
  to   f : Java!Field ( name <- e.nom, type <- 'List<' + e.cible.nom + '>', visibility <- #
    private )
}
```

Pour ranger les méthodes `g` dans leur classe, `Classe2JavaClass` ajoute la liaison `methods <- c.attributs->collect(a | thisModule.resolveTemp(a, 'g'))` : `resolveTemp` retrouve l'élément cible nommé `g` créé pour `a`.

Exercice 4.6. Fichiers COBOL → PSM COBOL/fichiers (T2M, rétro-ingénierie) → PIM « Contrats » (rétro-ingénierie, verticale) → PIM raffiné (endogène) → PSM Java/JPA + PSM PostgreSQL + PSM REST (projections) → code (M2T). Pour une application mobile, on **réutilise le PIM** (et le CIM) et on écrit une nouvelle projection PIM → PSM mobile.

Quiz

Q1. b) l'OMG, en 2001.

Q2. b) le PIM.

Q3. b) PIM + PDM ⇒ PSM ⇒ Code.

Q4. c) le PSM (Java/JPA).

Q5. b) un modèle M1.

Q6. b) endogène et horizontale.

Q7. c) rétro-ingénierie.

Q8. b) M2T.

Q9. a) par template.

Q10. b) garder la trace source → cible pour créer les clés étrangères (R4).

Q11. (a) CIM ; (b) PIM ; (c) PSM relationnel ; (d) Code ; (e) CIM.

Q12. (a) template (Acceleo, XSLT) ; (b) modélisation (ATL, QVT).

A.5 Chapitre 5 – Acceleo

Exercices

Exercice 5.1. `MEMBER(memberId;name;email)`.

Exercice 5.2.

```
[for (cls : EClass | root.eClassifiers->filter(EClass))]
[file ('csv/' + cls.name + '.csv', false, 'UTF-8')]
[for (a : EAttribute | cls.eAttributes) separator(',')][a.name/][/for]
[/file]
[/for]
```

Exercice 5.3.

```
[query public toSQLType(typeName : String) : String =
    if typeName = 'EString' then 'VARCHAR(100)'
    else if typeName = 'EInt' then 'INT'
    else if typeName = 'ELong' then 'BIGINT'
    else if typeName = 'EDouble' then 'DECIMAL(10,2)'
    else if typeName = 'EBoolean' then 'BOOLEAN'
    else if typeName = 'EDate' then 'DATE'
    else 'VARCHAR(255)' endif endif endif endif endif endif /]
...
    [ref.name/]_id [ref.eReferenceType.pkType()/][if (ref.lowerBound = 1)] NOT NULL[/if],
```

Dans `library.ecore`, `Loan.book` et `Loan.member` sont `[0..1]` : aucune colonne ne change ; il faudrait les déclarer `[1]` pour obtenir `book_id VARCHAR(100) NOT NULL`. Pour conserver aussi le lien de composition (« ce livre appartient à cette bibliothèque »), on parcourt les compositions qui *pointent vers* la classe et on ajoute une colonne `library_id` dans la table contenue.

Exercice 5.4. (1) `[comment @main/]` manque : le template n'est pas un point d'entrée ; (2) `root.eClassifiers` doit être filtré : `->filter(EClass)` ; (3) `[c.name]` doit s'écrire `[c.name/]` ; (4) `[/if` n'est pas fermé et `[/file]` manque avant `[/template]`.

Exercice 5.5.

```
[query public tsType(t : String) : String =
    if t = 'EString' then 'string' else if t = 'EInt' or t = 'EDouble' then 'number'
    else if t = 'EBoolean' then 'boolean' else if t = 'EDate' then 'Date'
    else 'any' endif endif endif endif /]

[for (cls : EClass | root.eClassifiers->filter(EClass))]
[file ('ts/' + cls.name + '.ts', false, 'UTF-8')]
export interface [cls.name/] {
[for (a : EAttribute | cls.eAttributes)]
  [a.name/][if (a.lowerBound = 0)]?[/if]: [a.eAttributeType.name.tsType()/];
[/for]
[for (r : EReference | cls.eReferences)]
  [r.name/][if (r.lowerBound = 0)]?[/if]: [r.eReferenceType.name/][if (r.many)]['[]'/][/if];
[/for]
}
[/file]
[/for]
```

Exercice 5.6. La méthode est **perdue** : le fichier est écrasé. Il faut l'écrire entre `// Start of user code methodes Loan` et `// End of user code`, c'est-à-dire dans la zone générée par `[protected]`.

Quiz

Q1. b) MOFM2T.

Q2. b) il est recopié tel quel.

Q3. b) il marque le template point d'entrée.

Q4. b) seulement les `EClass`.

Q5. b) écraser le fichier.

Q6. b) un template produit du texte, une query calcule une valeur.

Q7. c) entre les éléments, jamais après le dernier.

Q8. b) écrasé.

Q9. b) une chaîne vide : Book n'a aucune référence.

Q10. Pour Member, le résultat est `memberId, name` :

```
[for (a : EAttribute | cls.eAttributes->select(x | x.lowerBound >= 1)) separator(', ')][a.name
    /][/for]
```

Q11. Seulement le modèle `library.ecore`, puis relancer les trois générateurs.

A.6 Chapitre 6 – Transformation par programmation

Exercices

Exercice 6.1. `Library : 3` puis `Loan : 2` (Book et Member n'ont pas de référence).

Exercice 6.2.

```
int nbAttributsObligatoires(EPackage p) {
    int n = 0;
    for (EClassifier c : p.getEClassifiers())
        if (c instanceof EClass cls)
            for (EAttribute a : cls.getEAttributes())
                if (a.getLowerBound() > 0) n++;
    return n;
}
EClass racine(EPackage p) {
    Set<EClass> contenues = new HashSet<>();
    for (EClassifier c : p.getEClassifiers())
        if (c instanceof EClass cls)
            for (EReference r : cls.getEReferences())
                if (r.isContainment()) contenues.add(r.getEReferenceType());
    for (EClassifier c : p.getEClassifiers())
        if (c instanceof EClass cls && !contenues.contains(cls)) return cls;
    return null;
}
```

`library.ecore` a 11 attributs (1 + 5 + 3 + 2), dont **8** obligatoires : `name` ; `id`, `title`, `isbn`, `author` ; `memberId`, `name` ; `loanDate`. La racine est `Library`.

Exercice 6.3. (1) le `Writer` n'est jamais fermé et n'est pas en UTF-8 : utiliser `try (Writer w = new FileWriter(..., StandardCharsets.UTF_8))` ; (2) le transtypage `(EClass) c` échoue sur un `EDataType` ou un `EEnum` : tester `instanceof` ; (3) il manque les retours à la ligne et l'indentation, et la liste d'attributs a une virgule finale : utiliser `\n` et `String.join`.

Exercice 6.4.

```
String jsonSchema(EClass cls) {
    List<String> props = new ArrayList<>(), req = new ArrayList<>();
    for (EAttribute a : cls.getEAttributes()) {
        String t = switch (a.getEAttributeType().getName()) {
            case "EInt" -> "integer"; case "EBoolean" -> "boolean"; default -> "string"; };
        props.add("\"" + a.getName() + "\": {\"type\": \"" + t + "\"}");
        if (a.getLowerBound() > 0) req.add("\"" + a.getName() + "\"");
    }
    return "{\"title\": \"" + cls.getName() + "\", \"type\": \"object\",\n \"properties\": {"
         + String.join(", ", props) + "},\n \"required\": [" + String.join(", ", req) + "]}";
}
```

Exercice 6.5.

```
EPackage dto = EcoreFactory.eINSTANCE.createEPackage();
dto.setName("library_dto"); dto.setNsURI("http://ensias.ma/library_dto"); dto.setNsPrefix("dto
    ");
for (EClassifier c : p.getEClassifiers())
    if (c instanceof EClass cls) {
        EClass d = EcoreFactory.eINSTANCE.createEClass();
        d.setName(cls.getName() + "Dto");
        for (EAttribute a : cls.getEAttributes())
            d.getEStructuralFeatures().add(EcoreUtil.copy(a));   // copie : la source reste
    intacte
        dto.getEClassifiers().add(d);
    }
Resource r = rs.createResource(URI.createFileURI("gen/library_dto.ecore"));
r.getContents().add(dto);
r.save(null);
```

Exercice 6.6. (a) template : texte long et régulier ; (b) programmation : algorithme de graphe ; (c) template ; (d) programmation (intégration dans un outil, arguments), qui peut appeler un template.

Quiz

Q1. b) impérative.

Q2. a) `emf.common`, `emf.ecore`, `emf.ecore.xmi`.

Q3. b) un `ResourceSet`.

Q4. b) `isContainment()`.

Q5. b) `if (c instanceof EClass cls)`.

Q6. a) `String.join(", ", liste)`.

Q7. b) fermer automatiquement le fichier, même en cas d'exception.

Q8. c) les références simples et non contenantes.

Q9. b) `ref.many`.

Q10. Rien : le programme parcourt `getEClassifiers()` et trouve la nouvelle classe.

Q11. Programmation : calculs, tris, intégration dans une application, M2M ou validation. Template : nombreuses pages ou classes au texte long et régulier.

A.7 Chapitre 7 – ATL

Exercices

Exercice 7.1. Pour chaque famille ayant au moins deux enfants, la règle crée un foyer nommé « Foyer » suivi du nom de famille. Sailor a 3 enfants : `Household "Foyer Sailor"`. March (2 enfants) donne aussi `"Foyer March"`. Les membres ne sont pas transformés par cette règle.

Exercice 7.2.

```
helper context Families!Member def: role : String =
    if not self.familyFather.oclIsUndefined() then 'père'
    else if not self.familyMother.oclIsUndefined() then 'mère'
    else if not self.familySon.oclIsUndefined() then 'fils'
    else 'fille' endif endif endif;
```

Exercice 7.3. Ajouter la liaison `role <- s.role` dans les deux règles : `t : Persons!Male ( fullName <- s.firstName + ' ' + s.familyName, role <- s.role )`.

Exercice 7.4. Avec le helper `owner` du listing 7.6 :

```
rule MultiValuedAttribute2Table {
    from a : SimpleClass!Attribute (a.multiValued and not a.owner.isAbstract)
    to   t  : Relational!Table  ( name <- a.owner.name.toUpper() + '_' + a.name,
                                  columns <- Sequence{id, v}, primaryKey <- Sequence{id} ),
         id : Relational!Column ( name <- 'id', type <- 'BIGINT' ),
         v  : Relational!Column ( name <- a.name, type <- a.type.toSQL() )
}
```

Le filtre de `Attribute2Column` devient `(not a.multiValued and not a.owner.isAbstract)` pour qu'un attribut ne soit pris que par une règle ; `Package2Schema` doit aussi ranger les nouvelles tables : `tables <- p.classes->select(...)->union(...les attributs multivalués...)`.

Exercice 7.5. (1) `;` manquant après `create ... from IN : Families` ; (2) guillemets `" "` au lieu d'apostrophes `' '` ; (3) `Persons!Person` est abstraite : il faut deux règles `Male`/`Female` ; (4) le helper `familyName` n'est pas défini dans le module.

Exercice 7.6.

```
module StateMachine2Table;
create OUT : Table from IN : SM;

rule Machine2Table {
    from m : SM!StateMachine
    to   t : Table!Table ( name <- m.name, rows <- m.transitions )
}
rule Transition2Row {
    from tr : SM!Transition
    to   r  : Table!Row ( source <- tr.source.name, event <- tr.event, target <- tr.target.
    name )
}
```

Pour le feu : trois lignes (Rouge, after(25s), Vert), (Vert, after(20s), Orange), (Orange, after(5s), Rouge). Les propriétés ne peuvent pas s'appeler `from` et `to` : ce sont des mots réservés d'ATL.

Quiz

Q1. b) le méta-modèle source.

Q2. b) celui du fichier `.atl`.

Q3. b) le filtre (garde).

Q4. b) une liaison.

Q5. b) Non : `Person` est abstraite.

Q6. b) pas de paramètres et valeur mise en cache.

Q7. c) erreur à l'exécution.

Q8. b) résolution automatique par la trace.

Q9. b) exogène et horizontale.

Q10. b) uniquement lorsqu'elle est appelée.

Q11. 14 personnes (9 + 5) ; le module n'est pas modifié.

Q12. Le père et la mère sont obligatoires (`1..1`) :

```
helper context Families!Family def: nbMembers : Integer =
    2 + self.sons->size() + self.daughters->size();
```

Aide-mémoire et glossaire

B.1 OCL

```
context C inv nom: <expression booléenne>
context C::op(p : T) : R  pre: <...>  post: <... result ... self.x@pre ...>
context C::op(p : T) : R  body: <expression>
context C::a : T  init: <valeur>   |   derive: <expression>
context C def: nom : T = <expression>
let x : T = <expr> in <expr>
if <cond> then <e1> else <e2> endif            -- else obligatoire
not a.oclIsUndefined() implies <...>          -- garde d'une navigation 0..1
coll->size() / isEmpty() / notEmpty() / includes(o) / excludes(o) / includesAll(c)
coll->select(x | c) / reject(x | c) / collect(x | e) / forAll(x | c) / exists(x | c)
coll->isUnique(x | e) / one(x | c) / any(x | c) / sortedBy(x | e) / sum() / first() / last()
coll->iterate(x ; acc : T = init | expr)
coll->union(c) / intersection(c) / including(o) / excluding(o) / asSet() / asSequence()
o.oclIsKindOf(T) / o.oclIsTypeOf(T) / o.oclAsType(T) / o.oclIsUndefined()
T.allInstances()        Enum::LITTERAL        'chaîne'
```

B.2 Acceleo 3

```
[module nom('http://www.eclipse.org/emf/2002/Ecore')/]
[template public gen(root : EPackage)] [comment @main/] ... [/template]
[file (chemin, false, 'UTF-8')] ... [/file]          -- false : écraser, true : ajouter
[cls.name/]   [attr.name.toUpperFirst()/]   ['['/]
[for (a : EAttribute | cls.eAttributes) separator(', ')] ... [/for]
[if (cond)] ... [elseif (cond)] ... [else] ... [/if]
[let n : Integer = cls.eAttributes->size()] ... [/let]
[query public f(x : T) : R = expression /]
[genClass(cls)/]   ou   [cls.genClass()/]
// [protected ('id unique')] ... // [/protected]
[comment texte /]
root.eClassifiers->filter(EClass)   cls.eAttributes   cls.eReferences   ref.many   ref.
    containment
```

B.3 API EMF (Java)

```
ResourceSet rs = new ResourceSetImpl();
rs.getResourceFactoryRegistry().getExtensionToFactoryMap().put("ecore", new
    XMIResourceFactoryImpl());
EPackage p = (EPackage) rs.getResource(URI.createFileURI("model/library.ecore"), true).
    getContents().get(0);
for (EClassifier c : p.getEClassifiers()) if (c instanceof EClass cls) { ... }
cls.getEAttributes(); cls.getEReferences(); cls.getESuperTypes(); cls.isAbstract();
```

```
attr.getName(); attr.getEAttributeType().getName(); attr.getLowerBound(); attr.getUpperBound()
    ;
ref.isMany(); ref.isContainment(); ref.getEReferenceType(); ref.getEOpposite();
EcoreFactory.eINSTANCE.createEClass();  EcoreUtil.copy(o);  resource.save(null);
try (Writer w = new FileWriter(nom, StandardCharsets.UTF_8)) { w.write(...); }
```

B.4 ATL

```
-- @path MMs=/projet/Source.ecore
module M;  create OUT : MMcible from IN : MMsource;
helper context MMsource!Classe def: nom : Type = <expression OCL>;
helper context MMsource!Classe def: op(p : Type) : Type = <expression OCL>;
helper def: global : Type = <expression>;          -- appel : thisModule.global
rule R {
    from s : MMsource!ClasseSource (<filtre booléen>)
    to   t : MMcible!ClasseCible ( attr <- <expression sur s>,
                                   ref  <- <éléments source, résolus par la trace> ),
         u : MMcible!Autre ( ... )                 -- élément cible supplémentaire
}
lazy rule L { from s : ... to t : ... }            -- appel : thisModule.L(e)
rule P(x : T) { to t : ... do { ... } }            -- called rule
```

B.5 Glossaire

Terme	Définition
AQL	*Acceleo Query Language*, langage d'expressions d'Acceleo 4, proche d'OCL.
ATL	*ATLAS Transformation Language*, langage de transformation M2M hybride d'Eclipse.
CIM / PIM / PSM	modèle indépendant de l'informatique / de la plateforme / spécifique à une plateforme.
Composition	référence `containment` : l'objet contenu n'existe que dans son conteneur.
Conformité	un modèle respecte la structure, les multiplicités et toutes les contraintes de son méta-modèle.
DSL	*Domain-Specific Language*, langage dédié à un domaine.
Ecore	méta-méta-modèle d'EMF, implantation d'EMOF.
EMF	*Eclipse Modeling Framework* : Ecore, XMI, génération de code, API réflexive.
Endogène / exogène	transformation à méta-modèle identique / différent entre source et cible.
Helper	fonction OCL d'un module ATL, attachée ou non à un contexte.
Horizontale / verticale	transformation qui conserve / change le niveau d'abstraction.
IDM / MDE	Ingénierie Dirigée par les Modèles / *Model-Driven Engineering*.
M2M / M2T / T2M	modèle vers modèle / modèle vers texte / texte vers modèle.
MDA	*Model Driven Architecture*, démarche IDM de l'OMG (2001).
Méta-modèle	modèle qui définit un langage de modélisation (niveau M2).
MOF	*Meta Object Facility*, méta-méta-modèle de l'OMG (niveau M3), EMOF et CMOF.

Terme	Définition
MOFM2T	standard OMG de transformation modèle vers texte, implanté par Acceleo.
OCL	*Object Constraint Language*, langage de contraintes et de requêtes de l'OMG.
PDM	*Platform Description Model*, description d'une plateforme et référence des règles de projection.
Projection	transformation PIM → PSM.
QVT	*Query/View/Transformation*, standard OMG de transformation M2M.
Raffinement	transformation qui précise un modèle (PIM → PIM, PSM → PSM).
Rétro-ingénierie	transformation vers un niveau plus abstrait (Code → PSM → PIM).
Template	texte à trous rempli avec les données d'un modèle.
Trace	lien source → cible mémorisé par le moteur de transformation.
XMI	*XML Metadata Interchange*, format XML d'échange des modèles.
Zone protégée	partie d'un fichier généré conservée lors des régénérations.

Références bibliographiques

Normes et spécifications

[1] Object Management Group. *MDA Guide*, version 1.0.1, omg/03-06-01, 2003 ; *MDA Guide revision 2.0*, ormsc/14-06-01, 2014.

[2] Object Management Group. *Meta Object Facility (MOF) Core Specification*, version 2.5.1, 2016.

[3] Object Management Group. *Unified Modeling Language (UML)*, version 2.5.1, 2017.

[4] Object Management Group. *Object Constraint Language (OCL)*, version 2.4, 2014.

[5] Object Management Group. *XML Metadata Interchange (XMI) Specification*, version 2.5.1, 2015.

[6] Object Management Group. *Meta Object Facility (MOF) Query/View/Transformation (QVT)*, version 1.3, 2016.

[7] Object Management Group. *MOF Model to Text Transformation Language (MOFM2T)*, version 1.0, 2008.

[8] W3C. *XSL Transformations (XSLT) Version 1.0*, Recommandation, 1999.

Ouvrages et articles

[9] M. Brambilla, J. Cabot, M. Wimmer. *Model-Driven Software Engineering in Practice*, 2e éd., Morgan & Claypool, 2017.

[10] A. Kleppe, J. Warmer, W. Bast. *MDA Explained: The Model Driven Architecture – Practice and Promise*, Addison-Wesley, 2003.

[11] D. Steinberg, F. Budinsky, M. Paternostro, E. Merks. *EMF: Eclipse Modeling Framework*, 2e éd., Addison-Wesley, 2008.

[12] J. Warmer, A. Kleppe. *The Object Constraint Language: Getting Your Models Ready for MDA*, 2e éd., Addison-Wesley, 2003.

[13] D. C. Schmidt. « Model-Driven Engineering ». *IEEE Computer*, 39(2), p. 25–31, 2006.

[14] J. Bézivin. « On the unification power of models ». *Software and Systems Modeling*, 4(2), p. 171–188, 2005.

[15] K. Czarnecki, S. Helsen. « Feature-based survey of model transformation approaches ». *IBM Systems Journal*, 45(3), p. 621–645, 2006.

[16] T. Mens, P. Van Gorp. « A taxonomy of model transformation ». *Electronic Notes in Theoretical Computer Science*, 152, p. 125–142, 2006.

[17] F. Jouault, F. Allilaire, J. Bézivin, I. Kurtev. « ATL: A model transformation tool ». *Science of Computer Programming*, 72(1–2), p. 31–39, 2008.

[18] B. W. Boehm. *Software Engineering Economics*, Prentice Hall, 1981.

[19] B. Combemale. *Ingénierie Dirigée par les Modèles (IDM) – État de l'art*, rapport, IRIT, Toulouse, 2008.

[20] J.-M. Jézéquel, B. Combemale, D. Vojtisek. *Ingénierie Dirigée par les Modèles : des concepts à la pratique*, Ellipses, 2012.

Documentation en ligne

[21] Eclipse Foundation. *Eclipse Modeling Framework (EMF)*. https://eclipse.dev/modeling/emf/

[22] Eclipse Foundation / Obeo. *Acceleo – documentation et guide utilisateur*. https://eclipse.dev/acceleo/

[23] Eclipse Foundation. *ATL – ATL Transformation Language*, User Guide et exemples (dont Families2Persons). https://eclipse.dev/atl/

[24] Eclipse Foundation. *Eclipse OCL (OCLinEcore, Complete OCL)*. https://eclipse.dev/modeling/mdt/ocl/

[25] Eclipse Foundation / Obeo. *Sirius* et *Obeo Designer Community*. https://eclipse.dev/sirius/

[26] V. Aranega. *PyEcore : A Python(ic) implementation of EMF/Ecore*. https://github.com/pyecore/pyecore

Supports du module

[27] A. Berrahou. *Lecture 1 : Introduction à l'IDM et aux enjeux du génie logiciel moderne*, ENSIAS.

[28] A. Berrahou. *Lecture 2 : Model Driven Architecture (MDA)*, ENSIAS.

[29] A. Berrahou. *Lecture 3 : Transformation (génération de code) par template avec Acceleo*, ENSIAS.

[30] A. Berrahou. *Lecture 4 : Transformation (génération de code) par programmation*, ENSIAS.

[31] A. Berrahou. *Lecture 5 : Transformation par modèle (M2M) avec ATL*, ENSIAS.

[32] A. Berrahou. *Tutoriels et TP : méta-modélisation avec Obeo Designer et EMF ; Acceleo ; transformation par programmation ; ATL (Families2Persons)*, ENSIAS.

[33] E. Cariou. *Object Constraint Language (OCL)*, support de cours, Université de Pau et des Pays de l'Adour.